

방치형 월드 학습 가이드 문서

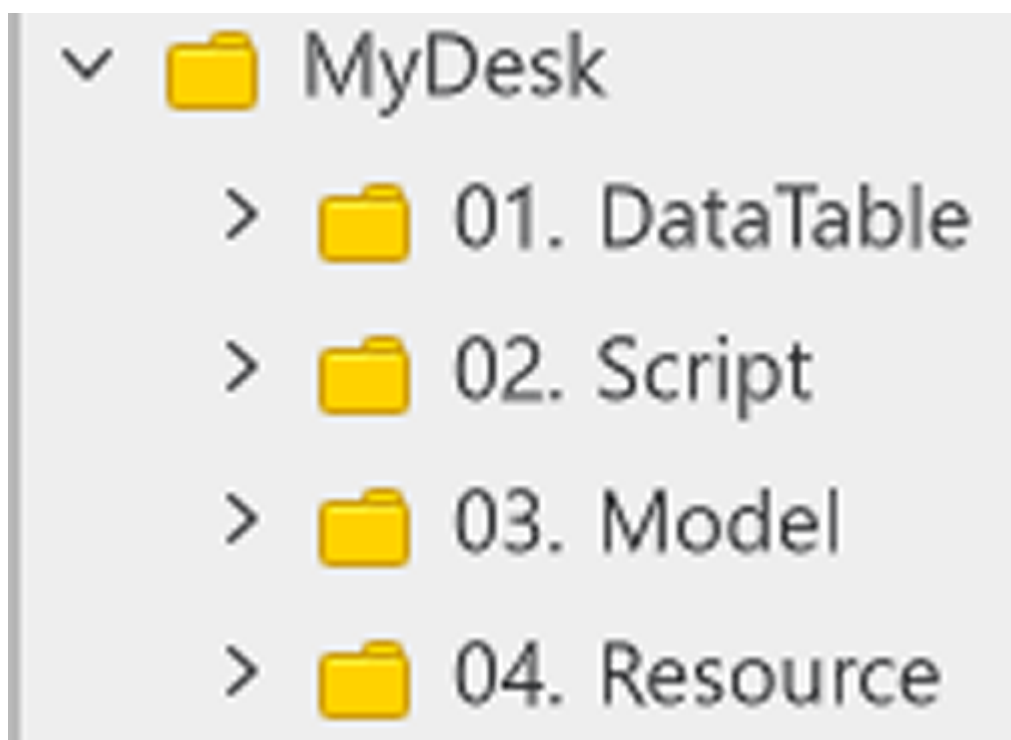
학습 목표

- 가이드 문서와 샘플 월드를 통하여 방치형 월드의 핵심 기능들을 구현하는 방법에 대해 학습합니다.
- TimerService를 활용하여 일정한 시간마다 반복적인 작업을 수행하는 방법을 학습합니다.
- 오브젝트 풀 기법을 제작하고 사용하며, 오브젝트 풀의 장단점을 학습합니다.
- 제작된 월드를 원하는 형태로 가공하고 제작할 수 있습니다.

학습 핵심 기능

- **TimerService의 이해**: 메이플스토리 월드에서 지정한 시간에 맞춰 기능을 수행하는 TimerService에 대해 학습합니다.
- **오브젝트 풀의 이해**: 게임의 최적화 기법인 오브젝트 풀에 대해 이해하고 어떻게 사용해야 하는지 학습합니다.

폴더 구성



```
<style type="text/css"> .tg {border-collapse:collapse;border-spacing:0;} .tg td{border-color:black;border-style:solid;border-width:1px;font-family:Arial, sans-serif;font-size:14px; overflow:hidden;padding:10px 5px;word-break:normal;} .tg th{border-color:black;border-style:solid;border-width:1px;font-family:Arial, sans-serif;font-size:14px; font-weight:normal;overflow:hidden;padding:10px 5px;word-break:normal;} .tg .tg-
```

```
lboi{border-color:currentColor;text-align:left;vertical-align:middle} .tg .tg-baqh{border-color:currentColor;text-align:center;vertical-align:top} .tg .tg-nrix{border-color:currentColor;text-align:center;vertical-align:middle} .tg .tg-xwyw{border-color:currentColor;text-align:center;vertical-align:middle} .tg .tg-0a7q{border-color:currentColor;text-align:left;vertical-align:middle} </style>
```

번호	이름	설명
1	DataTable	샘플 월드를 구성하기 위한 DataSet을 담은 폴더입니다.
2	Script	Component와 Logic과 같이 Script와 관련된 요소들을 담은 폴더입니다.
3	Model	캐릭터, 몬스터와 같이 미리 생성한 Model들을 관리하기 위한 폴더입니다.
4	Resource	월드를 제작할 때 사용할 리소스를 관리하기 위한 폴더입니다.

구현 순서

-  **기초 학습**: 메이플스토리 월드의 기초적인, 또는 해당 장르의 가장 기초적인 기능을 담은 학습 항목입니다.
-  **심화 학습**: 샘플 월드에 구현된 기능을 학습하고 구현하는 항목입니다. 해당 과정의 학습을 통해 장르의 기능 확장과 학습 이해도를 넓힐 수 있습니다.

[기초 학습] 맵 배치하기

- 메이플스토리 월드에서 맵을 교체하는 방법에 대해 학습합니다.
- RectTileMap을 활용하여 맵을 제작하는 방법에 대해 학습합니다.
- 엔티티의 계층 구조를 활용하여 엔티티들을 관리하는 방법에 대해 학습합니다.

[기초 학습] 캐릭터 제작하기

- 캐릭터를 구성하기 위한 DataSet을 제작하는 방법을 학습하고 제작합니다.
- 캐릭터의 정보를 사용하기 위한 Logic을 제작합니다.
- 캐릭터 Model을 제작하고 사용하는 방법에 대해 학습합니다.
- 캐릭터를 다른 슬롯으로 이동하도록 기능을 구현합니다.
- 테스트를 위한 소환 기능을 구현합니다.

[기초 학습] 몬스터 제작하기

- 몬스터의 정보를 관리하기 위한 DataSet을 제작하는 방법을 학습하고 제작합니다.
- Logic을 통해 몬스터의 정보를 불러올 수 있도록 기능을 구현합니다.
- Model을 활용하여 하나의 동일한 몬스터를 여러 마리로 소환할 수 있도록 기능을 구성합니다.
- 몬스터가 지정한 경로를 이동할 수 있도록 기능을 구현합니다.

[기초 학습] 캐릭터의 공격 구현하기

- 캐릭터가 일정 범위 내에 몬스터를 감지하여 공격 대상으로 등록하는 기능을 구현합니다.
- 캐릭터가 공격을 수행하여 몬스터에게 데미지를 줄 수 있도록 기능을 구현합니다.
- 캐릭터가 스킬을 사용할 수 있도록 기능을 구현합니다.
- 캐릭터의 공격을 일정한 주기로 수행할 수 있도록 기능을 구현합니다.

[💡 기초 학습] 몬스터 스폰하기

- 몬스터 스폰을 관리하는 DataSet을 제작합니다.
- 제작된 DataSet에 맞춰 몬스터가 스폰될 수 있도록 기능을 구현합니다.

[💡 기초 학습] 오브젝트 풀 생성하기

- 오브젝트 풀 기법이 무엇인지 작동 원리를 이해합니다.
- 오브젝트 풀 기법을 통해 얻는 이점과, 주의해야 하는 항목에 대해 학습합니다.

[💡 기초 학습] 소환 시스템 구현하기

- 플레이어가 캐릭터를 소환할 수 있도록 버튼과 기능을 구현합니다.
- 무한히 캐릭터를 소환하지 못하도록 재화와 소환을 검증하도록 기능을 변경합니다.
- 몬스터 처치시 재화를 지급하여 추가적인 소환이 가능하도록 기능을 변경합니다.

[📖 심화 학습] 저장 및 불러오기 구현하기

- 플레이어의 최고 기록을 저장하는 기능을 구현합니다.
- 플레이어의 최고 기록을 가져오는 기능을 구현합니다.
- DataStorageService를 활용하여 정보를 가져오는 방법에 대해 학습합니다.

[📖 심화 학습] 오프라인 보상 구현하기

- 플레이어의 마지막 접속 시간과 현재 시간을 비교하는 기능을 구현합니다.
- 지정한 계산식에 따라 재화를 지급하는 기능을 구현합니다.

[💡 기초 학습] UI 구성하기

- UI를 구성하는 방법에 대해 학습합니다.
- 플레이어가 유닛을 소환하기 위한 재화의 보유량을 확인하기 위한 UI를 구성합니다.
- 결과 UI를 제작하는 방법을 학습합니다.
- 현재 웨이브와 관련된 정보 UI를 제작하는 방법에 대해 학습합니다.
- 남은 시간을 표기하는 타이머 UI를 구현하는 방법을 학습합니다.

[💡 기초 학습] 맵 배치하기

영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 맵 타입 이해하기: 00:03:00

학습 목표

- 메이플스토리 월드에서 맵을 교체하는 방법에 대해 학습합니다.
- RectTileMap을 활용하여 맵을 제작하는 방법에 대해 학습합니다.
- 엔티티의 계층 구조를 활용하여 엔티티들을 관리하는 방법에 대해 학습합니다.

해당 영상 시청 후 수행해 볼 내용

- 게임이 진행되기 위한 맵을 구성할 수 있습니다.
- 원하는 TileSet을 활용하여 맵을 구성할 수 있습니다.
- Hierarchy의 계층 구조를 활용하여 엔티티를 관리할 수 있습니다.

맵 타입 이해하기

- 메이플스토리 월드에서 제공하는 맵의 타입은 총 3가지가 있습니다.
- 각 맵마다 맵의 발판이 되는 Tile을 배치하는 방식에 변화가 생깁니다.

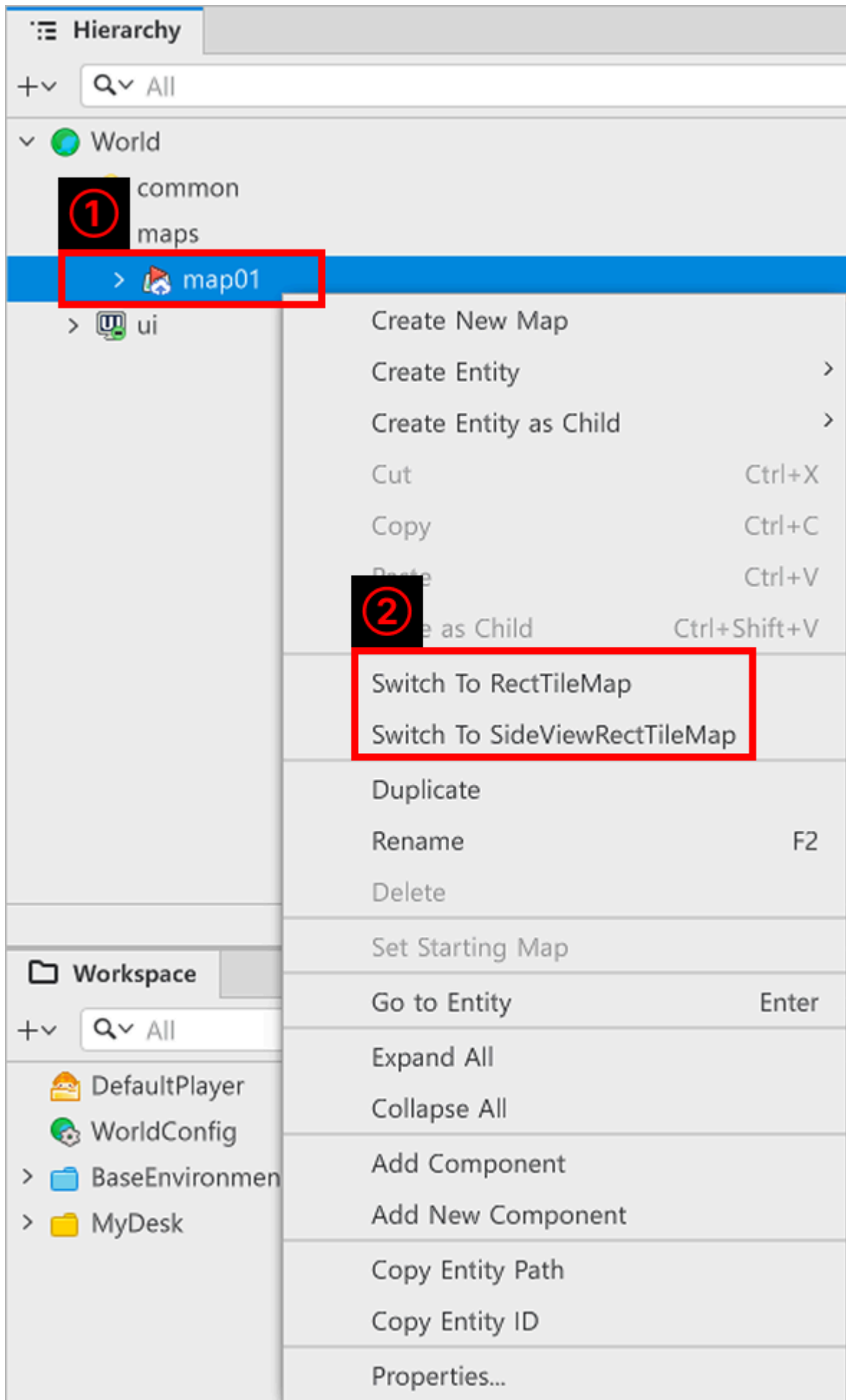
```
<style type="text/css"> .tg {border-collapse:collapse;border-spacing:0;} .tg td{border-color:black;border-style:solid;border-width:1px;font-family:Arial, sans-serif;font-size:14px; overflow:hidden;padding:10px 5px;word-break:normal;} .tg th{border-color:black;border-style:solid;border-width:1px;font-family:Arial, sans-serif;font-size:14px; font-weight:normal;overflow:hidden;padding:10px 5px;word-break:normal;} .tg .tg-lboi{border-color:currentColor;text-align:left;vertical-align:middle} .tg .tg-baqh{border-color:currentColor;text-align:center;vertical-align:middle} .tg .tg-nrix{border-color:currentColor;text-align:center;vertical-align:middle} .tg .tg-xwyw{border-color:currentColor;text-align:center;vertical-align:middle} .tg .tg-0a7q{border-color:currentColor;text-align:left;vertical-align:middle} </style>
```

번호	이름	설명
1	TileMap	메이플스토리과 같은 2D 횡스크롤 타입의 맵을 제작할 때 사용합니다. MapleTile을 통해 경사로, 벽 등을 제작할 수 있습니다.

번호	이름	설명
2	RectTileMap	탭다운 뷰 형태의 맵을 제작할 때 사용합니다. 바둑판 형태의 Tile을 배치하여 맵을 구성할 수 있습니다.
3	SideViewRectTileMap	2D 횡스크롤 타입의 맵을 제작할 때 사용합니다. RectTileMap과 같이 바둑판 형태의 Tile을 배치하여 2D 횡스크롤 타입의 맵을 구성할 수 있습니다.

- 샘플 월드의 경우 탭다운 뷰 형식의 맵을 사용하므로 RectTileMap 방식을 사용합니다.

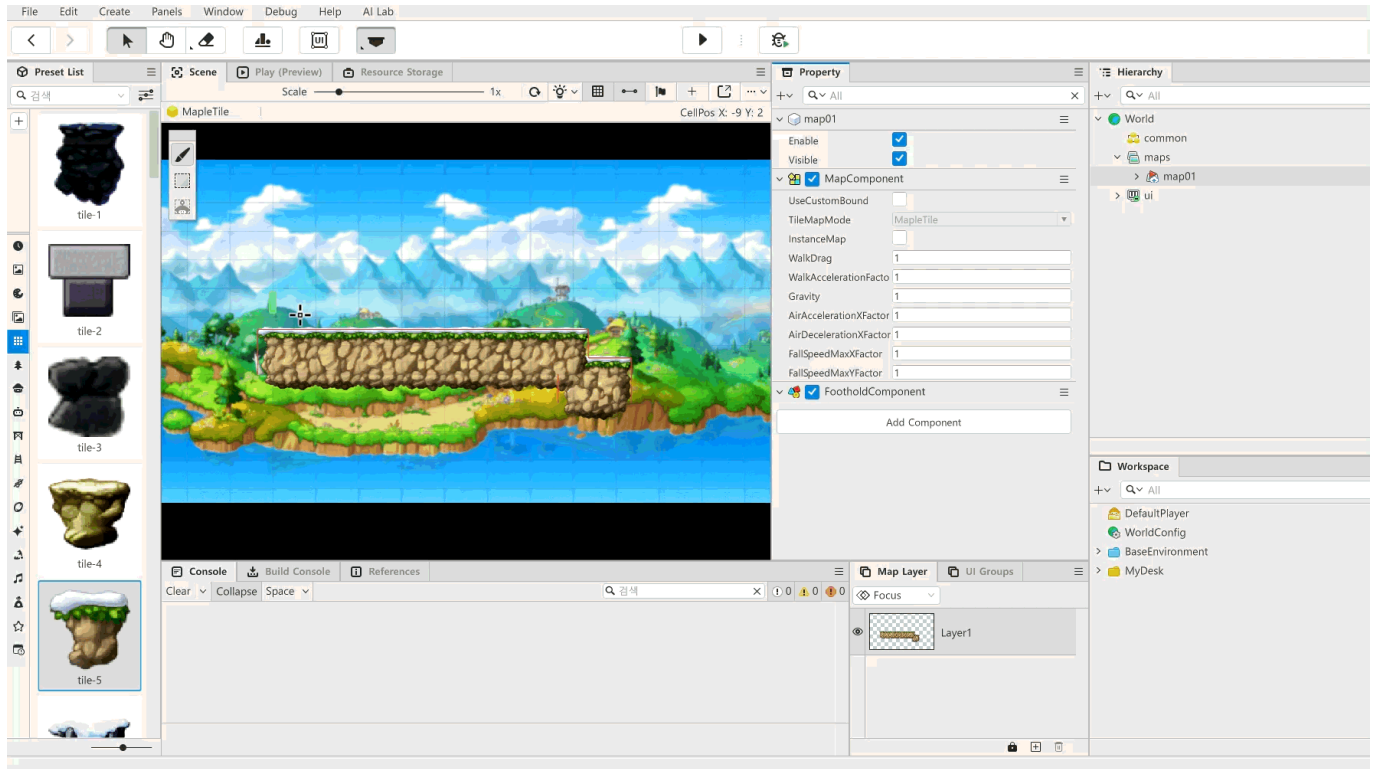
맵 전환하기



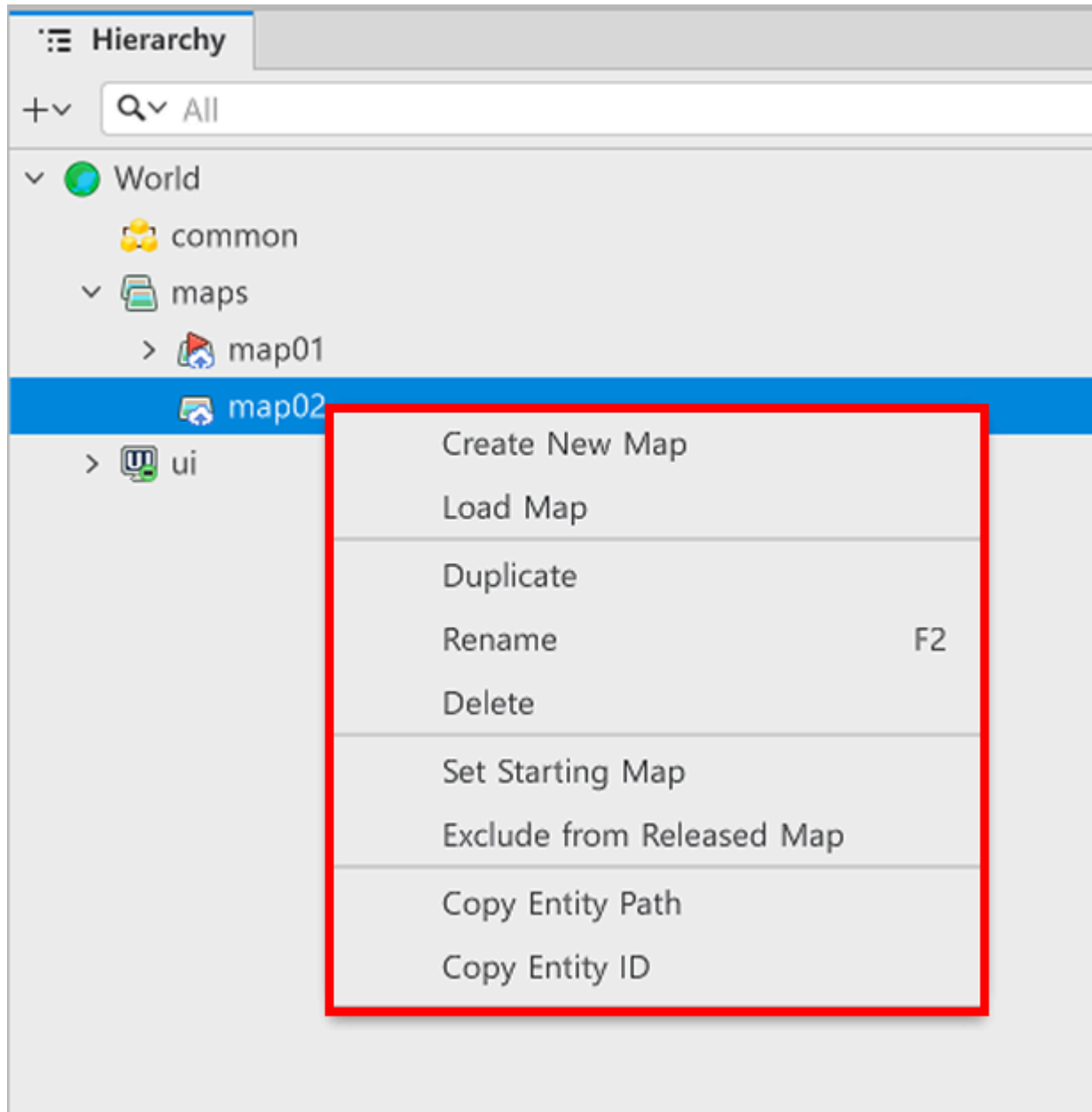
- **Hierarchy** 창에서 편집 중인 맵을 마우스 우클릭합니다.

- **Switch To** 메뉴 버튼을 통해 원하는 맵으로 전환할 수 있습니다.
 - **Switch To TileMap**: 현재 편집 중인 맵의 타입을 TileMap 방식으로 변경합니다.
 - **Switch To RectTileMap**: 현재 편집 중인 맵의 타입을 RectTileMap 방식으로 변경합니다.
 - **Switch To SideViewRectTileMap**: 현재 편집 중인 맵의 타입을 SideViewRectTileMap 방식으로 변경합니다.

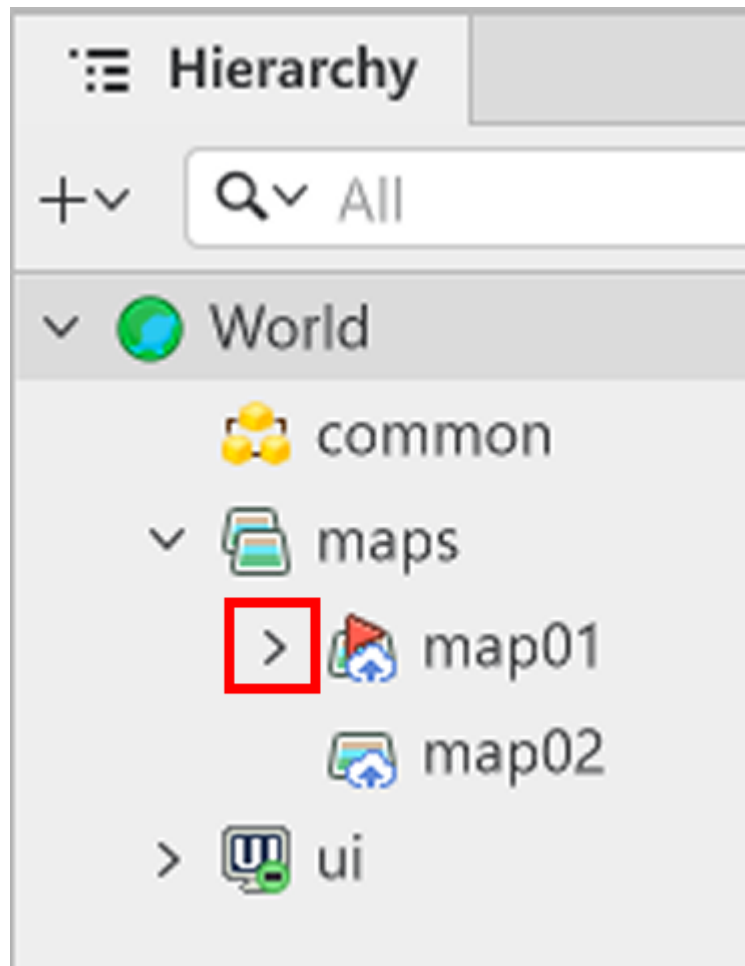
⚠ 맵 전환시 주의사항



- 맵 타입을 전환할 경우 기존에 배치된 Tile의 정보는 모두 초기화됩니다.

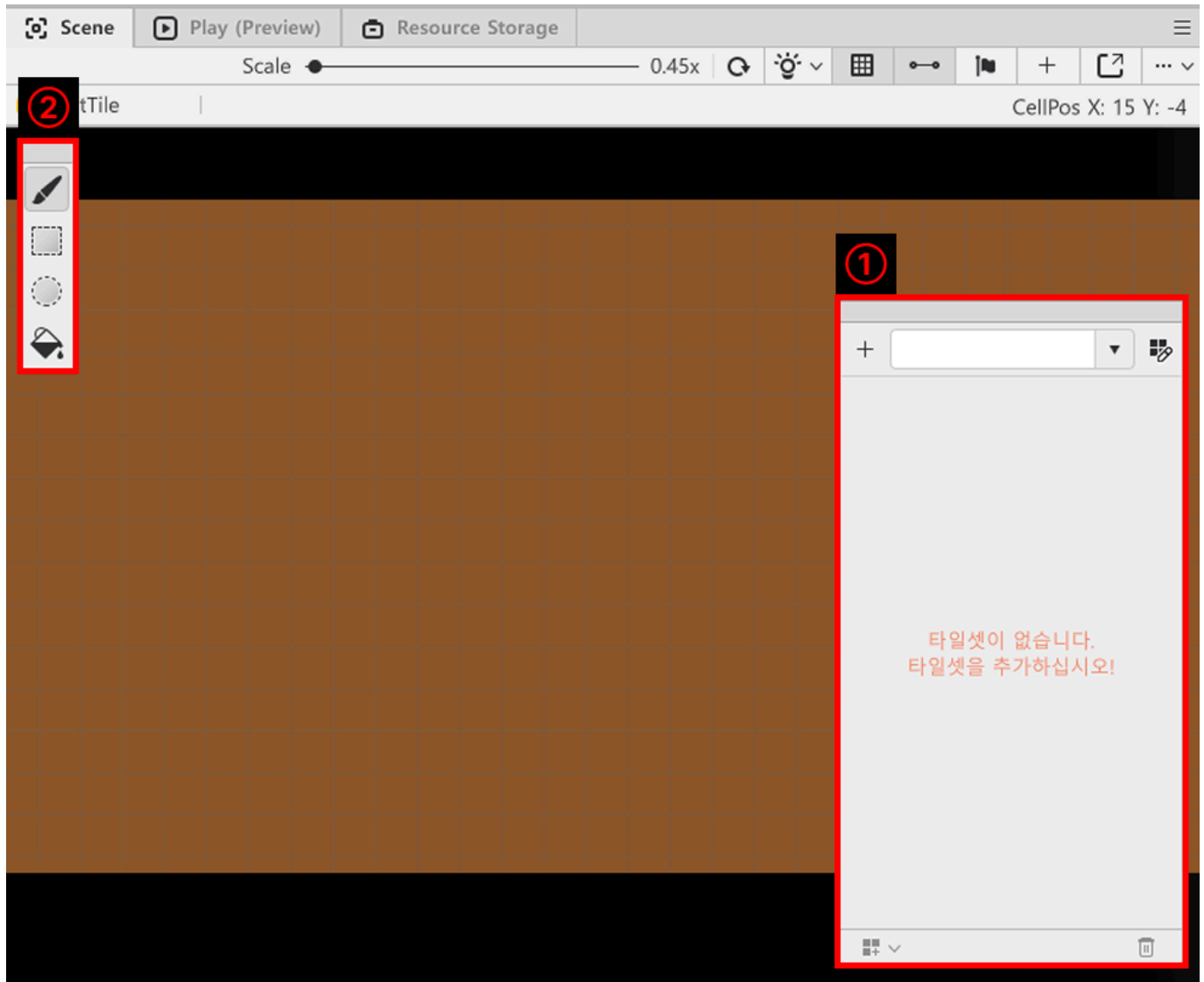


- 맵 타입은 현재 편집중인 맵만 변경할 수 있습니다.



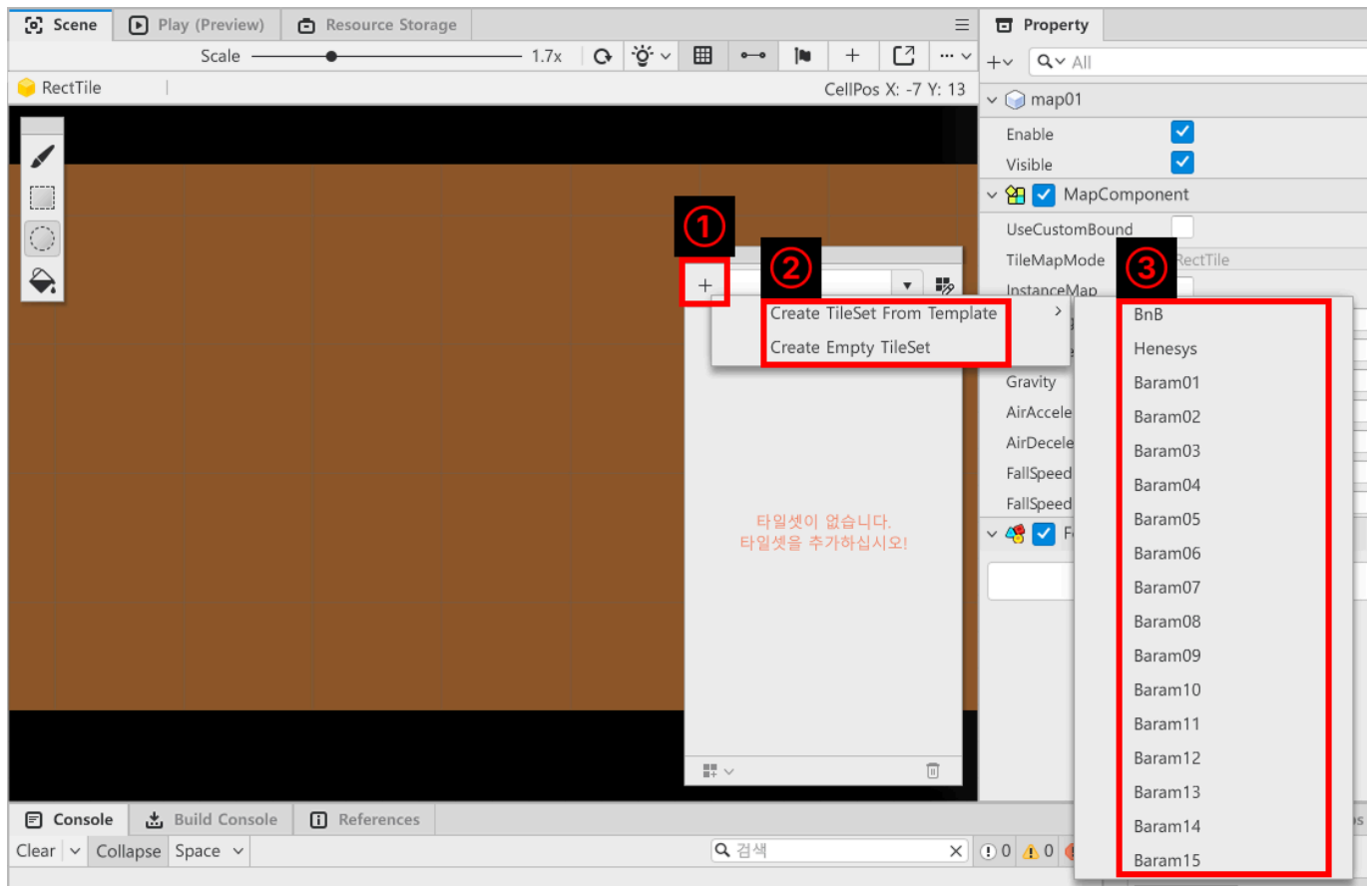
- 현재 편집중인 맵은 > 표시가 존재합니다.
- 해당 표시가 있는 맵의 형식을 변경할 수 있습니다.

RectTileMap 구성 살펴보기



1. TileSet을 관리하거나 Tile의 성질을 변경하는 등, Tile과 관련된 편집을 위한 에디터입니다.
2. 선택한 Tile을 Scene에 배치하기 위한 툴입니다.
 - 브러시: 붓으로 그림을 그리듯 타일을 배치할 수 있습니다. 클릭으로 점처럼 찍거나 드래그로 연속적으로 Tile을 배치할 수 있습니다.
 - 사각형: 마우스 드래그로 사각형 형태의 범위를 지정하면 지정된 범위에 선택한 타일을 배치합니다.
 - 원형: 마우스 드래그로 원 형태의 범위를 지정하면 지정된 범위에 선택한 타일을 배치합니다.

TileSet 제작하기

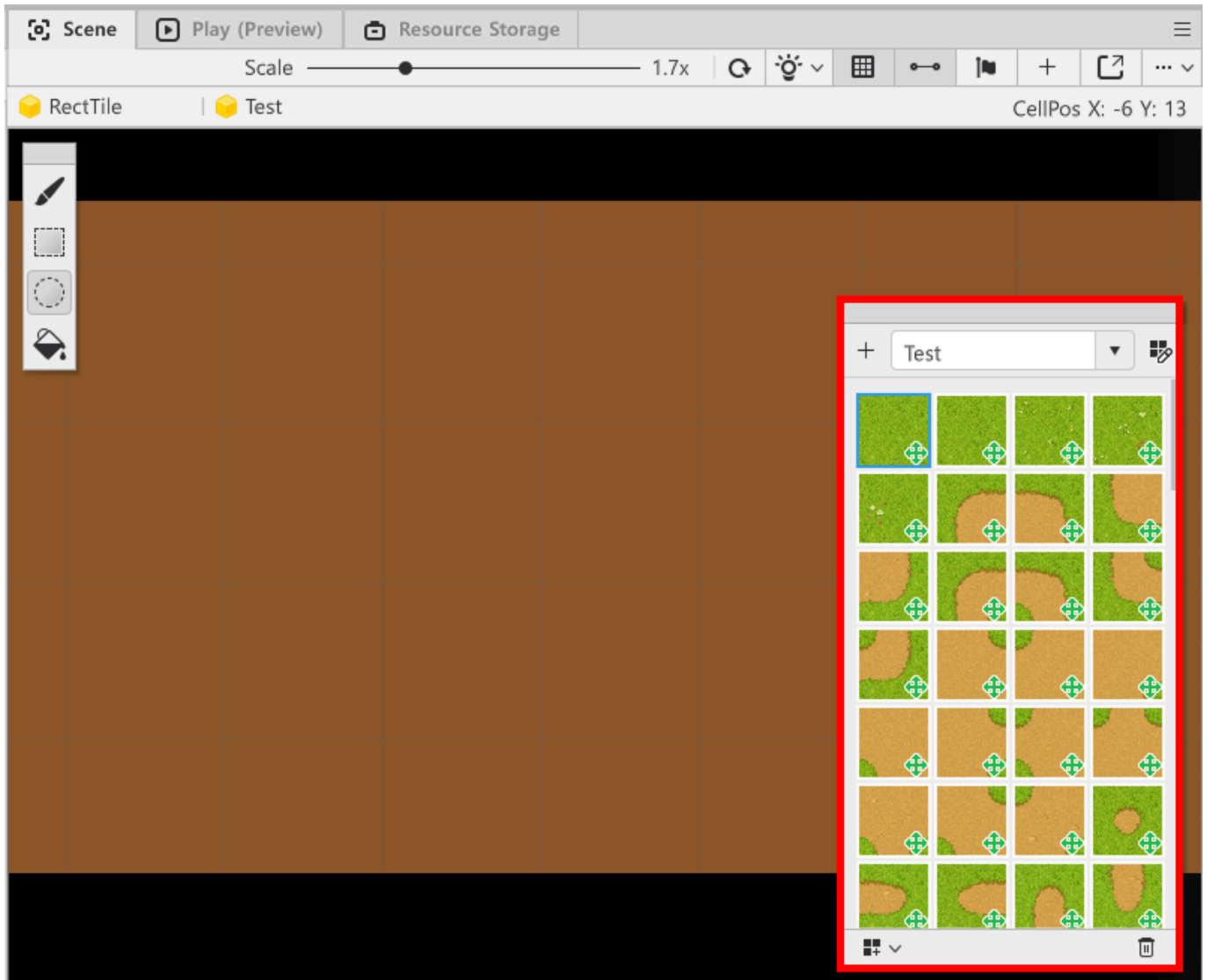


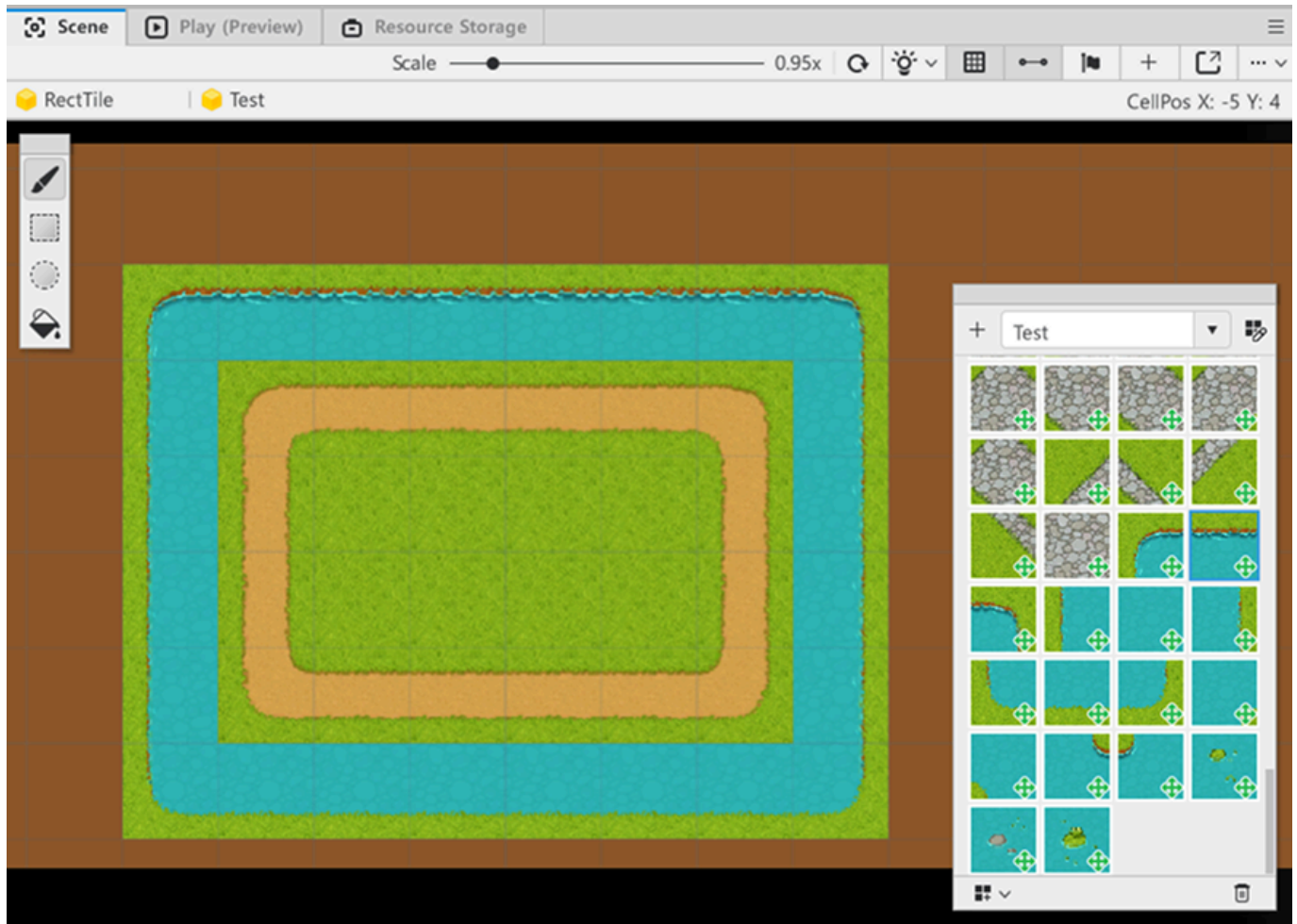
1. TileSet 에디터에서 + 버튼을 클릭합니다.
2. Create 메뉴에서 `Create TileSet From Template`을 클릭하여 메이플스토리 월드가 제공하는 Tile 리소스를 사용합니다.
 - 만약 자신만의 TileSet을 제작하고 싶을 경우 Create Empty TileSet을 통해 빈 TileSet을 생성할 수 있습니다.
3. 원하는 리소스를 선택하여 TileSet을 제작합니다.

💡 TileSet이란?



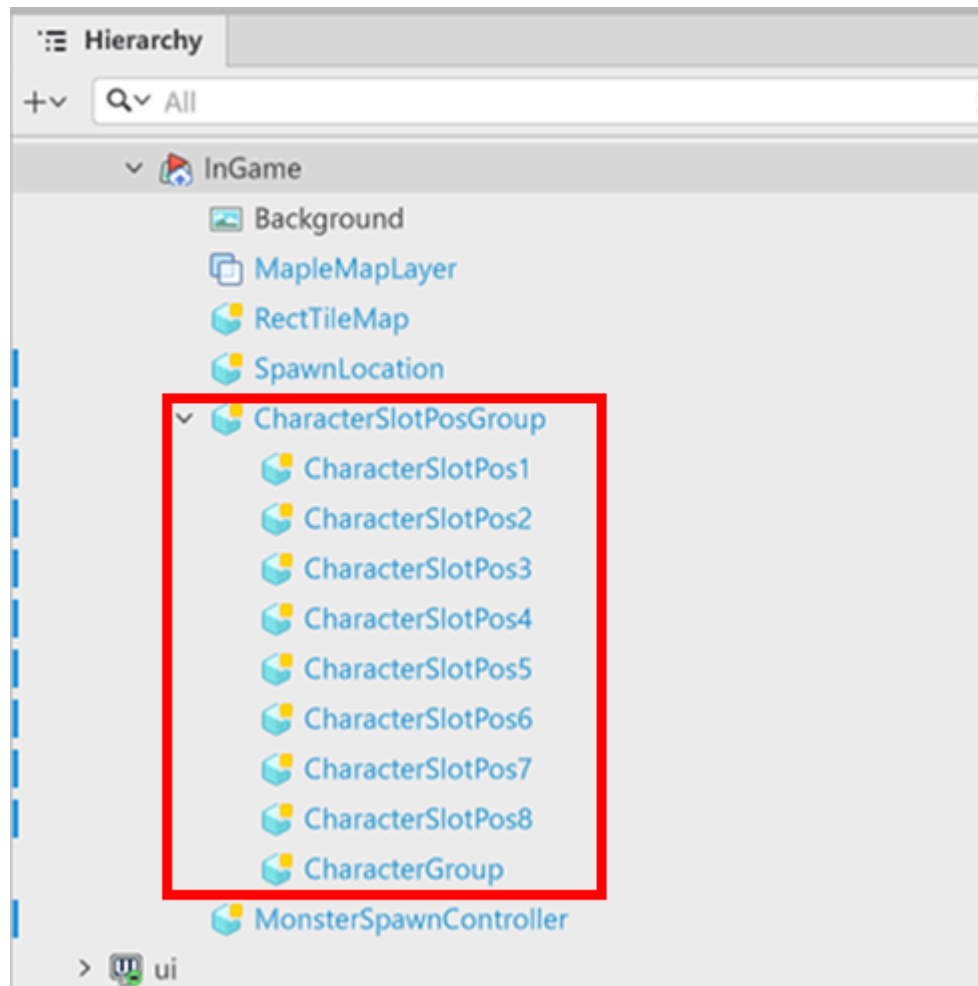
- **TileSet**은 **Tile**들을 모은 팔레트의 개념입니다.
- 그림을 그릴 때 팔레트에 물감을 짰 뒤 짜 둔 물감을 사용합니다.
- 다양한 종류의 물감인 **Tile**을 하나의 **TileSet**이라는 팔레트에 물감을 짜 둡니다.
- 개발자는 **TileSet**이라는 팔레트에 넣어 둔 **Tile**을 찍어 **Scene**에 칠하기만 하면 됩니다.





- 다양한 Tile들을 Scene 화면에 배치하여 맵을 형성할 수 있습니다.

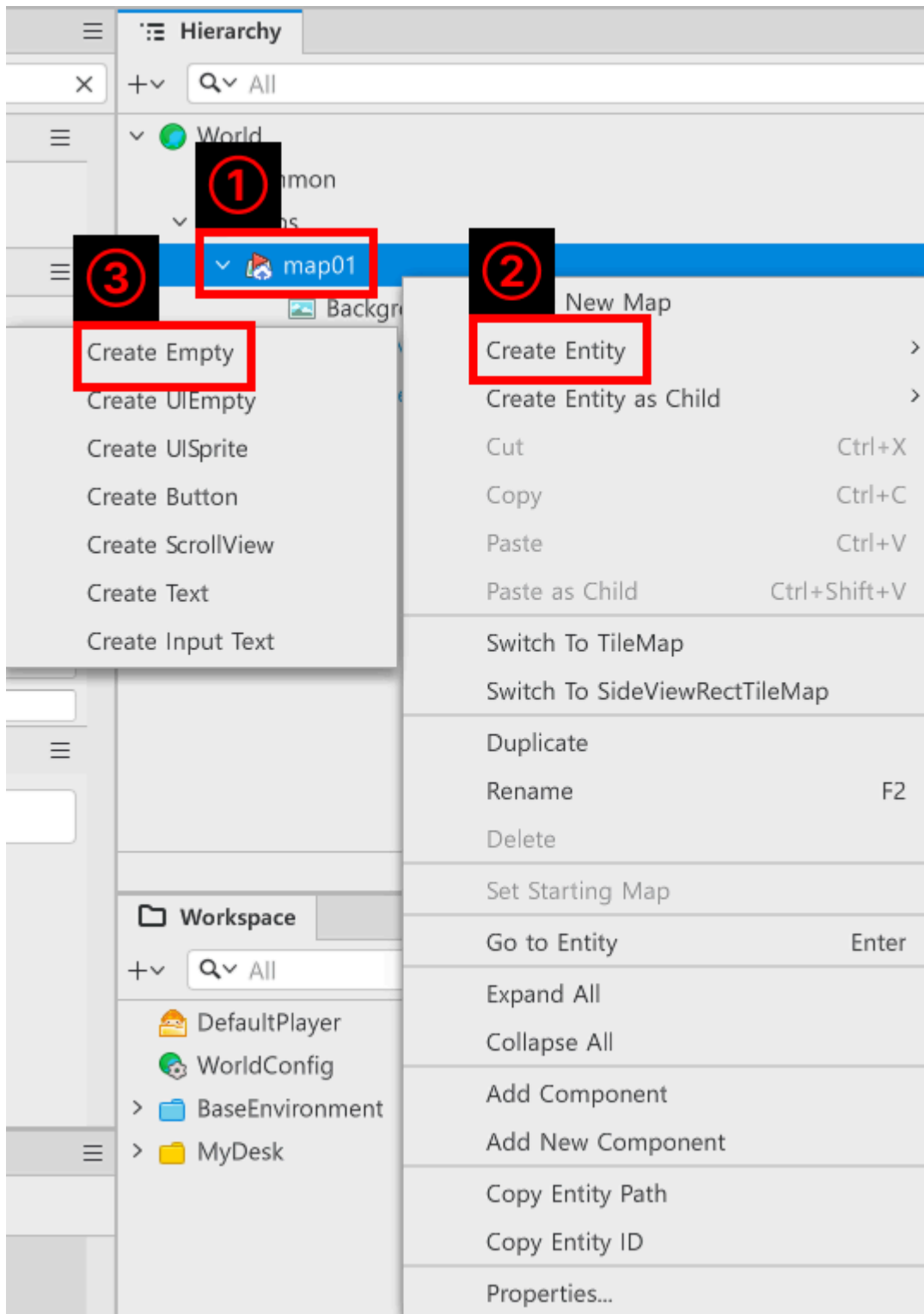
슬롯 배치하기



- 캐릭터의 배치 위치를 지정하고, 사용하기 위한 슬롯을 제작하기 위해 엔티티를 생성해야 합니다.
- 다수의 엔티티를 하나의 맵에 정렬하면 관리에 어려움이 발생합니다.
- 이런 문제를 해결하기 위해 빈 엔티티의 자식으로 관리하는 방법이 있습니다.

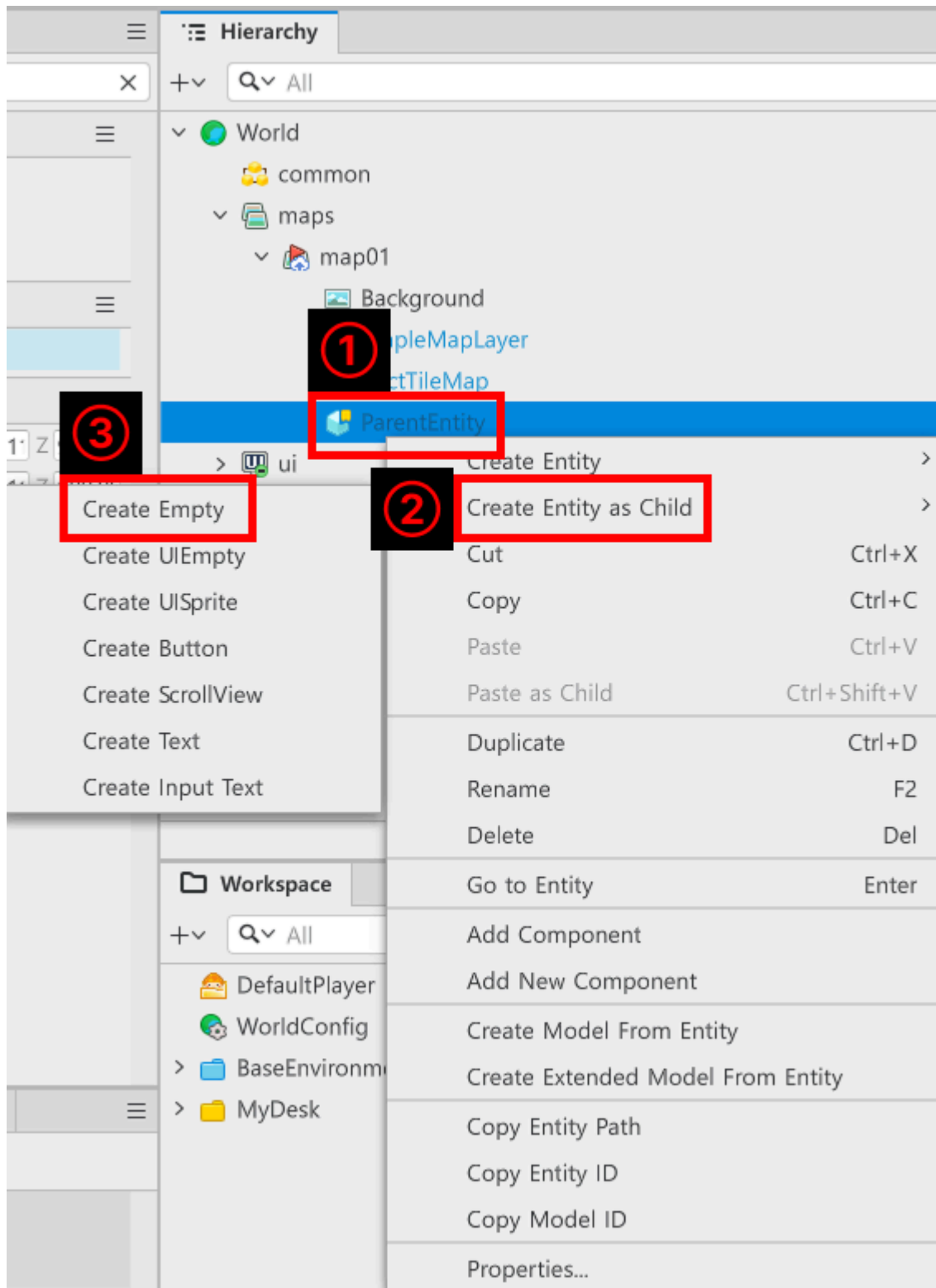
엔티티 생성하기

- 엔티티의 생성 방법은 크게 2가지가 있습니다.
 - 일반적인 엔티티의 생성: 현재 편집중인 맵의 자식으로 엔티티를 생성할 수 있습니다.
 - 자식 엔티티로 생성: 현재 선택중인 엔티티의 자식으로 엔티티를 배치합니다.
- 필요에 따라 빈 엔티티를 생성한 뒤, 자식 엔티티를 빈 엔티티의 자식으로 설정하여 엔티티의 특성에 따라 구분할 수 있습니다.
 - 샘플 월드의 경우 CharacterSlotPosGroup이라는 엔티티에 자식으로 CharacterSlotPos를 설정하였습니다.
 - 이를 통해 CharacterSlotPosGroup 엔티티는 자신의 자식인 CharacterSlotPos들을 담아 포장지 역할을 할 수 있습니다.



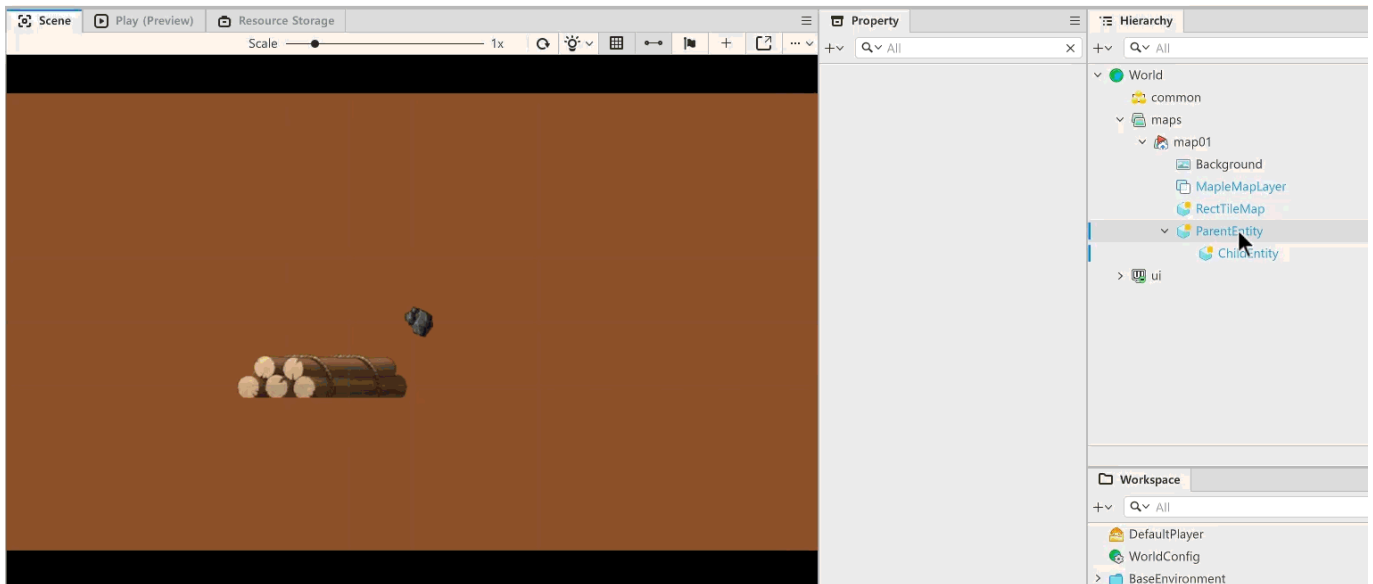
- **Hierarchy** 창에서 편집중인 맵을 마우스 우클릭합니다.
- **Create Entity**를 클릭하여 확장 메뉴를 엽니다.
- **Create Empty**를 클릭하여 빈 엔티티를 생성합니다.

자식으로 엔티티 생성하기



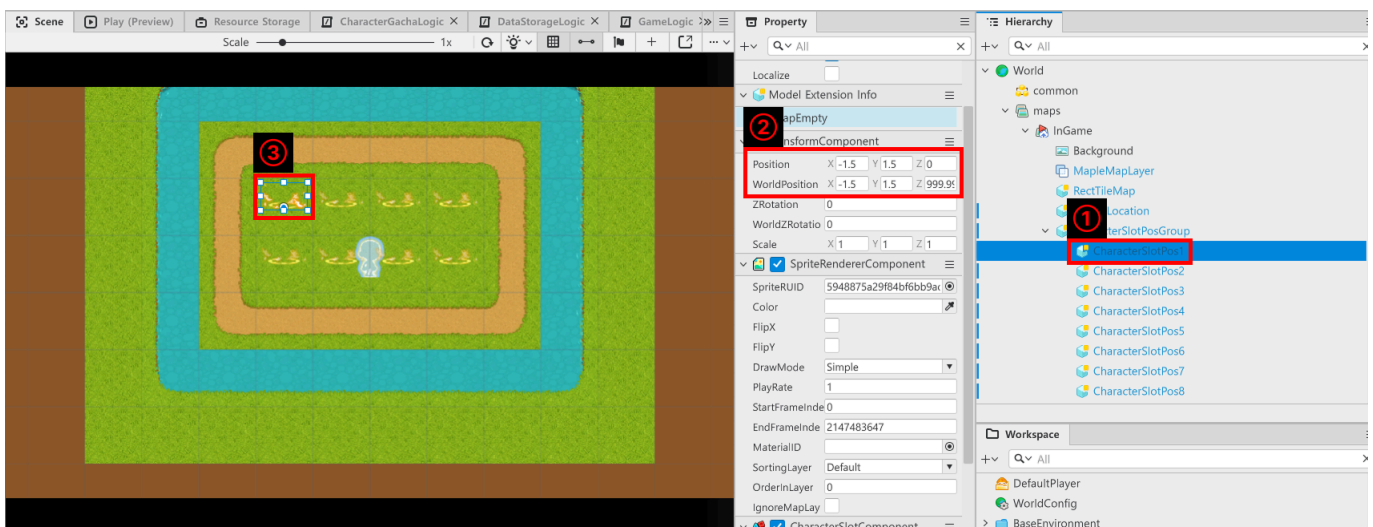
- 부모로 사용할 엔티티를 마우스 우클릭합니다.
- **Create Entity as Child**를 클릭하여 확장메뉴를 엽니다.
- **Create Empty**를 통해 빈 엔티티를 자식 엔티티로 생성할 수 있습니다.

⚠ 엔티티 그룹으로 담을 때 주의사항



- 자식 엔티티는 부모 엔티티의 위치에 영향을 받을 수 있습니다.
- 따라서 단순히 엔티티들을 관리하기 위한 용도라면 부모 엔티티의 좌표가 이동하지 않도록 주의해야 합니다.

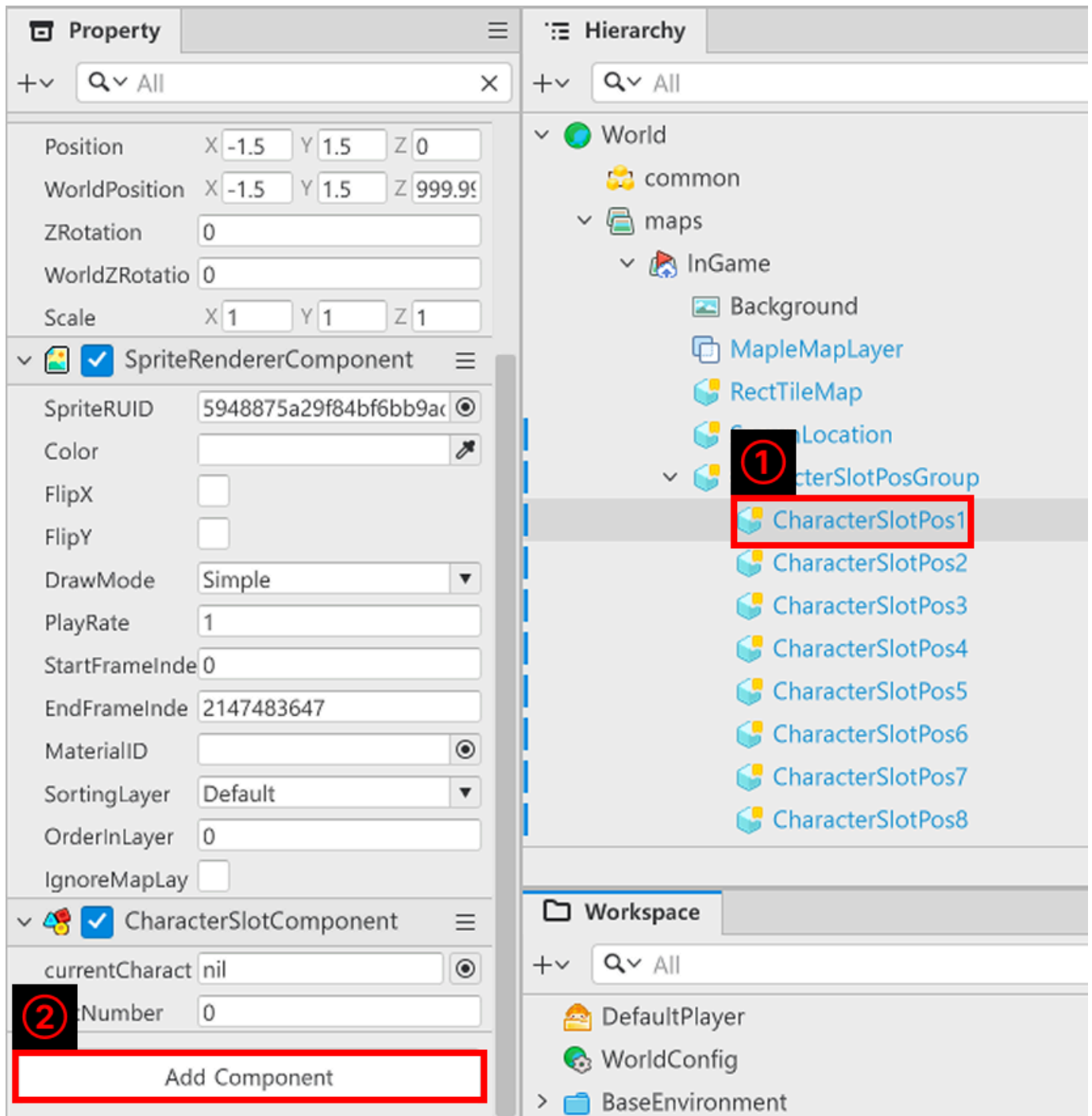
슬롯 엔티티 설정하기



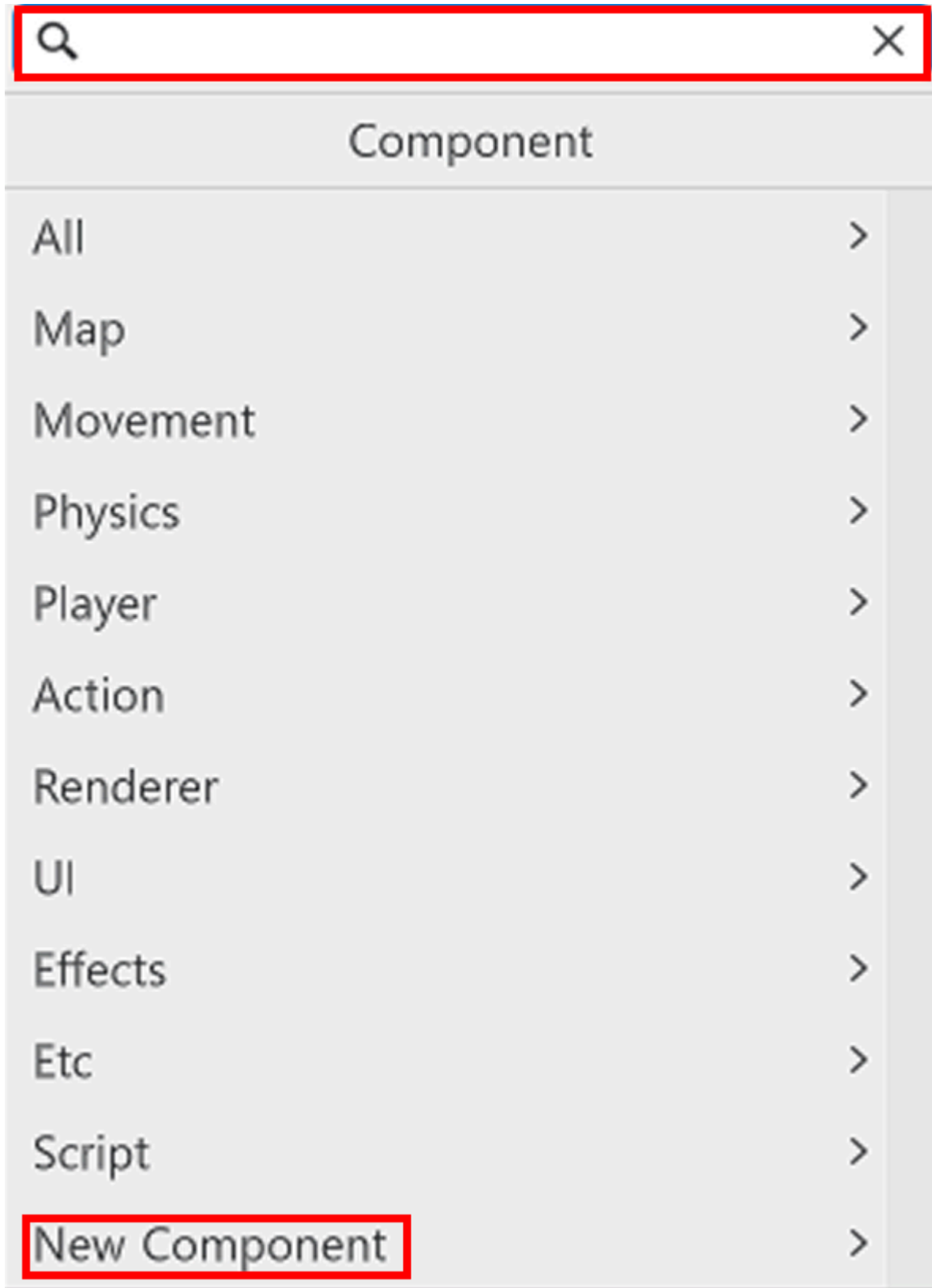
- 생성한 엔티티를 클릭합니다.
- **Property** 창에서 **TransformComponent**의 Position 또는 WorldPosition 값을 수정하여 캐릭터가 스폰될 위치를 지정합니다.
- 정상적인 위치에 배치되었는지 Scene 화면에서 체크합니다.

슬롯 엔티티에 Component 부여하기

- Component는 메이플스토리 월드에서 엔티티가 특정한 기능을 수행하도록 만들기 위한 요소입니다.
- Component를 통해 엔티티가 이미지를 출력, 데이터의 처리 등 다양한 기능들을 수행할 수 있습니다.



- **Hierarchy**에서 Component를 추가하려는 엔티티를 클릭합니다.
- **Property** 창의 가장 하단에 있는 **Add Component**를 클릭합니다.



- Component의 이름을 검색하여 Component를 추가할 수 있습니다.
- 혹은 새롭게 제작해야 할 경우 **New Component**를 통해 새 Component를 제작 및 추가할 수 있습니다.
- 샘플 월드에서는 CharacterSlotComponent로 제작하였습니다.
- CharacterSlotPos를 관리하기 위한 CharacterSlotPosGroup에는 CharacterSlotGroupComponent를 부여합니다.
- 아래는 CharacterSlotComponent, CharacterSlotPosGroup Component입니다.


```

@Component
script CharacterSlotComponent extends Component

property Entity currentCharacterEntity = nil

property number slotNumber = 0

@ExecSpace("ClientOnly")
method boolean CheckCanSpawnCharacter()
-- 만약 슬롯에 등록된 캐릭터가 없을 경우 캐릭터 스폰 가능 여부를 true로 반환합니다.
if not(self.currentCharacterEntity) then
    return true
else
-- 그렇지 않을 경우 false로 반환합니다.
    return false
end
end

@ExecSpace("ClientOnly")
method void InitCharacterEntity(Entity CharacterEntity)
-- currentCharacterEntity를 전달받은 엔티티로 등록합니다.
self.currentCharacterEntity = CharacterEntity
end

@ExecSpace("ClientOnly")
method void InitSlotNumber(number SlotNumber)
-- 게임 시작시 자기 자신에게 할당되는 번호를 등록합니다.
self.slotNumber = SlotNumber
end

@ExecSpace("ClientOnly")
method number GetSlotNumber()
-- 자기 자신에게 할당된 SlotNumber를 반환합니다.
return self.slotNumber
end

end

```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

```

@Component
script CharacterSlotGroupComponent extends Component

property table characterSlotList = {}

property Entity characterGroup = nil

@Description("현재 드래그 중인 엔티티가 있는지 확인하는 잠금 변수")
property boolean isDragging = false

```

```

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 게임이 시작되면 자식 슬롯 List를 찾습니다.
self:InitCharacterSlotList()
-- 게임이 시작되면 characterGroup을 찾습니다.
self:InitCharacterGroup()
end

@ExecSpace("ClientOnly")
method void InitCharacterSlotList()
-- characterSlotList가 비어있을 경우 초기화를 시도합니다.
if not invalid(self.characterSlotList) or not next(self.characterSlotList) then
-- 현재 엔티티의 자식들을 가져옵니다.
local childrenCount = self.Entity.Children.Count
-- 1번부터 childrenCount만큼 반복합니다.
for i = 1, childrenCount do
-- 자식 엔티티의 이름 형식을 지정합니다.
local entityName = "CharacterSlotPos"..i
-- 자기 자신의 자식 엔티티를 이름으로 찾습니다.
local getChildEntity = self.Entity:GetChildByName(entityName)
-- 정상적으로 값을 찾았다면 이를 등록합니다.
if invalid(getChildEntity) then
self.characterSlotList[i] = getChildEntity.CharacterSlotComponent
self.characterSlotList[i]:InitSlotNumber(i)
end
end
end
end

@ExecSpace("ClientOnly")
method void InitCharacterGroup()
-- characterGroup이 비어있는지 확인합니다.
if not invalid(self.characterGroup) then
-- 비어있을 경우 CharacterGroup을 찾아 등록합니다.
local findEntity = self.Entity:GetChildByName("CharacterGroup")
-- 정상적으로 찾았을 경우 CharacterGroup을 등록합니다.
self.characterGroup = findEntity
end
end

@ExecSpace("ClientOnly")
method number ReturnEmptySlotNumber()
-- characterSlotList의 수 만큼 탐색을 시작합니다.
for i = 1, #self.characterSlotList do
-- i번째 슬롯이 비어있는지 체크합니다.
if self.characterSlotList[i]:CheckCanSpawnCharacter() then
-- 만약 i번째 슬롯이 비어있을 경우, i를 반환합니다.
return i
end
end
-- 만약 빈 슬롯이 없을 경우 0을 반환합니다.
return 0
end

```

```

@ExecSpace("ClientOnly")
method void SpawnCharacter(number SlotNumber, CharacterData CharacterData)
-- 전달받은 Data를 기준으로 Model을 복제합니다.
local SpawnCharacter = _SpawnService:SpawnByModelId(CharacterData.ModelID,
CharacterData.CharacterID, Vector3.zero, self.characterGroup)
-- 스폰된 캐릭터의 위치를 전달받은 SlotNumber에 위치합니다.
SpawnCharacter.TransformComponent.WorldPosition =
self.characterSlotList[SlotNumber].Entity.TransformComponent.WorldPosition
-- 스폰된 캐릭터에게 캐릭터 정보를 등록하고 활성화시킵니다.
SpawnCharacter.UnitController:InitCharacterData(CharacterData,
self.characterSlotList[SlotNumber].Entity)
-- 전달받은 SlotNumber에 스폰된 캐릭터를 등록합니다.
self.characterSlotList[SlotNumber]:InitCharacterEntity(SpawnCharacter)
end

@ExecSpace("ClientOnly")
method void SwitchSlotCharacter(number TargetSlotNumber, Entity TargetEntity,
number BeforeSlotNumber)
if not isValid(self.characterSlotList[TargetSlotNumber].currentCharacterEntity)
then
-- 빈 슬롯일 경우 이동하려는 슬롯이 비어있는 상태이므로 이전을 수행합니다.

TargetEntity.UnitController:ChangeSlot(self.characterSlotList[TargetSlotNumber].En
tity)
-- 이동하려는 슬롯에 전달받은 엔티티를 등록합니다.
self.characterSlotList[TargetSlotNumber]:InitCharacterEntity(TargetEntity)
-- 기존의 슬롯을 nil 상태로 전환합니다.
self.characterSlotList[BeforeSlotNumber]:InitCharacterEntity(nil)
else
-- 슬롯에 이미 다른 엔티티가 있는 경우 타겟 슬롯에 원래 들어있던 기존 엔티티를 변수에 저장
해둡니다.
local existingEntity =
self.characterSlotList[TargetSlotNumber].currentCharacterEntity

-- 기존 엔티티를 비어있는 Before 슬롯 위치로 이동시킵니다.

existingEntity.UnitController:ChangeSlot(self.characterSlotList[BeforeSlotNumber].
Entity)

-- 드래그해 온 엔티티를 타겟 슬롯 위치로 이동시킵니다.

TargetEntity.UnitController:ChangeSlot(self.characterSlotList[TargetSlotNumber].En
tity)

-- 각 슬롯의 정보를 서로 교환합니다.
self.characterSlotList[BeforeSlotNumber]:InitCharacterEntity(existingEntity)
self.characterSlotList[TargetSlotNumber]:InitCharacterEntity(TargetEntity)
end
end

@ExecSpace("ClientOnly")
method void ClearCharacterList()
-- characterGroup이 유효한지 먼저 확인합니다.
if isValid(self.characterGroup) then

```

```

    local children = self.characterGroup.Children

    -- 자식 엔티티들을 뒤에서부터(역순으로) 순회하며 삭제합니다.
    for i = children.Count, 1, -1 do
        local childEntity = children[i]
        if isvalid(childEntity) then
            childEntity:Destroy()
        end
    end
end

-- characterSlotList에 남아있는 엔티티 참조 데이터를 nil로 초기화합니다.
for i = 1, #self.characterSlotList do
    local slot = self.characterSlotList[i]
    if isvalid(slot) then
        slot:InitCharacterEntity(nil)
    end
end
end
end
end

```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

BGM 출력하기

- 단순히 엔티티 뿐만 아니라 맵에도 Component를 부여할 수 있습니다.

-

@Component script CharacterSlotComponent extends Component

property Entity currentCharacterEntity = nil

property number slotNumber = 0

@ExecSpace("ClientOnly") method boolean CheckCanSpawnCharacter() -- 만약 슬롯에 등록된 캐릭터가 없을 경우 캐릭터 스폰 가능 여부를 true로 반환합니다. if not(self.currentCharacterEntity) then return true else -- 그렇지 않을 경우 false로 반환합니다. return false end end

@ExecSpace("ClientOnly") method void InitCharacterEntity(Entity CharacterEntity) -- currentCharacterEntity를 전달받은 엔티티로 등록합니다. self.currentCharacterEntity = CharacterEntity end

@ExecSpace("ClientOnly") method void InitSlotNumber(number SlotNumber) -- 게임 시작시 자기 자신에게 할당되는 번호를 등록합니다. self.slotNumber = SlotNumber end

@ExecSpace("ClientOnly") method number GetSlotNumber() -- 자기 자신에게 할당된 SlotNumber를 반환합니다. return self.slotNumber end

end

> 해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

```

```lua
@Component
script CharacterSlotGroupComponent extends Component

property table characterSlotList = {}

property Entity characterGroup = nil

@Description("현재 드래그 중인 엔티티가 있는지 확인하는 잠금 변수")
property boolean isDragging = false

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 게임이 시작되면 자식 슬롯 List를 찾습니다.
self:InitCharacterSlotList()
-- 게임이 시작되면 characterGroup을 찾습니다.
self:InitCharacterGroup()
end

@ExecSpace("ClientOnly")
method void InitCharacterSlotList()
-- characterSlotList가 비어있을 경우 초기화를 시도합니다.
if not isvalid(self.characterSlotList) or not next(self.characterSlotList) then
-- 현재 엔티티의 자식들을 가져옵니다.
local childrenCount = self.Entity.Children.Count
-- 1번부터 childrenCount만큼 반복합니다.
for i = 1, childrenCount do
-- 자식 엔티티의 이름 형식을 지정합니다.
local entityName = "CharacterSlotPos"..i
-- 자기 자신의 자식 엔티티를 이름으로 찾습니다.
local getChildEntity = self.Entity:GetChildByName(entityName)
-- 정상적으로 값을 찾았다면 이를 등록합니다.
if isvalid(getChildEntity) then
self.characterSlotList[i] = getChildEntity.CharacterSlotComponent
self.characterSlotList[i]:InitSlotNumber(i)
end
end
end

end

end

@ExecSpace("ClientOnly")
method void InitCharacterGroup()
-- characterGroup이 비어있는지 확인합니다.
if not isvalid(self.characterGroup) then
-- 비어있을 경우 CharacterGroup을 찾아 등록합니다.
local findEntity = self.Entity:GetChildByName("CharacterGroup")
-- 정상적으로 찾았을 경우 CharacterGroup을 등록합니다.
self.characterGroup = findEntity

```

```

end
end

@ExecSpace("ClientOnly")
method number ReturnEmptySlotNumber()
-- characterSlotList의 수 만큼 탐색을 시작합니다.
for i = 1, #self.characterSlotList do
-- i번째 슬롯이 비어있는지 체크합니다.
if self.characterSlotList[i]:CheckCanSpawnCharacter() then
-- 만약 i번째 슬롯이 비어있을 경우, i를 반환합니다.
return i
end
end
-- 만약 빈 슬롯이 없을 경우 0을 반환합니다.
return 0
end

@ExecSpace("ClientOnly")
method void SpawnCharacter(number SlotNumber, CharacterData CharacterData)
-- 전달받은 Data를 기준으로 Model을 복제합니다.
local SpawnCharacter = _SpawnService:SpawnByModelId(CharacterData.ModelID,
CharacterData.CharacterID, Vector3.zero, self.characterGroup)
-- 스폰된 캐릭터의 위치를 전달받은 SlotNumber에 위치합니다.
SpawnCharacter.TransformComponent.WorldPosition =
self.characterSlotList[SlotNumber].Entity.TransformComponent.WorldPosition
-- 스폰된 캐릭터에게 캐릭터 정보를 등록하고 활성화시킵니다.
SpawnCharacter.UnitController:InitCharacterData(CharacterData,
self.characterSlotList[SlotNumber].Entity)
-- 전달받은 SlotNumber에 스폰된 캐릭터를 등록합니다.
self.characterSlotList[SlotNumber]:InitCharacterEntity(SpawnCharacter)
end

@ExecSpace("ClientOnly")
method void SwitchSlotCharacter(number TargetSlotNumber, Entity TargetEntity,
number BeforeSlotNumber)
if not isValid(self.characterSlotList[TargetSlotNumber].currentCharacterEntity)
then
-- 빈 슬롯일 경우 이동하려는 슬롯이 비어있는 상태이므로 이전을 수행합니다.

TargetEntity.UnitController:ChangeSlot(self.characterSlotList[TargetSlotNumber].En
tity)
-- 이동하려는 슬롯에 전달받은 엔티티를 등록합니다.
self.characterSlotList[TargetSlotNumber]:InitCharacterEntity(TargetEntity)
-- 기존의 슬롯을 nil 상태로 전환합니다.
self.characterSlotList[BeforeSlotNumber]:InitCharacterEntity(nil)
else
-- 슬롯에 이미 다른 엔티티가 있는 경우 타겟 슬롯에 원래 들어있던 기존 엔티티를 변수에 저장
해둡니다.
local existingEntity =
self.characterSlotList[TargetSlotNumber].currentCharacterEntity

-- 기존 엔티티를 비어있는 Before 슬롯 위치로 이동시킵니다.

existingEntity.UnitController:ChangeSlot(self.characterSlotList[BeforeSlotNumber].

```

```
Entity)
```

```
-- 드래그해 온 엔티티를 타겟 슬롯 위치로 이동시킵니다.
```

```
TargetEntity.UnitController:ChangeSlot(self.characterSlotList[TargetSlotNumber].Entity)
```

```
-- 각 슬롯의 정보를 서로 교환합니다.
```

```
self.characterSlotList[BeforeSlotNumber]:InitCharacterEntity(existingEntity)
```

```
self.characterSlotList[TargetSlotNumber]:InitCharacterEntity(TargetEntity)
```

```
end
```

```
end
```

```
@ExecSpace("ClientOnly")
```

```
method void ClearCharacterList()
```

```
-- characterGroup이 유효한지 먼저 확인합니다.
```

```
if isvalid(self.characterGroup) then
```

```
 local children = self.characterGroup.Children
```

```
-- 자식 엔티티들을 뒤에서부터(역순으로) 순회하며 삭제합니다.
```

```
for i = children.Count, 1, -1 do
```

```
 local childEntity = children[i]
```

```
 if isvalid(childEntity) then
```

```
 childEntity:Destroy()
```

```
 end
```

```
end
```

```
end
```

```
-- characterSlotList에 남아있는 엔티티 참조 데이터를 nil로 초기화합니다.
```

```
for i = 1, #self.characterSlotList do
```

```
 local slot = self.characterSlotList[i]
```

```
 if isvalid(slot) then
```

```
 slot:InitCharacterEntity(nil)
```

```
 end
```

```
end
```

```
end
```

```
end
```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

## BGM 출력하기

- 단순히 엔티티 뿐만 아니라 맵에도 Component를 부여할 수 있습니다.
- **Hierarchy**에서 편집중인 맵에 **Add Component**를 통해 새 Component를 추가합니다.
- 해당 Component의 이름을 BGMPlayer로 작성합니다.
- 이어서 코드를 다음과 같이 작성합니다.

```
@Component
script BGMPayer extends Component

property string bgmID = ""

@ExecSpace("ClientOnly")
method void OnBeginPlay()
self:PlayBGM()
end

@ExecSpace("ClientOnly")
method void PlayBGM()
-- 등록된 bgmID를 기준으로 배경음을 재생합니다.
_SoundService:PlayBGM(self.bgmID, 1.0)
end

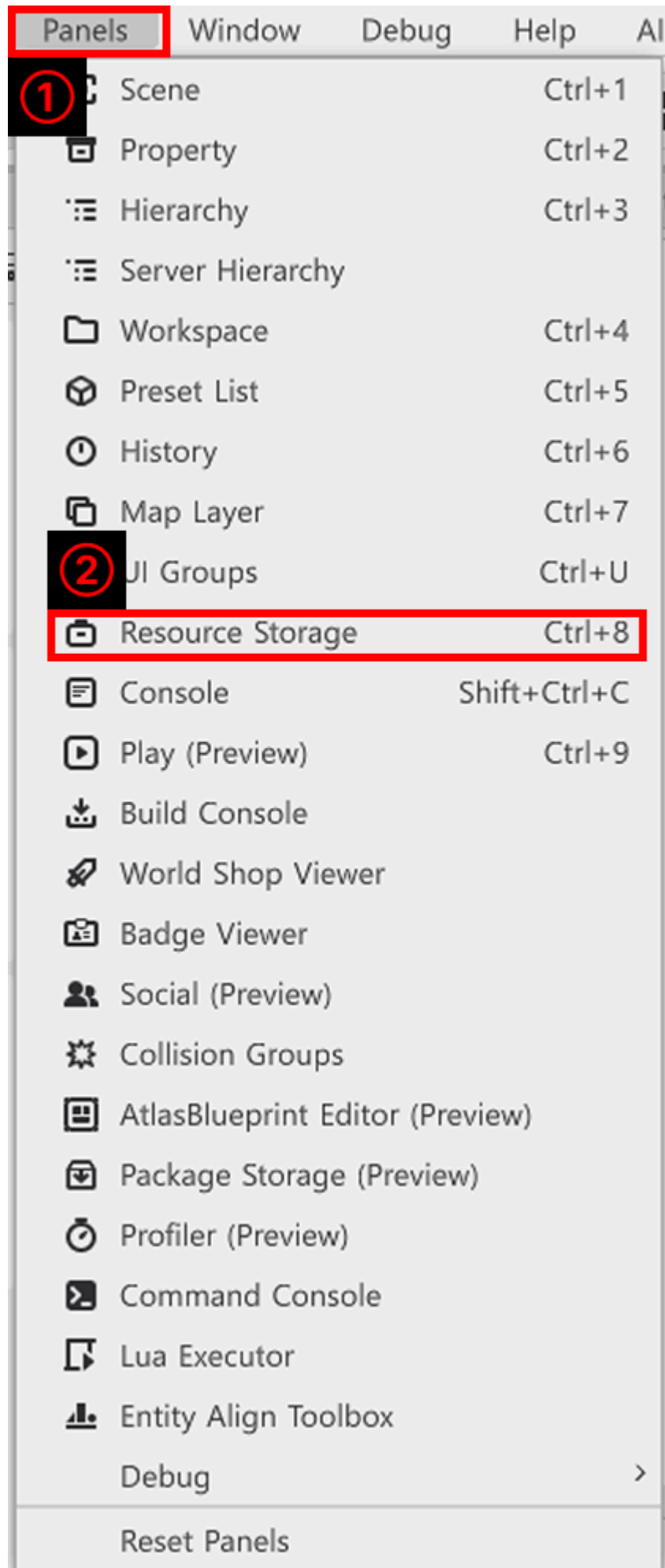
end
```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

## 리소스 검색하기

- BGM을 출력하기 위해선 어떤 사운드 파일을 재생할 지 지정해야 합니다.
- 메이플스토리 월드에서는 다양한 리소스를 제공하고 있습니다.





- 메이플스토리 월드 에디터 상단의 `lua @Component script BGMPlayer extends Component`

```
property string bgmID = ""
```

```
@ExecSpace("ClientOnly") method void OnBeginPlay() self:PlayBGM() end
```

```
@ExecSpace("ClientOnly") method void PlayBGM() -- 등록된 bgmID를 기준으로 배경음을 재생합니다.
_SoundService:PlayBGM(self.bgmID, 1.0) end
```

```
end
```

> 해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

<br>

### 리소스 검색하기

- BGM을 출력하기 위해선 어떤 사운드 파일을 재생할 지 지정해야 합니다.
- 메이플스토리 월드에서는 다양한 리소스를 제공하고 있습니다.

<p align="center">



</p>

- 메이플스토리 월드 에디터 상단의 `Panels`를 클릭합니다.
- `Resource Storage`를 클릭하여 `Resource Storage`창을 엽니다.

<p align="center">



</p>

- 출력된 `Resource Storage`창에서 필요한 리소스의 RUID를 복사합니다.

<p align="center">



</p>

- 다른 방법으로는 `AI Lab`의 기능을 이용하는 방법이 있습니다.
- 메이플 스토리 월드 에디터 상단의 `AI Lab`을 클릭합니다.
- 메뉴 내 `Resource Search`를 클릭합니다.

<p align="center">



</p>

- 이름, 키워드, 타입, 카테고리 등 다양한 방법으로 리소스를 검색할 수 있습니다.
- 찾은 BGM 리소스 ID를 앞서 제작한 BGMPlayer의 bgmID에 등록합니다.

<br>

## ## 최소 구현 기준

- 하나의 맵에 실제 게임이 작동하는 Map을 구성합니다.
- 캐릭터가 배치될 수 있는 슬롯과 몬스터가 이동하는 길을 구성합니다.
- 플레이어가 게임 내 배경음을 출력하도록 구현합니다.

<br>

## ## 응용 및 확장 방향

- 환경 설정을 통해 배경음을 조절할 수 있도록 변수를 지정합니다.
- 배경음이 맵에 따라 변경될 수 있도록 기능을 구현합니다.
- 샘플 월드와 다른 형태로 캐릭터가 있을 장소와 몬스터가 이동할 길을 구성합니다.

---

## # [💡 기초 학습] 캐릭터 제작하기

<br>

<br>

<br>

## ## 영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 맵 타입 이해하기: `00:20:06`

<br>

## ## 학습 목표

- 캐릭터를 구성하기 위한 DataSet을 제작하는 방법을 학습하고 제작합니다.
- 캐릭터의 정보를 사용하기 위한 Logic을 제작합니다.
- 캐릭터 Model을 제작하고 사용하는 방법에 대해 학습합니다.
- 캐릭터를 다른 슬롯으로 이동하도록 기능을 구현합니다.
- 테스트를 위한 소환 기능을 구현합니다.

<br>

## ## 해당 영상 시청 후 수행해 볼 내용

- 각 등급별 캐릭터를 3개 이상 제작하여 캐릭터의 다양성을 갖춥니다.
- DataSet의 데이터를 확장하여 제작한 캐릭터가 사용될 수 있도록 데이터를 준비합니다.
- 캐릭터의 이동이 가능하도록 슬롯 이동 기능을 구현합니다.

<br>

## ## 캐릭터 Model 제작하기

### ### Model이란?

<p align="center">



</p>

> 밀가루 반죽을 틀에 찍어내면 동일한 모양이 나오듯, Model은 동일한 형태의 엔티티를 사용하기 위한 기능입니다.

- Model은 메이플스토리 월드에서 동일한 형태, 기능을 미리 저장하는 하나의 형식입니다.
- 엔티티, 엔티티의 계층 구조, 사용하는 Component, Property에 할당된 값 등 다양한 요소들을 반복적으로 이용할 경우 사용하기 좋습니다.
- 특히 동일한 외형과 행동을 가지는 몬스터의 경우 Model을 만든 뒤 Model을 복제하여 사용하는 방식이 효과적입니다.

<br>

#### 왜 Model을 사용해야 합니까?

- 재사용성
  - 동일한 몬스터, 아이템, 장애물 등을 반복적으로 배치할 때 일일이 오브젝트를 생성하고 스크립트와 수치를 세팅할 필요가 없습니다.
  - 완성된 Model 원본 하나만 있으면 무한히 복제하여 사용할 수 있습니다.
- 일괄 업데이트
  - 게임 밸런스 조정이나 디자인 변경 시, 맵에 배치된 수많은 오브젝트를 일일이 수정할 필요가 없습니다.
  - Model 원본 딱 하나만 수정하면 Scene 내에 존재하는 모든 복제본에 변경 사항이 즉시 자동 적용됩니다.
- 게임 실행 중 동적 생성
  - 총알, 스킬 이펙트, 소환수처럼 씬에 미리 배치해 둘 수 없고 게임 진행 중에 실시간으로 생겨나는 객체들을 코드를 통해 필요한 순간에 즉각적으로 불러오고 관리할 수 있습니다.
- 안전하고 효율적인 팀 협업
  - 여러 명의 개발자가 하나의 Scene을 동시에 수정하면 충돌이 발생해 게임이 망가질 위험이 큼니다.
  - 엔티티를 Model 단위로 모듈화하면 기획자, 디자이너, 프로그래머가 각자 맡은 Model 파일만 독립적으로 작업할 수 있어 충돌 없이 안전한 협업이 가능해집니다.

<br>

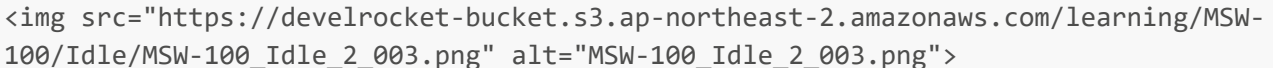
#### 캐릭터 Model 생성하기

<p align="center">

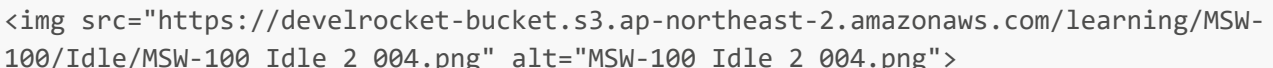


</p>

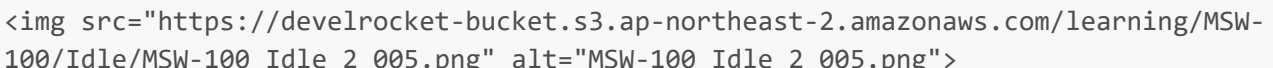
- `Hierarchy`에 빈 엔티티를 생성한 뒤 이름을 변경합니다.
  - 샘플 월드에서는 이름 규칙으로 **\*\*Character\_캐릭터 이름\*\***을 사용하고 있습니다.
- 생성한 엔티티에 다음의 Component를 모두 부여합니다.
- `AvatarRendererComponent`
  - 엔티티가 메이플스토리 캐릭터 외형을 출력하기 위한 Component입니다.
- `CostumeManagerComponent`
  - 캐릭터의 외형을 편집하기 위한 Component입니다.
- `TriggerComponent`
  - 캐릭터의 공격 범위를 지정하기 위한 Component입니다.
- `StateComponent`
  - 캐릭터의 현재 상태를 확인하기 위한 Component입니다.
- `UnitController`
  - 직접 제작이 필요한 Component입니다.
  - 캐릭터의 스탯, 행동 등을 제어하기 위한 Component입니다.

<p align="center">  
 MSW-100\_Idle\_2\_003.png</p>

- 자식 엔티티로 빈 엔티티를 생성한 뒤 **\*\*DragComponent\*\***로 이름을 변경합니다.
- 생성한 엔티티에 다음의 Component들을 부여합니다.
- `AvatarRendererComponent`
  - 캐릭터의 외형을 동일하게 출력하기 위한 Component입니다.
- `CostumeManagerComponent`
  - 캐릭터의 외형을 동일하게 출력하기 위한 Component입니다.
- `TouchReceiveComponent`
  - 캐릭터가 마우스의 클릭, 터치에 반응하도록 만들기 위한 Component입니다.
- `DragComponent`
  - 직접 생성한 Component입니다.
  - 캐릭터의 드래그에 따른 슬롯의 이동을 구현하기 위한 Component입니다.

<p align="center">  
 MSW-100\_Idle\_2\_004.png</p>

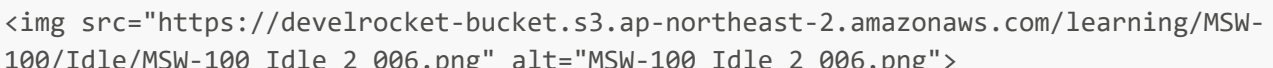
- Component의 추가까지 완료되었다면 `Hierarchy`에서 캐릭터로 사용할 엔티티를 마우스 우클릭합니다.
- `Create Model From Entity`를 클릭하여 엔티티를 Model로 제작합니다.

<p align="center">  
 MSW-100\_Idle\_2\_005.png</p>

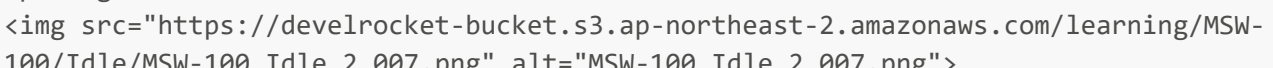
- `Workspace`, `MyDesk`안에 **\*\*Model\_엔티티 이름\*\***으로 엔티티가 생성되었다면 정상적으로 Model화가 된 상태입니다.

<br>

### ### Model 편집하기

<p align="center">  
 MSW-100\_Idle\_2\_006.png</p>

- `Workspace`, `MyDesk`에 있는 Model을 더블클릭하여 Model 편집창으로 이동할 수 있습니다.
- Model 편집창으로 이동할 경우 `Scene` 화면이 회색 배경에 엔티티만 있는 모습을 볼 수 있습니다.
- 해당 상태에서 Model을 편집하면 동일한 Model을 사용 할 경우 동일한 값으로 사용할 수 있습니다.
- 캐릭터의 특성을 부여하기 위해 다음의 Component들을 `Property` 창에서 수정합니다.

<p align="center">  
 MSW-100\_Idle\_2\_007.png</p>

</p>

- `CostumeManagerComponent`
  - 캐릭터의 외형을 꾸미기 위한 Component입니다.
  - `Property` 창의 각 요소들을 변경하여 원하는 외형의 캐릭터로 수정합니다.
  - 모든 수정을 완료한 뒤 `UseCustomEquipOnly`를 체크 상태로 전환합니다.

<p align="center">



</p>

- TriggerComponent의 수정을 진행합니다.
- ColliderType을 **Circle**로 변경합니다.
- CollisionGroup의 패널 버튼을 클릭하여 `Collision Groups` 창을 엽니다.

<p align="center">



</p>

- `Add Collision Group`을 통해 새로운 Collision Group을 생성합니다.
- 생성된 Collision Group의 이름을 변경하여 2개의 Collision Group을 생성합니다.
  - **CharacterAttackRange**: 캐릭터의 공격 범위를 지정하는 Collision Group입니다.
  - **Monster**: 몬스터의 충돌 범위를 지정하는 Collision Group입니다.

<p align="center">



</p>

- `Collision Groups` 창의 상단에 `Matrix`를 클릭합니다.
- 모든 체크 박스를 해제한 뒤 `CharacterAttackRange`와 `Monster`가 교차하는 체크 박스의 체크를 활성화합니다.

<br>

####  Matrix를 보는 방법

<p align="center">



</p>

- `Matrix` 창은 Collision Group들이 나열되어 있습니다.
- 좌측의 요소와 상단의 요소를 비교하여 Collision Group 간 충돌 여부를 검사시킬 수 있습니다.
  - EX: **CharacterAttackRange**와 **Monster**는 서로 충돌을 감지해야 하는가?
    - 체크박스를 활성화하면 충돌 감지 상태가 됩니다.
    - 체크박스를 비활성화하면 두 Collision Group은 서로를 감지하지 않습니다.

<br>

## 캐릭터 DataSet 구성하기

### ### DataSet의 목적

- 메이플스토리 월드에서 DataSet은 엑셀이나 데이터베이스의 테이블과 같은 형태로 게임의 고정된 수치나 정보를 저장하는 기능입니다.
- 데이터와 로직의 분리
  - 체력, 공격력 등의 수치를 스크립트 안에 직접 작성하지 않고 분리함으로써 코드를 깔끔하게 유지할 수 있습니다.
- 유지보수 및 밸런스 패치 용이성
  - 캐릭터의 능력치를 수정할 때 코드를 건드릴 필요 없이, DataSet에서 값만 변경하면 즉시 게임에 반영됩니다.
- 협업의 효율성
  - 프로그래머는 로직 개발에 집중하고, 기획자는 DataSet을 통해 콘텐츠와 밸런스를 독립적으로 조정할 수 있습니다.
- 높은 확장성
  - 새로운 캐릭터나 몬스터를 추가할 때, 로직 수정 없이 DataSet에 새로운 행을 추가하는 것만으로 쉽게 구현할 수 있습니다.

<br>

### ### DataSet 제작하기

```
<p align="center">

</p>
```

- `Workspace`에서 `MyDesk`를 마우스 우클릭합니다.
- `Create DataSet`을 클릭하여 DataSet을 제작합니다.
- 샘플 월드에서는 **CharacterData**라는 이름으로 작성했습니다.
- 아래는 **CharacterData**의 구성입니다.

```
<style type="text/css">
.tg {border-collapse:collapse;border-spacing:0;}
.tg td{border-color:black;border-style:solid;border-width:1px;font-family:Arial, sans-serif;font-size:14px;
overflow:hidden;padding:10px 5px;word-break:normal;}
.tg th{border-color:black;border-style:solid;border-width:1px;font-family:Arial, sans-serif;font-size:14px;
font-weight:normal;overflow:hidden;padding:10px 5px;word-break:normal;}
.tg .tg-lboi{border-color:currentColor;text-align:left;vertical-align:middle}
.tg .tg-9wq8{border-color:currentColor;text-align:center;vertical-align:middle}
</style>
<table class="tg"><thead>
<tr>
<th class="tg-9wq8">번호</th>
<th class="tg-9wq8">이름</th>
<th class="tg-9wq8">설명</th>
</tr></thead>
<tbody>
<tr>
<td class="tg-lboi">1</td>
<td class="tg-lboi">CharacterID</td>
<td class="tg-lboi">캐릭터를 분류하기 위한 ID입니다.</td>
</tr>
<tr>
```

```

 <td class="tg-lboi">2</td>
 <td class="tg-lboi">Grade</td>
 <td class="tg-lboi">캐릭터의 등급을 분류하기 위한 값입니다.</td>
 </tr>
 <tr>
 <td class="tg-lboi">3</td>
 <td class="tg-lboi">BasicAttackValue</td>
 <td class="tg-lboi">캐릭터의 기본 공격력을 지정하기 위한 값입니다.</td>
 </tr>
 <tr>
 <td class="tg-lboi">4</td>
 <td class="tg-lboi">AttackAnimationID</td>
 <td class="tg-lboi">캐릭터가 공격을 수행할 때 재생해야 하는 애니메이션의 Key를 지정하는
값입니다.</td>
 </tr>
 <tr>
 <td class="tg-lboi">5</td>
 <td class="tg-lboi">BasicAttackEffectID</td>
 <td class="tg-lboi">기본 공격 시행 시 캐릭터에게 출력해야 하는 이펙트의 ID 값입니다.
 </td>
 </tr>
 <tr>
 <td class="tg-lboi">6</td>
 <td class="tg-lboi">BasicAttackHitEffectID</td>
 <td class="tg-lboi">기본 공격 시행 시 적에게 출력해야 하는 이펙트의 ID 값입니다.</td>
 </tr>
 <tr>
 <td class="tg-lboi">7</td>
 <td class="tg-lboi">AttackRange</td>
 <td class="tg-lboi">캐릭터가 적을 감지하기 위한 범위를 설정하는 값입니다.</td>
 </tr>
 <tr>
 <td class="tg-lboi">8</td>
 <td class="tg-lboi">BasicAttackCoolTime</td>
 <td class="tg-lboi">기본 공격의 쿨타임을 지정하기 위한 값입니다.</td>
 </tr>
 <tr>
 <td class="tg-lboi">9</td>
 <td class="tg-lboi">ModelID</td>
 <td class="tg-lboi">캐릭터 Model의 Entry ID를 지정하기 위한 값입니다.</td>
 </tr>
 <tr>
 <td class="tg-lboi">10</td>
 <td class="tg-lboi">SkillData</td>
 <td class="tg-lboi">캐릭터의 스킬을 지정하기 위한 값입니다.</td>
 </tr>
</tbody></table>


```

- 이어서 소환을 구현하기 위해 GachaRateData를 제작합니다.
- GachaRateData는 2개의 값을 가집니다.
  - **\*\*Grade\*\***: 캐릭터의 등급을 구분하기 위한 값입니다.
  - **\*\*RateValue\*\***: 지정된 등급의 확률을 계산하기 위한 값입니다.



<br>

### ### StructType 제작하기

<p align="center">



</p>

- StructType은 Property들을 하나의 요소로 관리하기 위한 방법입니다.
- StructType을 선언한 뒤, Component 또는 Logic에서 생성한 StructType으로 선언 할 경우, StructType 내에 선언한 Property로 접근할 수 있습니다.

<p align="center">



</p>

- `Workspace`, `MyDesk`를 마우스 우클릭합니다.
- `Create Scripts`를 클릭해 스크립트 생성 메뉴를 엽니다.
- `Create StructType`을 눌러 StructType Script를 생성합니다.

<p align="center">



</p>

- 앞서 제작한 CharacterData의 요소 이름을 Property로 선언합니다.
- 이를 통해 DataSet의 정보를 사용할 때 모든 Property를 하나씩 선언할 필요 없이 StructType을 호출하여 관리할 수 있습니다.
- 앞으로 DataSet을 제작할 경우 StructType을 제작하여 관리합니다.

<br>

### ## 캐릭터 Logic 구현하기

- 랜덤한 캐릭터를 선정하고, 선정된 캐릭터의 정보를 반환하는 Logic을 제작합니다.

<p align="center">



</p>

- `Workspace`, `MyDesk`를 마우스 우클릭합니다.
- `Create Scripts`를 클릭하여 메뉴를 엽니다.
- `Create Logic`을 통해 Logic을 제작합니다.
- 샘플 월드에서는 생성한 Logic의 이름을 CharacterGachaLogic으로 선언합니다.
- 아래는 CharacterGachaLogic의 코드입니다.

```
```lua
```

```
@Logic
```

```
script CharacterGachaLogic extends Logic
```

```

property table characterData = {}

property table gachaRateData = {}

property table skillData = {}

property CharacterSlotGroupComponent characterSlotGroupController = nil

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 캐릭터 정보 DataSet을 등록합니다.
self:InitCharacterData()
-- 캐릭터 뽑기 정보를 등록합니다.
self:InitGachaRateData()
-- 캐릭터 생성과 슬롯 관리를 담당하는 characterSlotGroupController 초기화를 시도합니다.
self:InitCharacterSlotGroup()
end

@ExecSpace("ClientOnly")
method void InitCharacterData()
-- CharacterData DataSet을 가지고 있는지 점검합니다.
if not isvalid(self.characterData) or not next(self.characterData) then
-- 만약 characterData가 등록되어 있지 않을 경우, DataService를 이용해 Workspace의
CharacterData DataSet을 찾습니다.
local getData = _DataService:GetTable("CharacterData")
-- CharacterData를 정상적으로 가져왔는지 확인합니다.
if not isvalid(getData) then
log_error("CharacterData DataSet을 찾을 수 없습니다!")
return
end
-- 정상적으로 CharacterData DataSet을 찾았다면 CharacterData의 rowCount를 가져옵니
다.
local rowCount = getData:GetRowCount()
-- rowCount 수 만큼 DataSet을 순회하며 데이터를 찾아 등록합니다.
for i = 1, rowCount do
local setData = CharacterData()
setData.CharacterID = getData:GetCell(i, "CharacterID")
setData.Grade = tonumber(getData:GetCell(i, "Grade"))
setData.BasicAttackValue = tonumber(getData:GetCell(i,
"BasicAttackValue"))
setData.AttackAnimationID = tostring(getData:GetCell(i,
"AttackAnimationID"))
setData.BasicAttackEffectID = tostring(getData:GetCell(i,
"BasicAttackEffectID"))
setData.BasicAttackHitEffectID = tostring(getData:GetCell(i,
"BasicAttackHitEffectID"))
setData.AttackRange = tonumber(getData:GetCell(i, "AttackRange"))
setData.SkillData = tostring(getData:GetCell(i, "SkillData"))
setData.BasicAttackCoolTime = tonumber(getData:GetCell(i,
"BasicAttackCoolTime"))
setData.ModelID = tostring(getData:GetCell(i, "ModelID"))
if not isvalid(self.characterData[setData.Grade]) or not
next(self.characterData[setData.Grade]) then
self.characterData[setData.Grade] = {}

```

```

        end
        table.insert(self.characterData[setData.Grade], setData)
    end
end
end

@ExecSpace("ClientOnly")
method void InitGachaRateData()
-- gachaRateData가 비어있는지 검사합니다.
if not isvalid(self.gachaRateData) or not next(self.gachaRateData) then
-- gachaRateData가 비어있을 경우 초기화를 위해 GachaRateData를 가져옵니다.
local getData = _DataService:GetTable("GachaRateData")
-- 만약 전달받은 getData값이 nil일 경우 에러 로그를 출력하고 코드 실행을 중단합니다.
if not isvalid(getData) then
    log_error("GachaRateData를 찾을 수 없습니다!")
    return
end
-- 정상적으로 DataSet을 찾았다면 GachaRateData의 rowCount를 가져옵니다.
local GetRowCount = getData:GetRowCount()
-- rowCount의 수 만큼 Data를 찾아 등록합니다.
for i = 1, GetRowCount do
    local rateData = GachaRateData()
    rateData.Grade = tonumber(getData:GetCell(i, "Grade"))
    rateData.RateValue = tonumber(getData:GetCell(i, "RateValue"))
    table.insert(self.gachaRateData, rateData)
end
end
end

@ExecSpace("ClientOnly")
method void InitCharacterSlotGroup()
-- 현재 맵이 InGame인지 점검합니다.
if _UserService.LocalPlayer.CurrentMapName == "InGame" then
-- 현재 맵이 InGame일 경우 경로를 기반으로 CharacterSlotPosGroup을 찾습니다.
local findEntity =
_EntityService:GetEntityByPath("/maps/InGame/CharacterSlotPosGroup")
if isvalid(findEntity) then
-- 정상적으로 엔티티를 찾았다면 CharacterSlotGroupComponent를 등록합니다.
self.characterSlotGroupController = findEntity.CharacterSlotGroupComponent
end
end
end

@ExecSpace("ClientOnly")
method number GetRandomGrade()
-- 저장된 gradeCount 개수를 가져옵니다.
local gradeCount = #self.gachaRateData
-- 모든 gradeCount의 확률을 더한 값을 가져옵니다.
-- 이 때 GachaRateData는 반드시 정수값만 있어야합니다.
local totalValue = 0
for i = 1, gradeCount do
    totalValue = totalValue + self.gachaRateData[i].RateValue
end
end

```

```

-- 1에서 부터 totalValue 사이의 랜덤한 값을 가져옵니다.
local getRandomNumber = _UtilLogic:RandomIntegerRange(1, totalValue)
-- gachaRateData에서 하나씩 값을 뺏셈합니다.
for i = 1, gradeCount do
    getRandomNumber = getRandomNumber - self.gachaRateData[i].RateValue
    if getRandomNumber <= 0 then
        return self.gachaRateData[i].Grade
    end
end
-- 정상적으로 값이 나오지 않았을 경우, 에러 로그와 함께 가장 낮은 등급을 반환합니다.
log_error("정상적인 랜덤 등급값을 얻는데 실패했습니다!")
return 1
end

@ExecSpace("ClientOnly")
method CharacterData GetRandomCharacterData()
-- GetRandomCharacterByGradeValue로 선택된 랜덤 캐릭터를 반환합니다.
return self:GetRandomCharacterByGradeValue(self:GetRandomGrade())
end

@ExecSpace("ClientOnly")
method CharacterData GetRandomCharacterByGradeValue(number GradeValue)
-- 전달받은 GradeValue값을 기준으로 해당 등급의 랜덤한 캐릭터 하나를 반환합니다.
local getRandomList = self.characterData[GradeValue]

local getRandomCharacter = _UtilLogic:RandomIntegerRange(1, #getRandomList)
return getRandomList[getRandomCharacter]
end

@ExecSpace("ClientOnly")
method void RequestSpawnCharacter()
-- 현재 소환 토큰의 수가 충분한지 검사합니다.
if _GameLogic.summonToken >= 1 then
    -- 현재 소환 가능한 슬롯이 있는지 검사합니다.
    local getSlotNumber =
self.characterSlotGroupController:ReturnEmptySlotNumber()
    -- 정상적인 슬롯 번호를 받았을 경우 캐릭터를 생성합니다.
    if getSlotNumber >= 1 then
        local getData = self:GetRandomCharacterData()
        self.characterSlotGroupController:SpawnCharacter(getSlotNumber, getData)
        -- 소환 토큰의 값을 1 감소시키고 UI를 갱신하도록 요청합니다.
        _GameLogic:UpdateSummonToken(-1)
    else
        -- 빈 캐릭터 슬롯이 없을 경우 토스트 메시지를 출력합니다.
        _UIToast:ShowMessage(_LocalizationService:GetText("UI_NeedSpace"))
        log("현재 캐릭터 슬롯이 가득 찼습니다!")
    end
else
    -- 소환 토큰이 부족할 경우 토스트 메시지를 출력합니다.
    _UIToast:ShowMessage(_LocalizationService:GetText("UI_NeedToken"))
end
end
end

```

```
end
```

해당 코드는 PlainText 기준으로 작성된 코드입니다.

캐릭터 제어를 위한 Component 구현하기

- 캐릭터의 행동과 데이터를 가져오기 위한 UnitController, 캐릭터의 슬롯 이동을 구현하기 위한 DragComponent를 구성해야 합니다.
- 아래는 UnitController, DragComponent의 전문입니다.

```
@Component
script UnitController extends Component

property Entity avatarBodyComponent = nil

property table enemyTable = {}

property number basicAtkDelta = 0

property CharacterData characterData = CharacterData()

property DragComponent dragComponent = nil

property Entity currentSlot = nil

property number skillDelta = 0

property SkillData characterSkillData = SkillData()

property TriggerComponent triggerComponent = nil

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 아바타를 재생하기 위한 avatarBodyComponent를 찾아 등록합니다.
if self.avatarBodyComponent == nil then
    local findComponent = self.Entity.AvatarRenderComponent:GetBodyEntity()
    if isvalid(findComponent) then
        self.avatarBodyComponent = findComponent
    end
end
-- 자식 엔티티로 dragComponent를 찾아 등록합니다.
if not isvalid(self.dragComponent) then
    local findComponent = self.Entity:GetChildByName("DragEntity", true)
    if isvalid(findComponent) then
        self.dragComponent = findComponent.DragComponent
    end
end
-- dragComponent의 초기화 및 엔티티 비활성화를 수행합니다.
```

```

self:InitDragComponentSprite()
self.Entity.Enable = false
-- triggerComponent가 비어있을 경우 이를 초기화합니다.
if not invalid(self.triggerComponent) then
    -- 자기 자신에게서 TriggerComponent를 찾습니다.
    local findComponent = self.Entity.TriggerComponent
    -- 자신의 TriggerComponent를 찾았다면 이를 triggerComponent에 등록합니다.
    if invalid(findComponent) then
        self.triggerComponent = findComponent
    end
end
end
end

@ExecSpace("ClientOnly")
method void InitCharacterData(CharacterData Data, Entity SlotEntity)
-- 캐릭터를 활성화시킵니다.
self.Entity.Enable = true
-- 캐릭터에 대한 정보를 받아 등록합니다.
self.characterData = Data
-- 캐릭터의 스킬 정보를 찾아 등록합니다.
self.characterSkillData =
_SkillDataLogic:GetSkillDataBySkillID(self.characterData.SkillData)
-- 현재 슬롯을 전달받은 SlotEntity로 등록합니다.
self.currentSlot = SlotEntity
-- 공격 범위를 전달받은 Data 기준으로 갱신합니다.
self.triggerComponent.CircleRadius = self.characterData.AttackRange
end

@ExecSpace("ClientOnly")
method void InitDragComponentSprite()
-- DragEntity의 아바타를 현재 사용중인 캐릭터의 아바타와 동일하게 만들기 위해 값들을 초기화합
니다.
local currentSprite = self.Entity.CostumeManagerComponent
local dragComponent =
self.Entity:GetChildByName("DragEntity").CostumeManagerComponent
dragComponent.CustomBodyEquip = currentSprite.CustomBodyEquip
dragComponent.CustomCapeEquip = currentSprite.CustomCapeEquip
dragComponent.CustomCapEquip = currentSprite.CustomCapEquip
dragComponent.CustomCoatEquip = currentSprite.CustomCoatEquip
dragComponent.CustomEarAccessoryEquip = currentSprite.CustomEarAccessoryEquip
dragComponent.CustomEarEquip = currentSprite.CustomEarEquip
dragComponent.CustomEyeAccessoryEquip = currentSprite.CustomEyeAccessoryEquip
dragComponent.CustomFaceAccessoryEquip = currentSprite.CustomFaceAccessoryEquip
dragComponent.CustomFaceEquip = currentSprite.CustomFaceEquip
dragComponent.CustomGloveEquip = currentSprite.CustomGloveEquip
dragComponent.CustomHairEquip = currentSprite.CustomHairEquip
dragComponent.CustomLongcoatEquip = currentSprite.CustomLongcoatEquip
dragComponent.CustomOneHandedWeaponEquip =
currentSprite.CustomOneHandedWeaponEquip
dragComponent.CustomPantsEquip = currentSprite.CustomPantsEquip
dragComponent.CustomShoesEquip = currentSprite.CustomShoesEquip
dragComponent.CustomSubWeaponEquip = currentSprite.CustomSubWeaponEquip
dragComponent.CustomTwoHandedWeaponEquip =
currentSprite.CustomTwoHandedWeaponEquip

```

```

end

@ExecSpace("ClientOnly")
method void ChangeSlot(Entity SlotEntity)
-- 현재 슬롯을 전달받은 슬롯으로 변경합니다.
self.currentSlot = SlotEntity
-- 현재 캐릭터의 위치를 전달받은 슬롯의 위치로 초기화합니다.
self.Entity.TransformComponent.WorldPosition =
SlotEntity.TransformComponent.WorldPosition
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- 델타타임은 조건 없이 항상 더해줍니다.
self.skillDelta = self.skillDelta + delta
self.basicAtkDelta = self.basicAtkDelta + delta

-- 스킬 공격 쿨타임을 확인하여 조건을 충족할 경우 스킬 공격을 수행합니다.
if self.skillDelta >= self.characterSkillData.SkillCoolTime then
    self:SkillAttack()
end

-- 만약 스킬 공격이 성공하여 쿨타임이 0으로 초기화되었다면, 이번 프레임엔 기본 공격을 쉬어갑니
다.
if self.skillDelta == 0 then
    return
end

-- 기본 공격 쿨타임을 확인하여 조건을 충족할 경우 기본 공격을 수행합니다.
if self.basicAtkDelta >= self.characterData.BasicAttackCoolTime then
    self:BasicAttack()
end
end

@ExecSpace("ClientOnly")
method void BasicAttack()
-- 캐릭터의 StateComponent를 가져옵니다.
local bodySelector = self.Entity.StateComponent
-- 캐릭터의 현재 상태 이름을 가져옵니다.
local currentState = bodySelector.CurrentStateName
-- 현재 상태가 IDLE이 아닐 경우, 기본 공격 수행을 하지 않습니다.
if currentState ~= "IDLE" then
    return
end

---@type Entity
local targetEntity = nil
-- 테이블을 역순으로 순회하며 유효하지 않은 엔티티를 안전하게 제거합니다.
for i = #self.enemyTable, 1, -1 do
    local enemy = self.enemyTable[i]
    -- 엔티티가 아직 월드에 존재하는지 확인합니다.
    if invalid(enemy) then
        -- 엔티티에 MonsterController가 있고 살아있는지 확인합니다.
        if enemy.MonsterController and enemy.MonsterController.isAlive then
            targetEntity = enemy
        end
    end
end

```

```

        break
    end
else
    -- 이미 파괴된 엔티티라면 테이블에서 제거합니다.
    table.remove(self.enemyTable, i)
end
end
-- 만약 지정된 targetEntity가 사용 불가능할 경우 공격을 수행하지 않습니다.
if not isValid(targetEntity) then
    return
end
-- 공격을 수행했다 가정하여 basicAtkDelta값을 0으로 설정합니다.
self.basicAtkDelta = 0
-- 공격 애니메이션을 출력시키기 위해 event를 생성, 전달합니다.
local event = ActionStateChangedEvent(self.characterData.AttackAnimationID,
self.characterData.AttackAnimationID, 1, SpriteAnimClipPlayType.Onetime)
self.avatarBodyComponent:SendEvent(event)
-- 출력해야 하는 캐릭터쪽 이펙트가 있을 경우 이를 출력합니다.
if not _UtilLogic:IsNilorEmptyString(self.characterData.BasicAttackEffectID) then
    _EffectService:PlayEffect(self.characterData.BasicAttackEffectID, self.Entity,
self.Entity.TransformComponent.WorldPosition, 0, Vector3.one)
end
-- 적에게 출력해야 하는 이펙트가 있을 경우 이를 출력합니다.
if not _UtilLogic:IsNilorEmptyString(self.characterData.BasicAttackHitEffectID)
then
    _EffectService:PlayEffect(self.characterData.BasicAttackHitEffectID,
targetEntity, targetEntity.TransformComponent.WorldPosition, 0, Vector3.one)
end
-- 타겟이 된 적의 MonsterController를 통해 GetDamage를 수행하도록 요청합니다.
targetEntity.MonsterController:GetDamage(self.characterData.BasicAttackValue)
end

@ExecSpace("ClientOnly")
method void SkillAttack()
    -- 애니메이션 재생을 위한 bodySelector를 찾아 등록합니다.
    local bodySelector = self.Entity.StateComponent
    -- 현재 상태를 찾기 위해 CurrentStateName을 등록합니다.
    local currentState = bodySelector.CurrentStateName
    -- 현재 상태가 IDLE이 아닐 경우 코드 실행을 중단합니다.
    if currentState ~= "IDLE" then
        return
    end
    ---@type Entity
    local targetEntity = nil
    -- 테이블을 역순으로 순회하며 유효하지 않은 엔티티를 안전하게 제거합니다.
    for i = #self.enemyTable, 1, -1 do
        local enemy = self.enemyTable[i]
        if isValid(enemy) then
            if enemy.MonsterController and enemy.MonsterController.isAlive then
                targetEntity = enemy
                break
            end
        else
            table.remove(self.enemyTable, i)
        end
    end
end

```



```

        end
    end
    -- 만약 targetEntity가 nil일 경우 코드 실행을 중단합니다.
    if not invalid(targetEntity) then
        return
    end
    -- 스킬 사용을 성공했다 가정하여 skillDelta값을 0으로 초기화합니다.
    self.skillDelta = 0
    -- 공격 애니메이션을 재생합니다.
    local event = ActionStateChangedEvent(self.characterData.AttackAnimationID,
    self.characterData.AttackAnimationID, 1, SpriteAnimClipPlayType.Onetime)
    self.avatarBodyComponent:SendEvent(event)
    -- 스킬 사용시 자기 자신에게 출력해야 하는 이펙트가 있을 경우 이펙트를 출력합니다.
    if not _UtilLogic:IsNilorEmptyString(self.characterSkillData.CharacterSkillEffect)
    then
        _EffectService:PlayEffect(self.characterSkillData.CharacterSkillEffect,
    self.Entity, self.Entity.TransformComponent.WorldPosition, 0, Vector3.one)
    end
    -- 피격 대상에게 출력해야 하는 이펙트가 있을 경우 이펙트를 출력합니다.
    if not _UtilLogic:IsNilorEmptyString(self.characterSkillData.SkillEffect) then
        _EffectService:PlayEffect(self.characterSkillData.SkillEffect, targetEntity,
    targetEntity.TransformComponent.WorldPosition, 0, Vector3.one)
    end
    -- 대상이 된 targetEntity의 MonsterContorller에게 GetDamage를 수행하도록 요청합니다.
    targetEntity.MonsterController:GetDamage(self.characterSkillData.SkillAttack)
end

@ExecSpace("ClientOnly")
@EventSender("Self")
handler HandleTriggerEnterEvent(TriggerEnterEvent event)
-- 충돌 범위에 들어온 엔티티가 CollisionGroups, Monster일 경우 이를 등록합니다.
if event.TriggerBodyEntity.TriggerComponent.CollisionGroup ==
CollisionGroups.Monster then
    if #self.enemyTable >= 1 then
        local isAlreadyHave = false
        for i = 1, #self.enemyTable do
            if self.enemyTable[i] == event.TriggerBodyEntity then
                isAlreadyHave = true
                break
            end
        end
        if isAlreadyHave == false then
            table.insert(self.enemyTable, event.TriggerBodyEntity)
        end
    else
        table.insert(self.enemyTable, event.TriggerBodyEntity)
    end
end

end
end

@ExecSpace("ClientOnly")
@EventSender("Self")
handler HandleTriggerLeaveEvent(TriggerLeaveEvent event)
-- 충돌 범위를 벗어난 엔티티가 Monster일 경우 공격 리스트에서 제거합니다.

```

```

if event.TriggerBodyEntity.TriggerComponent.CollisionGroup ==
CollisionGroups.Monster then
    if #self.enemyTable >= 1 then
        local listCount = #self.enemyTable
        for i = listCount, 1, -1 do
            if self.enemyTable[i] == event.TriggerBodyEntity then
                table.remove(self.enemyTable, i)
                break
            end
        end
    end
end
end
end
end
end

```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

```

@Component
script DragComponent extends Component

@DisplayName("클릭 입력 리스트")
property table Index = {}

@DisplayName("Hold Handler")
@Description("클릭 유지 감지 체크")
property any Handler = nil

@DisplayName("Release Handler")
@Description("클릭 해제 감지 체크")
property any ReleaseHandler = nil

property Entity slotEntity = nil

property Entity parentEntity = "4bc00000-b6fa-481d-a049-aed3dea2d411"

property CharacterSlotGroupComponent characterSlotGroup = nil

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- characterSlotGroup을 초기화합니다.
self:InitCharacterSlotGroup()
end

@ExecSpace("ClientOnly")
method void InitCharacterSlotGroup()
-- characterSlotGroup이 nil인지 검사합니다.
if not invalid(self.characterSlotGroup) then
-- characterSlotGroup이 nil일 경우, 현재 맵의 여부를 판단합니다.
if self.Entity.Parent.CurrentMapName == "InGame" then
-- 현재 맵이 InGame일 경우 InGame에 있는 CharacterSlotPosGroup을 찾습니다.
local findEntity =

```

```

_EntityService:GetEntityByPath("/maps/InGame/CharacterSlotPosGroup")
    -- 정상적으로 찾았는지 검증합니다.
    if invalid(findEntity) then
        -- 정상적으로 엔티티를 찾았다면 characterSlotGroup에 등록합니다.
        self.characterSlotGroup = findEntity.CharacterSlotGroupComponent
    end
end
end
end

@ExecSpace("ClientOnly")
method void Move(any screenholdEvent)
    -- 현재 Hold상태의 Id값을 가져옵니다.
    local touchId = screenholdEvent.TouchId
    -- 현재 Hold의 위치값을 가져옵니다.
    local touchPoint = screenholdEvent.TouchPoint
    -- 클릭 인덱스 전체를 탐색합니다.
    for i = 1, #self.Index do
        -- 현재 홀드중인 Id와 동일한 Index[touchId]가 있을 경우 엔티티의 위치를 현재 Hold의 위치값으로 이동합니다.
        if self.Index[i] == touchId then
            self.Entity.TransformComponent.WorldPosition =
            _UILogic:ScreenToWorldPosition(touchPoint):ToVector3()
            break
        end
    end
end
end

@ExecSpace("ClientOnly")
method void AddHandler(integer touchId)
    -- 이미 Handler가 존재하면 중복 처리를 방지하기 위해 코드 실행을 중단합니다.
    if self.Handler ~= nil then
        return
    end

    -- 이미 다른 캐릭터를 드래그 중이라면 무시하여 다중 드래그를 방지합니다.
    if invalid(self.characterSlotGroup) then
        if self.characterSlotGroup.isDragging == true then
            return
        end
    end
    -- 드래그 시작 시 다른 슬롯들이 반응하지 못하도록 잠금 처리합니다.
    self.characterSlotGroup.isDragging = true
end

    -- 현재 DragEntity가 가려진 상태라면 활성화합니다.
    if self.Entity.Visible == false then
        self.Entity.Visible = true
    end
    table.insert(self.Index, touchId)

    -- Hold뿐만 아니라 Release도 화면 전체에서 감지하도록 등록합니다.
    self.Handler = _InputService:ConnectEvent(ScreenTouchHoldEvent, self.Move)
    self.ReleaseHandler = _InputService:ConnectEvent(ScreenTouchReleaseEvent,
    self.OnRelease)

```

```

end

@ExecSpace("ClientOnly")
method void OnRelease(any event)
local TouchId = event.TouchId
-- 드래그가 해제되었으므로 모든 TouchId를 탐색하여 해제합니다.
for i = 1, #self.Index do
    if self.Index[i] == TouchId then
        table.remove(self.Index, i)
        if #self.Index <= 0 then
            -- 추가한 Hold와 Release 이벤트를 해제합니다.
            _InputService:DisconnectEvent(ScreenTouchHoldEvent, self.Handler)
            _InputService:DisconnectEvent(ScreenTouchReleaseEvent,
self.ReleaseHandler)
            self.Handler = nil
            self.ReleaseHandler = nil

            -- 드래그가 끝났으므로 잠금을 해제합니다.
            if invalid(self.characterSlotGroup) then
                self.characterSlotGroup.isDragging = false
            end

            -- 드롭한 위치를 가져옵니다.
            local dropPos = self.Entity.TransformComponent.WorldPosition

            -- 현재 위치에서 가장 가까운 타겟 슬롯을 탐색을 위해 현재 슬롯 번호를 중복이 없는
            -1로 설정합니다.
            local targetSlotNumber = -1
            -- 슬롯으로 인정할 최대 반경 거리
            local closestDist = 1.5
            -- characterSlotGroup과 characterSlotGroup의 characterSlotList가 사용 가
            능한지 점검합니다.
            if invalid(self.characterSlotGroup) and
self.characterSlotGroup.characterSlotList then
                -- for문을 돌면서 characterSlotList를 탐색합니다.
                for slotIndex = 1, #self.characterSlotGroup.characterSlotList do
                    -- slotIndex의 slotEntity를 가져옵니다.
                    local slotEntity =
self.characterSlotGroup.characterSlotList[slotIndex].Entity
                    -- slotPos의 slotEntity의 worldPosition을 가져옵니다.
                    local slotPos = slotEntity.TransformComponent.WorldPosition
                    -- dragEntity인 자기 자신과 슬롯의 거리를 계산합니다.
                    local dist = Vector2.Distance(dropPos:ToVector2(),
slotPos:ToVector2())
                    -- 앞서 지정한 closetDist보다 거리가 적을 경우 범위에 들어왔다고 판단
                    합니다.

                    if dist < closestDist then
                        -- 조건을 충족한 Slot의 거리로 설정합니다.
                        closestDist = dist
                        -- 조건을 충족한 slotIndex를 targetSlotNumber로 등록합니
                        다.

                        targetSlotNumber = slotIndex
                    end
                end
            end

```

```

        end

        -- 기존의 슬롯 번호를 가져옵니다.
        local beforeSlotNumber =
self.parentEntity.UnitController.currentSlot.CharacterSlotComponent:GetSlotNumber(
)
        -- targetSlotNumber가 -1이 아니고, targetSlotNumber가 beforeSlotNumber와
다를 경우 이동을 수행합니다.
        if targetSlotNumber ~= -1 and targetSlotNumber ~= beforeSlotNumber
then
            self.characterSlotGroup:SwitchSlotCharacter(targetSlotNumber,
self.parentEntity, beforeSlotNumber)
            end
        end
        break
    end
end

    -- 이동 처리가 끝난 후 원래 위치로 복귀 및 숨김처리를 수행합니다.
    self.Entity.TransformComponent.WorldPosition =
self.parentEntity.TransformComponent.WorldPosition
    self.Entity.Visible = false
end

@ExecSpace("ClientOnly")
@EventSender("Self")
handler HandleTouchEvent(TouchEvent event)
-- 입력 받은 event의 TouchId 값을 가져옵니다.
local TouchId = event.TouchId
-- AddHandler를 통해 필요한 이벤트를 등록합니다.
self:AddHandler(TouchId)
end

end

```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

최소 구현 기준

- 사용할 수 있는 Model 형태로 캐릭터를 제작합니다.
- Model로 제작된 캐릭터를 사용할 수 있도록 기능을 구현합니다.
- 등급에 따라 캐릭터가 생성될 수 있도록 등급별 1종 이상 캐릭터를 제작합니다.

응용 및 확장 방향

- 캐릭터의 소환 확률을 다른 방식을 사용하여 적용합니다.
- 캐릭터가 공격을 수행할 때 효과음이 재생될 수 있도록 기능을 구현합니다.
- 각 캐릭터마다 공격시 재생되는 애니메이션의 속도를 공격 속도에 맞춰 조절하는 기능을 추가합니다.

[💡 기초 학습] 몬스터 제작하기

영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 맵 타입 이해하기: ``lua @Logic script CharacterGachaLogic extends Logic

```
property table characterData = {}
```

```
property table gachaRateData = {}
```

```
property table skillData = {}
```

```
property CharacterSlotGroupComponent characterSlotGroupController = nil
```

```
@ExecSpace("ClientOnly") method void OnBeginPlay() -- 캐릭터 정보 DataSet을 등록합니다.
```

```
self:InitCharacterData() -- 캐릭터 뽑기 정보를 등록합니다. self:InitGachaRateData() -- 캐릭터 생성과 슬롯 관리를 담당하는 characterSlotGroupController 초기화를 시도합니다. self:InitCharacterSlotGroup() end
```

```
@ExecSpace("ClientOnly") method void InitCharacterData() -- CharacterData DataSet을 가지고 있는지 점검합니다. if not isvalid(self.characterData) or not next(self.characterData) then -- 만약 characterData가 등록되어 있지 않을 경우, DataService를 이용해 Workspace의 CharacterData DataSet을 찾습니다. local getData = _DataService:GetTable("CharacterData") -- CharacterData를 정상적으로 가져왔는지 확인합니다. if not isvalid(getData) then log_error("CharacterData DataSet을 찾을 수 없습니다!") return end -- 정상적으로 CharacterData DataSet을 찾았다면 CharacterData의 rowCount를 가져옵니다. local rowCount = getData:GetRowCount() -- rowCount 수 만큼 DataSet을 순회하며 데이터를 찾아 등록합니다. for i = 1, rowCount do local setData = CharacterData() setData.CharacterID = getData:GetCell(i, "CharacterID") setData.Grade = tonumber(getData:GetCell(i, "Grade")) setData.BasicAttackValue = tonumber(getData:GetCell(i, "BasicAttackValue")) setData.AttackAnimationID = tostring(getData:GetCell(i, "AttackAnimationID")) setData.BasicAttackEffectID = tostring(getData:GetCell(i, "BasicAttackEffectID")) setData.BasicAttackHitEffectID = tostring(getData:GetCell(i, "BasicAttackHitEffectID")) setData.AttackRange = tonumber(getData:GetCell(i, "AttackRange")) setData.SkillData = tostring(getData:GetCell(i, "SkillData")) setData.BasicAttackCoolTime = tonumber(getData:GetCell(i, "BasicAttackCoolTime")) setData.ModelID = tostring(getData:GetCell(i, "ModelID")) if not isvalid(self.characterData[setData.Grade]) or not next(self.characterData[setData.Grade]) then self.characterData[setData.Grade] = {} end table.insert(self.characterData[setData.Grade], setData) end end end
```

```
@ExecSpace("ClientOnly") method void InitGachaRateData() -- gachaRateData가 비어있는지 검사합니다. if not isvalid(self.gachaRateData) or not next(self.gachaRateData) then -- gachaRateData가 비어있을 경우 초기화를 위해 GachaRateData를 가져옵니다. local getData = _DataService:GetTable("GachaRateData") -- 만약 전달받은 getData값이 nil일 경우 에러 로그를 출력하고 코드 실행을 중단합니다. if not isvalid(getData) then log_error("GachaRateData를 찾을 수 없습니다!") return end -- 정상적으로 DataSet을 찾았다면 GachaRateData의 rowCount를 가져옵니다. local GetRowCount = getData:GetRowCount() -- rowCount의 수 만큼 Data를 찾아 등록
```

```

합니다. for i = 1, GetRowCount do local rateData = GachaRateData() rateData.Grade =
tonumber(getData:GetCell(i, "Grade")) rateData.RateValue = tonumber(getData:GetCell(i, "RateValue"))
table.insert(self.gachaRateData, rateData) end end end

```

```

@ExecSpace("ClientOnly") method void InitCharacterSlotGroup() -- 현재 맵이 InGame인지 점검합니다. if
_UserService.LocalPlayer.CurrentMapName == "InGame" then -- 현재 맵이 InGame일 경우 경로를 기반으로
CharacterSlotPosGroup을 찾습니다. local findEntity =
_EntityService.GetEntityByPath("/maps/InGame/CharacterSlotPosGroup") if invalid(findEntity) then -- 정상적으
로 엔티티를 찾았다면 CharacterSlotGroupComponent를 등록합니다. self.characterSlotGroupController =
findEntity.CharacterSlotGroupComponent end end end

```

```

@ExecSpace("ClientOnly") method number GetRandomGrade() -- 저장된 gradeCount 개수를 가져옵니다. local
gradeCount = #self.gachaRateData -- 모든 gradeCount의 확률을 더한 값을 가져옵니다. -- 이 때 GachaRateData
는 반드시 정수값만 있어야합니다. local totalValue = 0 for i = 1, gradeCount do totalValue = totalValue +
self.gachaRateData[i].RateValue end

```

```

-- 1에서 부터 totalValue 사이의 랜덤한 값을 가져옵니다. local getRandomNumber =
_UtilLogic.RandomIntegerRange(1, totalValue) -- gachaRateData에서 하나씩 값을 뺍셈합니다. for i = 1,
gradeCount do getRandomNumber = getRandomNumber - self.gachaRateData[i].RateValue if
getRandomNumber <= 0 then return self.gachaRateData[i].Grade end end -- 정상적으로 값이 나오지 않았을 경
우, 에러 로그와 함께 가장 낮은 등급을 반환합니다. log_error("정상적인 랜덤 등급값을 얻는데 실패했습니다!") return
1 end

```

```

@ExecSpace("ClientOnly") method CharacterData GetRandomCharacterData() --
GetRandomCharacterByGradeValue로 선택된 랜덤 캐릭터를 반환합니다. return
self:GetRandomCharacterByGradeValue(self:GetRandomGrade()) end

```

```

@ExecSpace("ClientOnly") method CharacterData GetRandomCharacterByGradeValue(number GradeValue) --
전달받은 GradeValue값을 기준으로 해당 등급의 랜덤한 캐릭터 하나를 반환합니다. local getRandomList =
self.characterData[GradeValue]

```

```

local getRandomCharacter = _UtilLogic.RandomIntegerRange(1, #getRandomList) return
getRandomList[getRandomCharacter] end

```

```

@ExecSpace("ClientOnly") method void RequestSpawnCharacter() -- 현재 소환 토큰의 수가 충분한지 검사합니다.
if _GameLogic.summonToken >= 1 then -- 현재 소환 가능한 슬롯이 있는지 검사합니다. local getSlotNumber =
self.characterSlotGroupController:ReturnEmptySlotNumber() -- 정상적인 슬롯 번호를 받았을 경우 캐릭터를 생성
합니다. if getSlotNumber >= 1 then local getData = self:GetRandomCharacterData()
self.characterSlotGroupController:SpawnCharacter(getSlotNumber, getData) -- 소환 토큰의 값을 1 감소시키고 UI
를 갱신하도록 요청합니다. _GameLogic:UpdateSummonToken(-1) else -- 빈 캐릭터 슬롯이 없을 경우 토스트 메세
지를 출력합니다. _UIToast:ShowMessage(_LocalizationService.GetText("UI_NeedSpace")) log("현재 캐릭터 슬롯이
가득 찹니다!") end else -- 소환 토큰이 부족할 경우 토스트 메시지를 출력합니다.
_UIToast:ShowMessage(_LocalizationService.GetText("UI_NeedToken")) end

```

```

end

```

```

end

```

> 해당 코드는 PlainText 기준으로 작성된 코드입니다.

캐릭터 제어를 위한 Component 구현하기

- 캐릭터의 행동과 데이터를 가져오기 위한 UnitController, 캐릭터의 슬롯 이동을 구현하기 위한 DragComponent를 구성해야 합니다.
- 아래는 UnitController, DragComponent의 전문입니다.

```

```lua
@Component
script UnitController extends Component

property Entity avatarBodyComponent = nil

property table enemyTable = {}

property number basicAtkDelta = 0

property CharacterData characterData = CharacterData()

property DragComponent dragComponent = nil

property Entity currentSlot = nil

property number skillDelta = 0

property SkillData characterSkillData = SkillData()

property TriggerComponent triggerComponent = nil

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 아바타를 재생하기 위한 avatarBodyComponent를 찾아 등록합니다.
if self.avatarBodyComponent == nil then
 local findComponent = self.Entity.AvatarRendererComponent:GetBodyEntity()
 if isvalid(findComponent) then
 self.avatarBodyComponent = findComponent
 end
end
-- 자식 엔티티로 dragComponent를 찾아 등록합니다.
if not isvalid(self.dragComponent) then
 local findComponent = self.Entity:GetChildByName("DragEntity", true)
 if isvalid(findComponent) then
 self.dragComponent = findComponent.DragComponent
 end
end
-- dragComponent의 초기화 및 엔티티 비활성화를 수행합니다.
self:InitDragComponentSprite()
self.Entity.Enable = false

```



```

-- triggerComponent가 비어있을 경우 이를 초기화합니다.
if not isvalid(self.triggerComponent) then
 -- 자기 자신에게서 TriggerComponent를 찾습니다.
 local findComponent = self.Entity.TriggerComponent
 -- 자신의 TriggerComponent를 찾았다면 이를 triggerComponent에 등록합니다.
 if isvalid(findComponent) then
 self.triggerComponent = findComponent
 end
end
end

@ExecSpace("ClientOnly")
method void InitCharacterData(CharacterData Data, Entity SlotEntity)
 -- 캐릭터를 활성화시킵니다.
 self.Entity.Enable = true
 -- 캐릭터에 대한 정보를 받아 등록합니다.
 self.characterData = Data
 -- 캐릭터의 스킬 정보를 찾아 등록합니다.
 self.characterSkillData =
 _SkillDataLogic:GetSkillDataBySkillID(self.characterData.SkillData)
 -- 현재 슬롯을 전달받은 SlotEntity로 등록합니다.
 self.currentSlot = SlotEntity
 -- 공격 범위를 전달받은 Data 기준으로 갱신합니다.
 self.triggerComponent.CircleRadius = self.characterData.AttackRange
end

@ExecSpace("ClientOnly")
method void InitDragComponentSprite()
 -- DragEntity의 아바타를 현재 사용중인 캐릭터의 아바타와 동일하게 만들기 위해 값들을 초기화합
 니다.
 local currentSprite = self.Entity.CostumeManagerComponent
 local dragComponent =
 self.Entity:GetChildByName("DragEntity").CostumeManagerComponent
 dragComponent.CustomBodyEquip = currentSprite.CustomBodyEquip
 dragComponent.CustomCapeEquip = currentSprite.CustomCapeEquip
 dragComponent.CustomCapEquip = currentSprite.CustomCapEquip
 dragComponent.CustomCoatEquip = currentSprite.CustomCoatEquip
 dragComponent.CustomEarAccessoryEquip = currentSprite.CustomEarAccessoryEquip
 dragComponent.CustomEarEquip = currentSprite.CustomEarEquip
 dragComponent.CustomEyeAccessoryEquip = currentSprite.CustomEyeAccessoryEquip
 dragComponent.CustomFaceAccessoryEquip = currentSprite.CustomFaceAccessoryEquip
 dragComponent.CustomFaceEquip = currentSprite.CustomFaceEquip
 dragComponent.CustomGloveEquip = currentSprite.CustomGloveEquip
 dragComponent.CustomHairEquip = currentSprite.CustomHairEquip
 dragComponent.CustomLongcoatEquip = currentSprite.CustomLongcoatEquip
 dragComponent.CustomOneHandedWeaponEquip =
 currentSprite.CustomOneHandedWeaponEquip
 dragComponent.CustomPantsEquip = currentSprite.CustomPantsEquip
 dragComponent.CustomShoesEquip = currentSprite.CustomShoesEquip
 dragComponent.CustomSubWeaponEquip = currentSprite.CustomSubWeaponEquip
 dragComponent.CustomTwoHandedWeaponEquip =
 currentSprite.CustomTwoHandedWeaponEquip
end

```

```

@ExecSpace("ClientOnly")
method void ChangeSlot(Entity SlotEntity)
-- 현재 슬롯을 전달받은 슬롯으로 변경합니다.
self.currentSlot = SlotEntity
-- 현재 캐릭터의 위치를 전달받은 슬롯의 위치로 초기화합니다.
self.Entity.TransformComponent.WorldPosition =
SlotEntity.TransformComponent.WorldPosition
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- 델타타임은 조건 없이 항상 더해줍니다.
self.skillDelta = self.skillDelta + delta
self.basicAtkDelta = self.basicAtkDelta + delta

-- 스킬 공격 쿨타임을 확인하여 조건을 충족할 경우 스킬 공격을 수행합니다.
if self.skillDelta >= self.characterSkillData.SkillCoolTime then
 self:SkillAttack()
end

-- 만약 스킬 공격이 성공하여 쿨타임이 0으로 초기화되었다면, 이번 프레임엔 기본 공격을 쉬어갑니
다.
if self.skillDelta == 0 then
 return
end

-- 기본 공격 쿨타임을 확인하여 조건을 충족할 경우 기본 공격을 수행합니다.
if self.basicAtkDelta >= self.characterData.BasicAttackCoolTime then
 self:BasicAttack()
end
end

@ExecSpace("ClientOnly")
method void BasicAttack()
-- 캐릭터의 StateComponent를 가져옵니다.
local bodySelector = self.Entity.StateComponent
-- 캐릭터의 현재 상태 이름을 가져옵니다.
local currentState = bodySelector.CurrentStateName
-- 현재 상태가 IDLE이 아닐 경우, 기본 공격 수행을 하지 않습니다.
if currentState ~= "IDLE" then
 return
end

---@type Entity
local targetEntity = nil
-- 테이블을 역순으로 순회하며 유효하지 않은 엔티티를 안전하게 제거합니다.
for i = #self.enemyTable, 1, -1 do
 local enemy = self.enemyTable[i]
 -- 엔티티가 아직 월드에 존재하는지 확인합니다.
 if isvalid(enemy) then
 -- 엔티티에 MonsterController가 있고 살아있는지 확인합니다.
 if enemy.MonsterController and enemy.MonsterController.isAlive then
 targetEntity = enemy
 break
 end
 end
end

```

```

 else
 -- 이미 파괴된 엔티티라면 테이블에서 제거합니다.
 table.remove(self.enemyTable, i)
 end
 end
end
-- 만약 지정된 targetEntity가 사용 불가능할 경우 공격을 수행하지 않습니다.
if not isValid(targetEntity) then
 return
end
-- 공격을 수행했다 가정하여 basicAtkDelta값을 0으로 설정합니다.
self.basicAtkDelta = 0
-- 공격 애니메이션을 출력시키기 위해 event를 생성, 전달합니다.
local event = ActionStateChangedEvent(self.characterData.AttackAnimationID,
self.characterData.AttackAnimationID, 1, SpriteAnimClipPlayType.Onetime)
self.avatarBodyComponent:SendEvent(event)
-- 출력해야 하는 캐릭터쪽 이펙트가 있을 경우 이를 출력합니다.
if not _UtilLogic:IsNilorEmptyString(self.characterData.BasicAttackEffectID) then
 _EffectService:PlayEffect(self.characterData.BasicAttackEffectID, self.Entity,
self.Entity.TransformComponent.WorldPosition, 0, Vector3.one)
end
-- 적에게 출력해야 하는 이펙트가 있을 경우 이를 출력합니다.
if not _UtilLogic:IsNilorEmptyString(self.characterData.BasicAttackHitEffectID)
then
 _EffectService:PlayEffect(self.characterData.BasicAttackHitEffectID,
targetEntity, targetEntity.TransformComponent.WorldPosition, 0, Vector3.one)
end
-- 타겟이 된 적의 MonsterController를 통해 GetDamage를 수행하도록 요청합니다.
targetEntity.MonsterController:GetDamage(self.characterData.BasicAttackValue)
end

@ExecSpace("ClientOnly")
method void SkillAttack()
 -- 애니메이션 재생을 위한 bodySelector를 찾아 등록합니다.
 local bodySelector = self.Entity.StateComponent
 -- 현재 상태를 찾기 위해 CurrentStateName을 등록합니다.
 local currentState = bodySelector.CurrentStateName
 -- 현재 상태가 IDLE이 아닐 경우 코드 실행을 중단합니다.
 if currentState ~= "IDLE" then
 return
 end
 ---@type Entity
 local targetEntity = nil
 -- 테이블을 역순으로 순회하며 유효하지 않은 엔티티를 안전하게 제거합니다.
 for i = #self.enemyTable, 1, -1 do
 local enemy = self.enemyTable[i]
 if isValid(enemy) then
 if enemy.MonsterController and enemy.MonsterController.isAlive then
 targetEntity = enemy
 break
 end
 else
 table.remove(self.enemyTable, i)
 end
 end
end
end

```

```

-- 만약 targetEntity가 nil일 경우 코드 실행을 중단합니다.
if not isValid(targetEntity) then
 return
end
-- 스킬 사용을 성공했다 가정하여 skillDelta값을 0으로 초기화합니다.
self.skillDelta = 0
-- 공격 애니메이션을 재생합니다.
local event = ActionStateChangedEvent(self.characterData.AttackAnimationID,
self.characterData.AttackAnimationID, 1, SpriteAnimClipPlayType.Onetime)
self.avatarBodyComponent:SendEvent(event)
-- 스킬 사용시 자기 자신에게 출력해야 하는 이펙트가 있을 경우 이펙트를 출력합니다.
if not _UtilLogic:IsNilorEmptyString(self.characterSkillData.CharacterSkillEffect)
then
 _EffectService:PlayEffect(self.characterSkillData.CharacterSkillEffect,
self.Entity, self.Entity.TransformComponent.WorldPosition, 0, Vector3.one)
end
-- 피격 대상에게 출력해야 하는 이펙트가 있을 경우 이펙트를 출력합니다.
if not _UtilLogic:IsNilorEmptyString(self.characterSkillData.SkillEffect) then
 _EffectService:PlayEffect(self.characterSkillData.SkillEffect, targetEntity,
targetEntity.TransformComponent.WorldPosition, 0, Vector3.one)
end
-- 대상이 된 targetEntity의 MonsterContorller에게 GetDamage를 수행하도록 요청합니다.
targetEntity.MonsterController:GetDamage(self.characterSkillData.SkillAttack)
end

@ExecSpace("ClientOnly")
@EventSender("Self")
handler HandleTriggerEnterEvent(TriggerEnterEvent event)
-- 충돌 범위에 들어온 엔티티가 CollisionGroups, Monster일 경우 이를 등록합니다.
if event.TriggerBodyEntity.TriggerComponent.CollisionGroup ==
CollisionGroups.Monster then
 if #self.enemyTable >= 1 then
 local isAlreadyHave = false
 for i = 1, #self.enemyTable do
 if self.enemyTable[i] == event.TriggerBodyEntity then
 isAlreadyHave = true
 break
 end
 end
 if isAlreadyHave == false then
 table.insert(self.enemyTable, event.TriggerBodyEntity)
 end
 else
 table.insert(self.enemyTable, event.TriggerBodyEntity)
 end
end
end

@ExecSpace("ClientOnly")
@EventSender("Self")
handler HandleTriggerLeaveEvent(TriggerLeaveEvent event)
-- 충돌 범위를 벗어난 엔티티가 Monster일 경우 공격 리스트에서 제거합니다.
if event.TriggerBodyEntity.TriggerComponent.CollisionGroup ==
CollisionGroups.Monster then

```

```

if #self.enemyTable >= 1 then
 local listCount = #self.enemyTable
 for i = listCount, 1, -1 do
 if self.enemyTable[i] == event.TriggerBodyEntity then
 table.remove(self.enemyTable, i)
 break
 end
 end
end
end
end
end
end

```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

```
@Component
script DragComponent extends Component

@DisplayName("클릭 입력 리스트")
property table Index = {}

@DisplayName("Hold Handler")
@Description("클릭 유지 감지 체크")
property any Handler = nil

@DisplayName("Release Handler")
@Description("클릭 해제 감지 체크")
property any ReleaseHandler = nil

property Entity slotEntity = nil

property Entity parentEntity = "4bc00000-b6fa-481d-a049-aed3dea2d411"

property CharacterSlotGroupComponent characterSlotGroup = nil

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- characterSlotGroup을 초기화합니다.
self:InitCharacterSlotGroup()
end

@ExecSpace("ClientOnly")
method void InitCharacterSlotGroup()
-- characterSlotGroup이 nil인지 검사합니다.
if not isValid(self.characterSlotGroup) then
-- characterSlotGroup이 nil일 경우, 현재 맵의 여부를 판단합니다.
if self.Entity.Parent.CurrentMapName == "InGame" then
-- 현재 맵이 InGame일 경우 InGame에 있는 CharacterSlotPosGroup을 찾습니다.
local findEntity =
_EntityService:GetEntityByPath("/maps/InGame/CharacterSlotPosGroup")
-- 정상적으로 찾았는지 검증합니다.
```

```

 if isValid(findEntity) then
 -- 정상적으로 엔티티를 찾았다면 characterSlotGroup에 등록합니다.
 self.characterSlotGroup = findEntity.CharacterSlotGroupComponent
 end
 end
end
end

@ExecSpace("ClientOnly")
method void Move(any screenholdEvent)
 -- 현재 Hold상태의 Id값을 가져옵니다.
 local touchId = screenholdEvent.TouchId
 -- 현재 Hold의 위치값을 가져옵니다.
 local touchPoint = screenholdEvent.TouchPoint
 -- 클릭 인덱스 전체를 탐색합니다.
 for i = 1, #self.Index do
 -- 현재 홀드중인 Id와 동일한 Index[touchId]가 있을 경우 엔티티의 위치를 현재 Hold의 위치값으로 이동합니다.
 if self.Index[i] == touchId then
 self.Entity.TransformComponent.WorldPosition =
 _UILogic:ScreenToWorldPosition(touchPoint):ToVector3()
 break
 end
 end
end
end

@ExecSpace("ClientOnly")
method void AddHandler(integer touchId)
 -- 이미 Handler가 존재하면 중복 처리를 방지하기 위해 코드 실행을 중단합니다.
 if self.Handler ~= nil then
 return
 end

 -- 이미 다른 캐릭터를 드래그 중이라면 무시하여 다중 드래그를 방지합니다.
 if isValid(self.characterSlotGroup) then
 if self.characterSlotGroup.isDragging == true then
 return
 end
 end

 -- 드래그 시작 시 다른 슬롯들이 반응하지 못하도록 잠금 처리합니다.
 self.characterSlotGroup.isDragging = true
end

 -- 현재 DragEntity가 가려진 상태라면 활성화합니다.
 if self.Entity.Visible == false then
 self.Entity.Visible = true
 end
 table.insert(self.Index, touchId)

 -- Hold뿐만 아니라 Release도 화면 전체에서 감지하도록 등록합니다.
 self.Handler = _InputService:ConnectEvent(ScreenTouchHoldEvent, self.Move)
 self.ReleaseHandler = _InputService:ConnectEvent(ScreenTouchReleaseEvent,
 self.OnRelease)
end

```

```

@ExecSpace("ClientOnly")
method void OnRelease(any event)
local TouchId = event.TouchId
-- 드래그가 해제되었으므로 모든 TouchId를 탐색하여 해제합니다.
for i = 1, #self.Index do
 if self.Index[i] == TouchId then
 table.remove(self.Index, i)
 if #self.Index <= 0 then
 -- 추가한 Hold와 Release 이벤트를 해제합니다.
 _InputService:DisconnectEvent(ScreenTouchHoldEvent, self.Handler)
 _InputService:DisconnectEvent(ScreenTouchReleaseEvent,
self.ReleaseHandler)
 self.Handler = nil
 self.ReleaseHandler = nil

 -- 드래그가 끝났으므로 잠금을 해제합니다.
 if invalid(self.characterSlotGroup) then
 self.characterSlotGroup.isDragging = false
 end

 -- 드롭한 위치를 가져옵니다.
 local dropPos = self.Entity.TransformComponent.WorldPosition

 -- 현재 위치에서 가장 가까운 타겟 슬롯을 탐색을 위해 현재 슬롯 번호를 중복이 없는
 -1로 설정합니다.
 local targetSlotNumber = -1
 -- 슬롯으로 인정할 최대 반경 거리
 local closestDist = 1.5
 -- characterSlotGroup과 characterSlotGroup의 characterSlotList가 사용 가
 능한지 점검합니다.
 if invalid(self.characterSlotGroup) and
self.characterSlotGroup.characterSlotList then
 -- for문을 돌면서 characterSlotList를 탐색합니다.
 for slotIndex = 1, #self.characterSlotGroup.characterSlotList do
 -- slotIndex의 slotEntity를 가져옵니다.
 local slotEntity =
self.characterSlotGroup.characterSlotList[slotIndex].Entity
 -- slotPos의 slotEntity의 worldPosition을 가져옵니다.
 local slotPos = slotEntity.TransformComponent.WorldPosition
 -- dragEntity인 자기 자신과 슬롯의 거리를 계산합니다.
 local dist = Vector2.Distance(dropPos:ToVector2(),
slotPos:ToVector2())
 -- 앞서 지정한 closetDist보다 거리가 적을 경우 범위에 들어왔다고 판단
 합니다.

 if dist < closestDist then
 -- 조건을 충족한 Slot의 거리로 설정합니다.
 closestDist = dist
 -- 조건을 충족한 slotIndex를 targetSlotNumber로 등록합니
 다.

 targetSlotNumber = slotIndex
 end
 end
 end
 end
 end
 end
 end
end

```

```

 -- 기존의 슬롯 번호를 가져옵니다.
 local beforeSlotNumber =
self.parentEntity.UnitController.currentSlot.CharacterSlotComponent:GetSlotNumber(
)
 -- targetSlotNumber가 -1이 아니고, targetSlotNumber가 beforeSlotNumber와
다를 경우 이동을 수행합니다.
 if targetSlotNumber ~= -1 and targetSlotNumber ~= beforeSlotNumber
then
 self.characterSlotGroup:SwitchSlotCharacter(targetSlotNumber,
self.parentEntity, beforeSlotNumber)
 end
 end
 break
 end
end

-- 이동 처리가 끝난 후 원래 위치로 복귀 및 숨김처리를 수행합니다.
self.Entity.TransformComponent.WorldPosition =
self.parentEntity.TransformComponent.WorldPosition
self.Entity.Visible = false
end

@ExecSpace("ClientOnly")
@EventSender("Self")
handler HandleTouchEvent(TouchEvent event)
-- 입력 받은 event의 TouchId 값을 가져옵니다.
local TouchId = event.TouchId
-- AddHandler를 통해 필요한 이벤트를 등록합니다.
self:AddHandler(TouchId)
end

end

```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

## 최소 구현 기준

- 사용할 수 있는 Model 형태로 캐릭터를 제작합니다.
- Model로 제작된 캐릭터를 사용할 수 있도록 기능을 구현합니다.
- 등급에 따라 캐릭터가 생성될 수 있도록 등급별 1종 이상 캐릭터를 제작합니다.

## 응용 및 확장 방향

- 캐릭터의 소환 확률을 다른 방식을 사용하여 적용합니다.
- 캐릭터가 공격을 수행할 때 효과음이 재생될 수 있도록 기능을 구현합니다.
- 각 캐릭터마다 공격시 재생되는 애니메이션의 속도를 공격 속도에 맞춰 조절하는 기능을 추가합니다.



## [💡 기초 학습] 몬스터 제작하기

---

### 영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 맵 타입 이해하기: 00:47:58

### 학습 목표

- 몬스터의 정보를 관리하기 위한 DataSet을 제작하는 방법을 학습하고 제작합니다.
- Logic을 통해 몬스터의 정보를 불러올 수 있도록 기능을 구현합니다.
- Model을 활용하여 하나의 동일한 몬스터를 여러 마리로 소환할 수 있도록 기능을 구성합니다.
- 몬스터가 지정한 경로를 이동할 수 있도록 기능을 구현합니다.

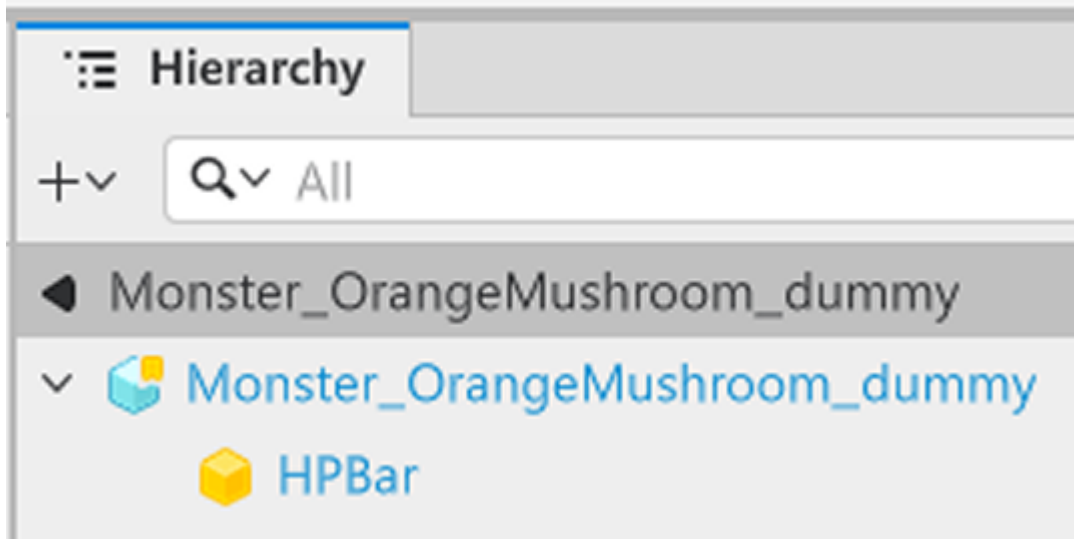
### 해당 영상 시청 후 수행해 볼 내용

- 다양한 몬스터를 사용할 수 있도록 몬스터의 Model을 제작합니다.
- 각 몬스터들이 정보를 가질 수 있도록 DataSet을 확장 또는 추가합니다.
- 제작한 맵에 맞춰 몬스터가 경로를 이동할 수 있도록 TweenLine을 수정합니다.

### 몬스터의 구성 이해하기

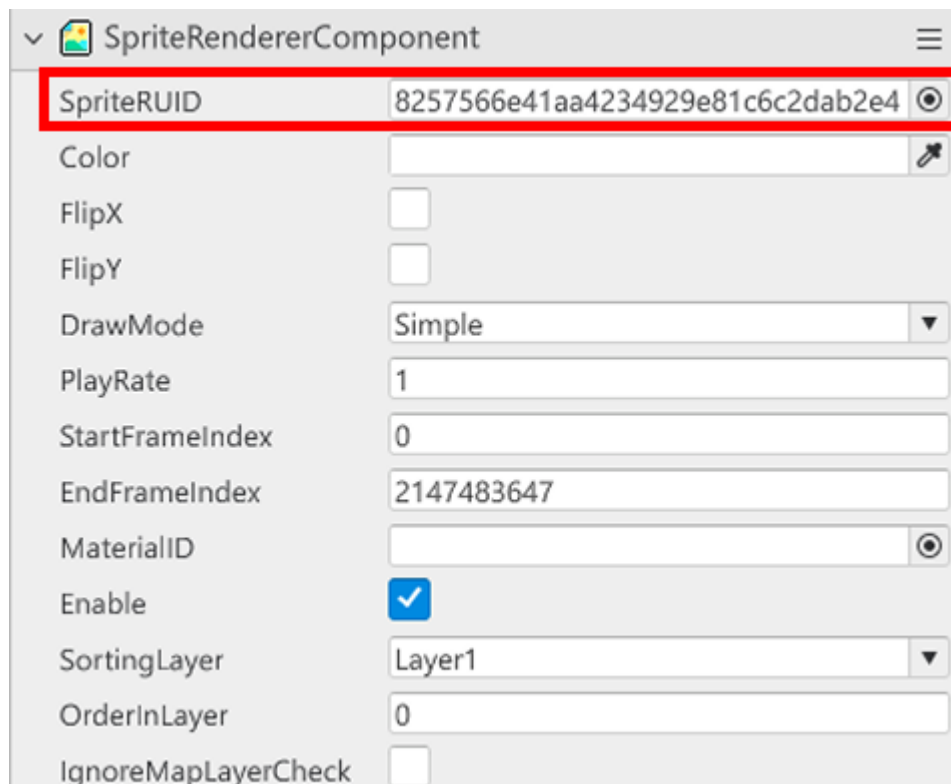
- 샘플 월드에서 몬스터가 하는 역할은 총 2가지입니다.
  1. 지정된 경로를 따라 이동하기
  2. 자신이 처치될 때 소환 재화를 지급하기
- 따라서 몬스터는 체력과 이동, 그리고 피격과 사망에 대한 처리만으로 가볍게 구현할 수 있습니다.

### 몬스터 엔티티 제작하기

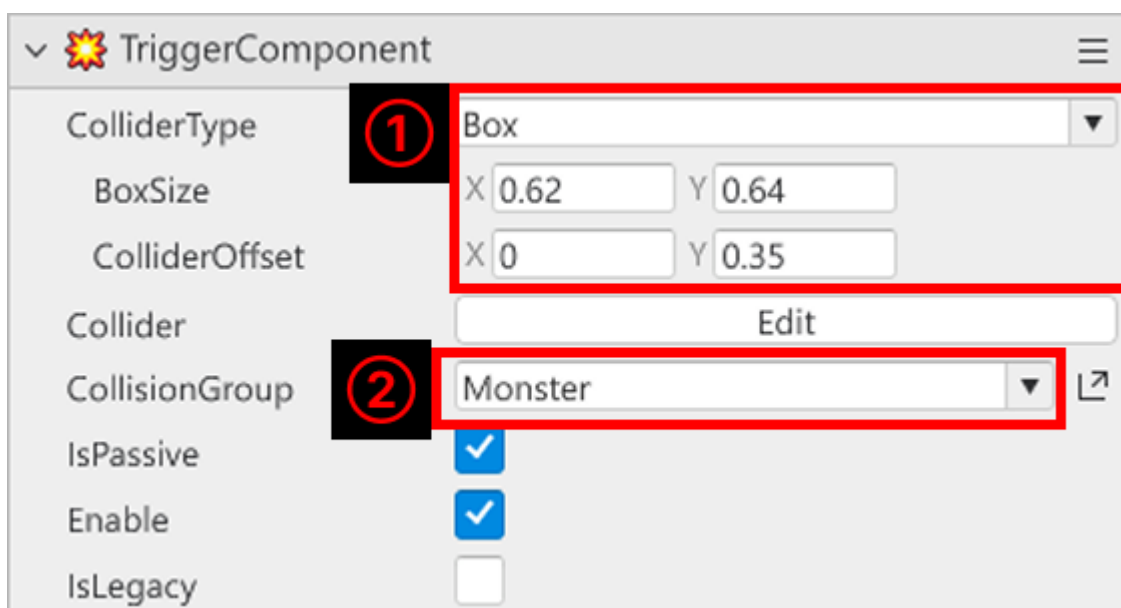


- 샘플 월드의 몬스터 Model은 2개의 엔티티를 가지고 있습니다.
  - Monster\_몬스터 이름 엔티티: 몬스터의 이미지 출력, 기능 처리를 담당하는 엔티티입니다.
  - HPBar: 몬스터의 남은 체력을 시각적으로 보여주기 위한 엔티티입니다.
  - Monster\_엔티티에 다음의 Component를 부여합니다.
- **SpriteRendererComponent**
  - 몬스터의 애니메이션을 재생하기 위한 Component입니다.
- **StateComponent**
  - 몬스터의 상태를 확인하기 위한 Component입니다.
- **DamageSkinComponent**
  - 몬스터가 캐릭터에게 피격될 때 데미지를 출력하기 위한 Component입니다.
- **TriggerComponent**
  - 캐릭터의 공격 범위에 들어왔는지 판단 기준이 될 Collider Component입니다.
- **TweenLineComponent**
  - 몬스터가 지정된 경로로 이동하게 만드는 Component입니다.
- **MonsterController**
  - 직접 제작한 Component입니다.
  - 몬스터의 체력, 생존 여부 등을 처리하는 Component입니다.
- 이어서 HPBar 엔티티에는 **SpriteRendererComponent**를 추가합니다.
- 모든 Component의 추가가 완료되었다면 엔티티를 Model로 제작합니다.

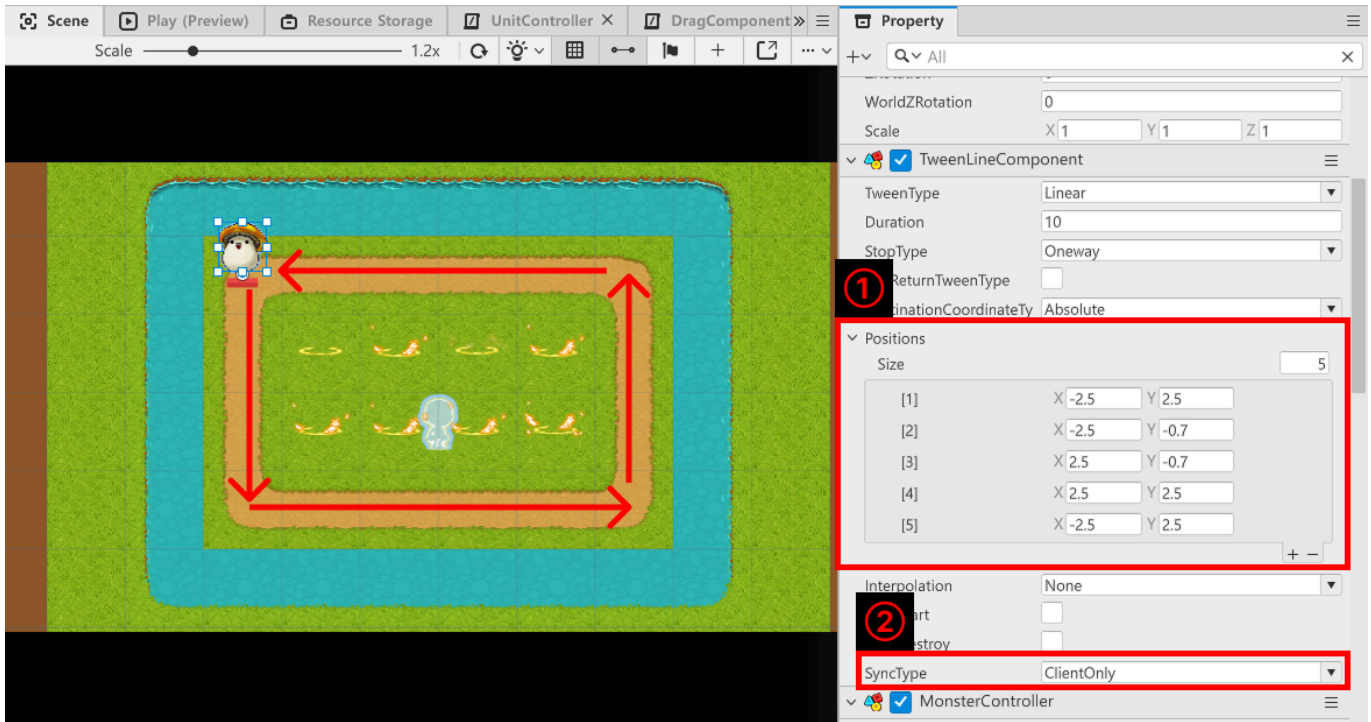
## Component 편집하기



- `SpriteRendererComponent`의 `SpriteRUID`에 값을 입력하여 애니메이션을 재생할 수 있습니다.
- 메이플스토리 월드가 제공하는 리소스 또는 직접 구한 리소스를 사용하여 몬스터 애니메이션을 재생합니다.
- 또한 HPBar의 `SpriteRendererComponent`를 체력표기를 위한 리소스로 `SpriteRUID`를 등록합니다.



- `TriggerComponent`의 요소인 `ColliderType`을 Box로 변경합니다.
- `BoxSize`를 애니메이션 리소스에 맞춰 수정합니다.
- `CollisionGroup`을 Monster로 전환하여 `CharacterAttackRange`와 충돌이 일어나도록 수정합니다.



- TweenLineComponent의 Positions를 설정하여 몬스터가 맵을 지정한 경로로 이동하도록 설정합니다.
- SyncType을 ClientOnly로 변경하여 클라이언트 환경에서 Component의 업데이트가 이뤄지도록 변경합니다.

### ⚠ TriggerComponent의 IsPassive

- TriggerComponent를 통한 충돌 연산은 연산 비용이 많이 드는 작업입니다.
- 모든 객체가 충돌을 감지하게 될 경우, 연산 비용이 크게 증가합니다.
- IsPassive를 활성화 할 경우, IsPassive가 활성화 된 객체는 충돌 연산을 실행하지 않습니다.
- 그러나 자기 자신과 충돌을 감지하는 다른 Collision Group은 IsPassive로 설정된 객체를 감지할 수 있습니다.
- 샘플 월드와 같이 충돌 감지가 1 : 1의 형식을 가질 경우, 다른 한 쪽을 IsPassive로 변경하여 연산 비용을 아낄 수 있습니다.

## 몬스터 Data 제작하기

- 몬스터를 관리하기 위한 Data와 이를 사용하기 위한 Logic, 그리고 값들을 하나의 요소로 관리할 수 있도록 StructType을 제작해야 합니다.
- 샘플 월드에서는 MonsterData DataSet, MonsterSpawnLogic, MonsterData StructType을 제작합니다.

### MonsterData DataSet 제작하기

- MonsterData DataSet은 다음과 같은 구조를 가지고 있습니다.
- MonsterID
  - 몬스터를 구분하기 위한 값입니다.
- OneCycleMoveTime

- 몬스터 엔티티에 부여한 TweenLineComponent가 몇 초에 걸쳐 목적지에 도달하게 만들 것 인지 설정하는 값입니다.
- StartHP
  - 몬스터가 생성될 때 초기화하는 HP 값입니다.
- ModelID
  - 앞서 제작한 몬스터 Model의 Entry ID를 등록하여 사용하기 위한 값입니다.
- 해당 값들을 기반으로 DataSet을 구성하여 제작합니다.
- DataSet의 형식을 사용하기 위해 StructType을 제작합니다.

## 몬스터 Component 구성하기

- 몬스터의 기능들을 구현하기 위해 MonsterController를 작성해야 합니다.
- 아래는 MonsterController의 코드 전문입니다.

```
@Component
script MonsterController extends Component

property TweenLineComponent tweenLineComponent = nil

property string moveAnimationKey = "8257566e41aa4234929e81c6c2dab2e4"

property string dieAnimationKey = "65a2bfb039614f2e9e4ccc354340153d"

property SpriteRendererComponent monsterSpriteRenderer = nil

property boolean isAlive = false

property MonsterData monsterData = MonsterData()

property number currentHP = 0

property Entity hpBar = nil

property MonsterSpawnController spawnController = nil

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- tweenLineComponent가 nil인지 점검합니다.
if not isvalid(self.tweenLineComponent) then
-- 만약 nil일 경우 자기 자신에 할당된 TweenLineComponent를 찾습니다.
local findEntity = self.Entity.TweenLineComponent
-- 정상적으로 Component를 찾았다면 이를 tweenLineComponent에 등록합니다.
if isvalid(findEntity) then
self.tweenLineComponent = findEntity
else
-- 정상적으로 Component를 찾지 못했으므로 에러 로그를 출력합니다.
log_error("Monster의 TweenLine이 nil입니다!")
return
end
```

```

end
-- monsterSpriteRenderer가 nil인지 체크합니다.
if not invalid(self.monsterSpriteRenderer) then
 -- 자기 자신의 Component중 SpriterRenderer를 찾아 등록합니다.
 self.monsterSpriteRenderer = self.Entity.SpriteRendererComponent
end

-- 만약 hpBar가 nil일 경우 이를 찾아 등록합니다.
if not invalid(self.hpBar) then
 local findEntity = self.Entity.GetChildByName("HPBar")
 if invalid(findEntity) then
 self.hpBar = findEntity
 end
end

self.isAlive = true
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
 -- 만약 몬스터가 한 바퀴를 돌았을 경우 다시 출발하도록 요청합니다.
 if self.tweenLineComponent.CurrentState == TweenState.Idle then
 self.tweenLineComponent.RestartFromCurrentPosition()
 end
end

@ExecSpace("ClientOnly")
method void InitData(MonsterData MonsterData, Vector3 StartPos,
 MonsterSpawnController MonsterSpawnController)
 -- 자신의 monsterData를 전달받은 MonsterData로 초기화합니다.
 self.monsterData = MonsterData
 -- tweenLineComponent의 Duration을 전달받은 값으로 초기화합니다.
 self.tweenLineComponent.Duration = self.monsterData.OneCycleMoveTime
 -- 현재 체력을 전달받은 HP로 초기화합니다.
 self.currentHP = self.monsterData.StartHP
 -- 현재 상태를 활성화로 전환합니다.
 self.isAlive = false
 self.Enable = false
 -- tweenLine을 최초 상태로 초기화합니다.
 self.tweenLineComponent.Stop(true)
 -- 현재 위치를 전달받은 위치로 수정합니다.
 self.Entity.TransformComponent.WorldPosition = StartPos
 -- 현재 hp에 맞춰 몬스터 체력바를 업데이트합니다.
 self.hpBar.TransformComponent.Scale.x = self.currentHP / self.monsterData.StartHP
 * 20
 -- MonsterSpawnController를 등록합니다.
 self.spawnController = MonsterSpawnController
end

@ExecSpace("ClientOnly")
method void SetEnableState()
 -- 몬스터가 행동할 수 있도록 필요한 값들을 초기화합니다.
 self.tweenLineComponent.Stop(true)
 self.tweenLineComponent.Play()

```

```

self.currentHP = self.monsterData.StartHP
self.monsterSpriteRenderer.SpriteRUID = self.moveAnimationKey
self.isAlive = true
self.Enable = true
end

@ExecSpace("ClientOnly")
method void GetDamage(number AtkValue)
-- 받은 데미지를 표기합니다.
_DamageSkinService:Play(self.Entity, "3271c3e79bf04ecba9a107d55495970d", 0.05,
{AtkValue}, DamageSkinTweenType.Default, false)
-- 현재 체력에서 전달받은 데미지만큼 감소시킵니다.
self.currentHP = self.currentHP - AtkValue
-- 만약 현재 체력이 0 이하일 경우 처치 관련 처리를 진행합니다.
if self.currentHP <= 0 then
-- 오버플로우 방지를 위해 0으로 초기화합니다.
self.currentHP = 0
-- 현재 생존 여부를 false로 변경합니다.
self.isAlive = false
-- 몬스터의 이동을 중단시킵니다.
self.tweenLineComponent:Pause()
-- 몬스터의 애니메이션을 사망으로 변경합니다.
self.monsterSpriteRenderer.SpriteRUID = self.dieAnimationKey
-- HPBar의 크기를 0으로 초기화합니다.
self.hpBar.TransformComponent.Scale.x = 0
-- 자기 자신이 처치되었으므로 MonsterSpawnManager에게 잔존 수를 감소하도록 요청합니다.
self.spawnController:UpdateRemainMonsterCount(-1)
-- GameLogic에게 소환 토큰을 1 증가시키도록 요청합니다.
_GameLogic:UpdateSummonToken(1)
else
-- 이외의 경우에는 설정한 크기에 맞춰 UI가 변경되도록 계산한 값을 적용합니다.
self.hpBar.TransformComponent.Scale.x = self.currentHP /
self.monsterData.StartHP * 20
end
end

@EventSender("Self")
handler HandleSpriteAnimPlayerEndFrameEvent(SpriteAnimPlayerEndFrameEvent event)
-- 몬스터 사망 애니메이션이 끝까지 출력되었다면 엔티티를 비활성화 시킵니다.
if self.monsterSpriteRenderer.SpriteRUID == self.dieAnimationKey then
self.Entity.Enable = false
end
end

end

```

해당 코드는 Plain Text를 기준으로 작성한 코드입니다.

## 몬스터의 정보를 불러오기 위한 Logic 제작하기

- 몬스터의 정보를 관리하고, 몬스터의 스폰을 관리하는 MonsterSpawnLogic을 제작합니다.

- MonsterSpawnLogic은 Logic Script로 제작된 Script입니다.
- 아래는 MonsterSpawnLogic의 코드 전문입니다.

```

@Logic
script MonsterSpawnLogic extends Logic

property table monsterData = {}

property table waveData = {}

@ExecSpace("ClientOnly")
method void OnBeginPlay()
self:InitMonsterData()
self:InitWaveData()
end

@ExecSpace("ClientOnly")
method void InitMonsterData()
-- monsterData가 사용 불가능한 상태인지 점검합니다.
if not isvalid(self.monsterData) or not next(self.monsterData) then
-- MonsterData DataSet이 정상적으로 있는지 확인합니다.
local findDataSet = _DataService:GetTable("MonsterData")
-- 만약 불러오는데 실패했다면 에러 로그를 남깁니다.
if not isvalid(findDataSet) then
log_error("MonsterData DataSet을 불러오는데 실패했습니다.")
return
end
-- 정상적으로 찾았다면 rowCount를 가져옵니다.
local rowCount = findDataSet:GetRowCount()
-- rowCount만큼 반복하면서 데이터를 순서대로 찾아 등록합니다.
for i = 1, rowCount do
local monsterID = tostring(findDataSet:GetCell(i, "MonsterID"))
-- MonsterID가 정상적으로 있는지 점검합니다.
if not _UtilLogic:IsNilorEmptyString(monsterID) then
-- MonsterID가 정상적으로 존재한다면 정상적인 값이라고 판단하여 데이터 불러오기
를 시작합니다.
local getData = MonsterData()
getData.MonsterID = monsterID
getData.OneCycleMoveTime = tonumber(findDataSet:GetCell(i,
"OneCycleMoveTime"))
getData.StartHP = tonumber(findDataSet:GetCell(i, "StartHP"))
getData.ModelID = tostring(findDataSet:GetCell(i, "ModelID"))
if not isvalid(self.monsterData[monsterID]) then
self.monsterData[monsterID] = {}
end
self.monsterData[monsterID] = getData
end
end
end
end

@ExecSpace("ClientOnly")

```



```

method void InitWaveData()
-- waveData가 사용 가능한 상태인지 점검합니다.
if not invalid(self.waveData) or not next(self.waveData) then
 local findDataSet = _DataService:GetTable("WaveData")
 -- 정상적으로 DataSet을 찾았는지 확인합니다.
 if invalid(findDataSet) then
 -- 정상적으로 데이터가 존재할 경우 rowCount를 가져옵니다.
 local rowCount = findDataSet:GetRowCount()
 -- rowCount 수 만큼 반복하여 데이터를 탐색합니다.
 for i = 1, rowCount do
 local getData = WaveData()
 getData.WaveIndex = tonumber(findDataSet:GetCell(i, "WaveIndex"))
 getData.SpawnMonster = tostring(findDataSet:GetCell(i,
"SpawnMonster"))
 getData.SpawnMonsterCount = tonumber(findDataSet:GetCell(i,
"SpawnMonsterCount"))
 if not invalid(self.waveData[i]) then
 self.waveData[i] = {}
 end
 self.waveData[i] = getData
 end
 end
end
end

@ExecSpace("ClientOnly")
method MonsterData GetMonsterDataByMonsterID(string MonsterID)
-- MonsterID를 Key값으로 가지는 데이터가 있는지 확인합니다.
if invalid(self.monsterData[MonsterID]) then
 -- 정상적으로 데이터가 존재할 경우 해당 데이터를 반환합니다.
 return self.monsterData[MonsterID]
end
-- 만약 데이터가 없을 경우, 에러 로그와 함께 nil을 반환합니다.
log_warning(MonsterID.."is Nil Data!")
return nil
end

@ExecSpace("ClientOnly")
method WaveData GetSpawnList(integer WaveIndex)
-- WaveIndex 값이 있는지 확인합니다.
if invalid(self.waveData[WaveIndex]) then
 -- 정상적으로 값이 있을 경우 해당 Data를 반환합니다.
 return self.waveData[WaveIndex]
end
end
end
end

```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

- 몬스터의 정보를 불러오고 사용하기 위한 Logic과 Component를 구성합니다.
- 저장된 DataSet의 정보대로 몬스터의 정보가 파싱되어야 합니다.
- 몬스터의 이동 경로가 제작한 맵에 맞춰 이동해야 합니다.

## 응용 및 확장 방향

- 몬스터의 등급을 부여하고 이를 사용할 수 있도록 구조를 변경합니다.
- 다양한 몬스터가 스폰될 수 있도록 몬스터의 종류를 확장합니다.

---

## [💡 기초 학습] 캐릭터의 공격 구현하기

---

### 영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 맵 타입 이해하기: ``lua @Component script MonsterController extends Component

```
property TweenLineComponent tweenLineComponent = nil
```

```
property string moveAnimationKey = "8257566e41aa4234929e81c6c2dab2e4"
```

```
property string dieAnimationKey = "65a2bfb039614f2e9e4ccc354340153d"
```

```
property SpriteRendererComponent monsterSpriteRenderer = nil
```

```
property boolean isAlive = false
```

```
property MonsterData monsterData = MonsterData()
```

```
property number currentHP = 0
```

```
property Entity hpBar = nil
```

```
property MonsterSpawnController spawnController = nil
```

```
@ExecSpace("ClientOnly") method void OnBeginPlay() -- tweenLineComponent가 nil인지 점검합니다. if not
isvalid(self.tweenLineComponent) then -- 만약 nil일 경우 자기 자신에 할당된 TweenLineComponent를 찾습니다.
local findEntity = self.Entity.TweenLineComponent -- 정상적으로 Component를 찾았다면 이를
tweenLineComponent에 등록합니다. if isvalid(findEntity) then self.tweenLineComponent = findEntity else -- 정
상적으로 Component를 찾지 못했으므로 에러 로그를 출력합니다. log_error("Monster의 TweenLine이 nil입니다!")
return end end -- monsterSpriteRenderer가 nil인지 체크합니다. if not isvalid(self.monsterSpriteRenderer) then -
- 자기 자신의 Component중 SpriterRenderer를 찾아 등록합니다. self.monsterSpriteRenderer =
self.Entity.SpriteRendererComponent end
```

```

-- 만약 hpBar가 nil일 경우 이를 찾아 등록합니다. if not isvalid(self.hpBar) then local findEntity =
self.Entity:GetChildByName("HPBar") if isvalid(findEntity) then self.hpBar = findEntity end end

self.isAlive = true end

@ExecSpace("ClientOnly") method void OnUpdate(number delta) -- 만약 몬스터가 한 바퀴를 돌았을 경우 다시 출
발하도록 요청합니다. if self.tweenLineComponent.CurrentState == TweenState.Idle then
self.tweenLineComponent.RestartFromCurrentPosition() end end

@ExecSpace("ClientOnly") method void InitData(MonsterData MonsterData, Vector3 StartPos,
MonsterSpawnController MonsterSpawnController) -- 자신의 monsterData를 전달받은 MonsterData로 초기화합
니다. self.monsterData = MonsterData -- tweenLineComponent의 Duration을 전달받은 값으로 초기화합니다.
self.tweenLineComponent.Duration = self.monsterData.OneCycleMoveTime -- 현재 체력을 전달받은 HP로 초기
화합니다. self.currentHP = self.monsterData.StartHP -- 현재 상태를 활성화로 전환합니다. self.isAlive = false
self.Enable = false -- tweenLine을 최초 상태로 초기화합니다. self.tweenLineComponent:Stop(true) -- 현재 위치를
전달받은 위치로 수정합니다. self.Entity.TransformComponent.WorldPosition = StartPos -- 현재 hp에 맞춰 몬스터
체력바를 업데이트합니다. self.hpBar.TransformComponent.Scale.x = self.currentHP / self.monsterData.StartHP *
20 -- MonsterSpawnController를 등록합니다. self.spawnController = MonsterSpawnController end

@ExecSpace("ClientOnly") method void SetEnableState() -- 몬스터가 행동할 수 있도록 필요한 값들을 초기화합니
다. self.tweenLineComponent:Stop(true) self.tweenLineComponent:Play() self.currentHP =
self.monsterData.StartHP self.monsterSpriteRenderer.SpriteUID = self.moveAnimationKey self.isAlive = true
self.Enable = true end

@ExecSpace("ClientOnly") method void GetDamage(number AtkValue) -- 받은 데미지를 표기합니다.
_DamageSkinService:Play(self.Entity, "3271c3e79bf04ecba9a107d55495970d", 0.05, {AtkValue},
DamageSkinTweenType.Default, false) -- 현재 체력에서 전달받은 데미지만큼 감소시킵니다. self.currentHP =
self.currentHP - AtkValue -- 만약 현재 체력이 0 이하일 경우 처치 관련 처리를 진행합니다. if self.currentHP <= 0
then -- 오버플로우 방지를 위해 0으로 초기화합니다. self.currentHP = 0 -- 현재 생존 여부를 false로 변경합니다.
self.isAlive = false -- 몬스터의 이동을 중단시킵니다. self.tweenLineComponent:Pause() -- 몬스터의 애니메이션을
사망으로 변경합니다. self.monsterSpriteRenderer.SpriteUID = self.dieAnimationKey -- HPBar의 크기를 0으로 초
기화합니다. self.hpBar.TransformComponent.Scale.x = 0 -- 자기 자신이 처치되었으므로 MonsterSpawnManager
에게 잔존 수를 감소하도록 요청합니다. self.spawnController:UpdateRemainMonsterCount(-1) -- GameLogic에게
소환 토큰을 1 증가시키도록 요청합니다. _GameLogic:UpdateSummonToken(1) else -- 이외의 경우에는 설정한 크
기에 맞춰 UI가 변경되도록 계산한 값을 적용합니다. self.hpBar.TransformComponent.Scale.x = self.currentHP /
self.monsterData.StartHP * 20 end end

@EventSender("Self") handler HandleSpriteAnimPlayerEndFrameEvent(SpriteAnimPlayerEndFrameEvent event)
-- 몬스터 사망 애니메이션이 끝까지 출력되었다면 엔티티를 비활성화 시킵니다. if
self.monsterSpriteRenderer.SpriteUID == self.dieAnimationKey then self.Entity.Enable = false end end

end

```

> 해당 코드는 Plain Text를 기준으로 작성한 코드입니다.

<br>

### 몬스터의 정보를 불러오기 위한 Logic 제작하기

- 몬스터의 정보를 관리하고, 몬스터의 스폰을 관리하는 MonsterSpawnLogic을 제작합니다.
- MonsterSpawnLogic은 Logic Script로 제작된 Script입니다.
- 아래는 MonsterSpawnLogic의 코드 전문입니다.

```

```lua
@Logic
script MonsterSpawnLogic extends Logic

property table monsterData = {}

property table waveData = {}

@ExecSpace("ClientOnly")
method void OnBeginPlay()
self:InitMonsterData()
self:InitWaveData()
end

@ExecSpace("ClientOnly")
method void InitMonsterData()
-- monsterData가 사용 불가능한 상태인지 점검합니다.
if not isvalid(self.monsterData) or not next(self.monsterData) then
-- MonsterData DataSet이 정상적으로 있는지 확인합니다.
local findDataSet = _DataService:GetTable("MonsterData")
-- 만약 불러오는데 실패했다면 에러 로그를 남깁니다.
if not isvalid(findDataSet) then
log_error("MonsterData DataSet을 불러오는데 실패했습니다.")
return
end
-- 정상적으로 찾았다면 rowCount를 가져옵니다.
local rowCount = findDataSet:GetRowCount()
-- rowCount만큼 반복하면서 데이터를 순서대로 찾아 등록합니다.
for i = 1, rowCount do
local monsterID = tostring(findDataSet:GetCell(i, "MonsterID"))
-- MonsterID가 정상적으로 있는지 점검합니다.
if not _UtilLogic:IsNilorEmptyString(monsterID) then
-- MonsterID가 정상적으로 존재한다면 정상적인 값이라고 판단하여 데이터 불러오기
를 시작합니다.
local getData = MonsterData()
getData.MonsterID = monsterID
getData.OneCycleMoveTime = tonumber(findDataSet:GetCell(i,
"OneCycleMoveTime"))
getData.StartHP = tonumber(findDataSet:GetCell(i, "StartHP"))
getData.ModelID = tostring(findDataSet:GetCell(i, "ModelID"))
if not isvalid(self.monsterData[monsterID]) then
self.monsterData[monsterID] = {}
end
self.monsterData[monsterID] = getData
end
end
end
end

@ExecSpace("ClientOnly")

```

```

method void InitWaveData()
-- waveData가 사용 가능한 상태인지 점검합니다.
if not isvalid(self.waveData) or not next(self.waveData) then
    local findDataSet = _DataService:GetTable("WaveData")
    -- 정상적으로 DataSet을 찾았는지 확인합니다.
    if isvalid(findDataSet) then
        -- 정상적으로 데이터가 존재할 경우 rowCount를 가져옵니다.
        local rowCount = findDataSet:GetRowCount()
        -- rowCount 수 만큼 반복하여 데이터를 탐색합니다.
        for i = 1, rowCount do
            local getData = WaveData()
            getData.WaveIndex = tonumber(findDataSet:GetCell(i, "WaveIndex"))
            getData.SpawnMonster = tostring(findDataSet:GetCell(i,
"SpawnMonster"))
            getData.SpawnMonsterCount = tonumber(findDataSet:GetCell(i,
"SpawnMonsterCount"))
            if not isvalid(self.waveData[i]) then
                self.waveData[i] = {}
            end
            self.waveData[i] = getData
        end
    end
end
end

@ExecSpace("ClientOnly")
method MonsterData GetMonsterDataByMonsterID(string MonsterID)
-- MonsterID를 Key값으로 가지는 데이터가 있는지 확인합니다.
if isvalid(self.monsterData[MonsterID]) then
    -- 정상적으로 데이터가 존재할 경우 해당 데이터를 반환합니다.
    return self.monsterData[MonsterID]
end
-- 만약 데이터가 없을 경우, 에러 로그와 함께 nil을 반환합니다.
log_warning(MonsterID.."is Nil Data!")
return nil
end

@ExecSpace("ClientOnly")
method WaveData GetSpawnList(integer WaveIndex)
-- WaveIndex 값이 있는지 확인합니다.
if isvalid(self.waveData[WaveIndex]) then
    -- 정상적으로 값이 있을 경우 해당 Data를 반환합니다.
    return self.waveData[WaveIndex]
end
end
end
end

```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

- 몬스터의 정보를 불러오고 사용하기 위한 Logic과 Component를 구성합니다.
- 저장된 DataSet의 정보대로 몬스터의 정보가 파싱되어야 합니다.
- 몬스터의 이동 경로가 제작한 맵에 맞춰 이동해야 합니다.

응용 및 확장 방향

- 몬스터의 등급을 부여하고 이를 사용할 수 있도록 구조를 변경합니다.
- 다양한 몬스터가 스폰될 수 있도록 몬스터의 종류를 확장합니다.

[💡 기초 학습] 캐릭터의 공격 구현하기

영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 맵 타입 이해하기: 00:56:53

학습 목표

- 캐릭터가 일정 범위 내에 몬스터를 감지하여 공격 대상으로 등록하는 기능을 구현합니다.
- 캐릭터가 공격을 수행하여 몬스터에게 데미지를 줄 수 있도록 기능을 구현합니다.
- 캐릭터가 스킬을 사용할 수 있도록 기능을 구현합니다.
- 캐릭터의 공격을 일정한 주기로 수행할 수 있도록 기능을 구현합니다.

해당 영상 시청 후 수행해 볼 내용

- 캐릭터가 몬스터에게 공격을 수행할 수 있습니다.
- 데이터 테이블의 값 변동에 따라 캐릭터의 스킬을 변경할 수 있습니다.

스킬 데이터 제작하기

- 캐릭터가 스킬을 사용하기 위한 DataSet을 제작합니다.
- 앞서 제작한 CharacterData DataSet의 SkillData 항목에 값을 입력하면 스킬을 변경할 수 있도록 구성합니다.
- 이를 통해 캐릭터의 스킬을 DataSet 변경만으로 부여할 수 있도록 구조를 설계합니다.

DataSet 구성요소

- **SkillID**
 - 스킬을 구분하기 위한 값입니다.
 - 해당 값을 CharacterData DataSet의 SkillData에 입력하여 사용합니다.
- **SkillEffect**
 - 캐릭터가 적에게 스킬을 시전할 때, 적 위치에 출력되는 이펙트입니다.
 - 스킬의 이펙트 리소스 ID를 등록하기 위한 값입니다.
- **CharacterSkillEffect**
 - 캐릭터가 적에게 스킬을 시전할 때, 캐릭터에게 출력되는 이펙트입니다.
 - 스킬의 이펙트 리소스 ID를 등록하기 위한 값입니다.
- **SkillAttack**
 - 스킬의 공격력을 지정하기 위한 값입니다.
- **SkillCoolTime**
 - 스킬의 쿨타임을 지정하기 위한 값입니다.

StructType 제작하기

The screenshot displays a software interface with two main panels. The left panel, titled 'SkillData', shows a table with columns 'ID', 'SkillID', 'SkillEffect', and 'CharacterSkillEffect'. The right panel, titled 'SkillData', shows the corresponding StructType definition in a code editor.

ID	SkillID	SkillEffect	CharacterSkillEffect
1	Skill_HeavySlas		6b6f3a5e8c804f
2	Skill_RapidArro		373b2346e2694
3	Skill_Thunder	bb8d0c3c8b994	

```

1 SkillData {
2     Property:
3         string SkillID = " " :
4         string SkillEffect = " " :
5         string CharacterSkillEffect = " " :
6         number SkillAttack = 0 :
7         number SkillCoolTime = 0 :
8     Method:
9 }
  
```

- SkillData의 DataSet 이름과 동일하게 StructType을 제작합니다.
- 샘플 월드에서는 DataSet과 동일한 이름으로 StructType이 제작되어 있습니다.

캐릭터의 공격 스크립트 살펴보기

- 캐릭터의 Event Handler를 통해 몬스터가 캐릭터의 공격 범위에 있는지, 벗어났는지 관리합니다.
- 아래는 캐릭터를 관리하는 UnitController Component의 코드입니다.

```

@ExecSpace("ClientOnly")
@EventSender("Self")
handler HandleTriggerEnterEvent(TriggerEnterEvent event)
-- 충돌 범위에 들어온 엔티티가 CollisionGroups, Monster일 경우 이를 등록합니다.
if event.TriggerBodyEntity.TriggerComponent.CollisionGroup ==
CollisionGroups.Monster then
    if #self.enemyTable >= 1 then
        local isAlreadyHave = false
        for i = 1, #self.enemyTable do
            if self.enemyTable[i] == event.TriggerBodyEntity then
                isAlreadyHave = true
                break
            end
        end
        if isAlreadyHave == false then
            table.insert(self.enemyTable, event.TriggerBodyEntity)
        end
    else
        table.insert(self.enemyTable, event.TriggerBodyEntity)
    end
end
end

@ExecSpace("ClientOnly")
@EventSender("Self")
handler HandleTriggerLeaveEvent(TriggerLeaveEvent event)
-- 충돌 범위를 벗어난 엔티티가 Monster일 경우 공격 리스트에서 제거합니다.
if event.TriggerBodyEntity.TriggerComponent.CollisionGroup ==
CollisionGroups.Monster then
    if #self.enemyTable >= 1 then
        local listCount = #self.enemyTable
        for i = listCount, 1, -1 do
            if self.enemyTable[i] == event.TriggerBodyEntity then
                table.remove(self.enemyTable, i)
                break
            end
        end
    end
end
end
end

```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

-

@ExecSpace("ClientOnly") @EventSender("Self") handler HandleTriggerEnterEvent(TriggerEnterEvent event) -- 충돌 범위에 들어온 엔티티가 CollisionGroups, Monster일 경우 이를 등록합니다. if event.TriggerBodyEntity.TriggerComponent.CollisionGroup == CollisionGroups.Monster then if #self.enemyTable >= 1 then local isAlreadyHave = false for i = 1, #self.enemyTable do if self.enemyTable[i] == event.TriggerBodyEntity then isAlreadyHave = true break end end if isAlreadyHave == false then


```
table.insert(self.enemyTable, event.TriggerBodyEntity) end else table.insert(self.enemyTable,
event.TriggerBodyEntity) end end end
```

```
@ExecSpace("ClientOnly") @EventSender("Self") handler HandleTriggerLeaveEvent(TriggerLeaveEvent event) --
충돌 범위를 벗어난 엔티티가 Monster일 경우 공격 리스트에서 제거합니다. if
event.TriggerBodyEntity.TriggerComponent.CollisionGroup == CollisionGroups.Monster then if
#self.enemyTable >= 1 then local listCount = #self.enemyTable for i = listCount, 1, -1 do if self.enemyTable[i]
== event.TriggerBodyEntity then table.remove(self.enemyTable, i) break end end end end end
```

> 해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

- `HandleTriggerEnterEvent`를 통해 몬스터가 캐릭터의 공격 범위에 들어왔는지 체크합니다.
- `HandleTriggerExitEvent`를 통해 몬스터가 공격 범위를 벗어날 경우 공격 가능 리스트에서 제거합니다.

```
```lua
@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- 델타타임은 조건 없이 항상 더해줍니다.
self.skillDelta = self.skillDelta + delta
self.basicAtkDelta = self.basicAtkDelta + delta

-- 스킬 공격 쿨타임을 확인하여 조건을 충족할 경우 스킬 공격을 수행합니다.
if self.skillDelta >= self.characterSkillData.SkillCoolTime then
 self:SkillAttack()
end

-- 만약 스킬 공격이 성공하여 쿨타임이 0으로 초기화되었다면, 이번 프레임엔 기본 공격을 쉬어갑니
다.
if self.skillDelta == 0 then
 return
end

-- 기본 공격 쿨타임을 확인하여 조건을 충족할 경우 기본 공격을 수행합니다.
if self.basicAtkDelta >= self.characterData.BasicAttackCoolTime then
 self:BasicAttack()
end
end

@ExecSpace("ClientOnly")
method void BasicAttack()
-- 캐릭터의 StateComponent를 가져옵니다.
local bodySelector = self.Entity.StateComponent
-- 캐릭터의 현재 상태 이름을 가져옵니다.
local currentState = bodySelector.CurrentStateName
-- 현재 상태가 IDLE이 아닐 경우, 기본 공격 수행을 하지 않습니다.
if currentState ~= "IDLE" then
 return
end
---@type Entity
local targetEntity = nil
```

```

-- 테이블을 역순으로 순회하며 유효하지 않은 엔티티를 안전하게 제거합니다.
for i = #self.enemyTable, 1, -1 do
 local enemy = self.enemyTable[i]
 -- 엔티티가 아직 월드에 존재하는지 확인합니다.
 if invalid(enemy) then
 -- 엔티티에 MonsterController가 있고 살아있는지 확인합니다.
 if enemy.MonsterController and enemy.MonsterController.isAlive then
 targetEntity = enemy
 break
 end
 else
 -- 이미 파괴된 엔티티라면 테이블에서 제거합니다.
 table.remove(self.enemyTable, i)
 end
end
-- 만약 지정된 targetEntity가 사용 불가능할 경우 공격을 수행하지 않습니다.
if not invalid(targetEntity) then
 return
end
-- 공격을 수행했다 가정하여 basicAtkDelta값을 0으로 설정합니다.
self.basicAtkDelta = 0
-- 공격 애니메이션을 출력시키기 위해 event를 생성, 전달합니다.
local event = ActionStateChangedEvent(self.characterData.AttackAnimationID,
self.characterData.AttackAnimationID, 1, SpriteAnimClipPlayType.Onetime)
self.avatarBodyComponent:SendEvent(event)
-- 출력해야 하는 캐릭터쪽 이펙트가 있을 경우 이를 출력합니다.
if not _UtilLogic:IsNilorEmptyString(self.characterData.BasicAttackEffectID) then
 _EffectService:PlayEffect(self.characterData.BasicAttackEffectID, self.Entity,
self.Entity.TransformComponent.WorldPosition, 0, Vector3.one)
end
-- 적에게 출력해야 하는 이펙트가 있을 경우 이를 출력합니다.
if not _UtilLogic:IsNilorEmptyString(self.characterData.BasicAttackHitEffectID)
then
 _EffectService:PlayEffect(self.characterData.BasicAttackHitEffectID,
targetEntity, targetEntity.TransformComponent.WorldPosition, 0, Vector3.one)
end
-- 타겟이 된 적의 MonsterController를 통해 GetDamage를 수행하도록 요청합니다.
targetEntity.MonsterController:GetDamage(self.characterData.BasicAttackValue)
end

@ExecSpace("ClientOnly")
method void SkillAttack()
 -- 애니메이션 재생을 위한 bodySelector를 찾아 등록합니다.
 local bodySelector = self.Entity.StateComponent
 -- 현재 상태를 찾기 위해 CurrentStateName을 등록합니다.
 local currentState = bodySelector.CurrentStateName
 -- 현재 상태가 IDLE이 아닐 경우 코드 실행을 중단합니다.
 if currentState ~= "IDLE" then
 return
 end
 ---@type Entity
 local targetEntity = nil
 -- 테이블을 역순으로 순회하며 유효하지 않은 엔티티를 안전하게 제거합니다.
 for i = #self.enemyTable, 1, -1 do

```

```

 local enemy = self.enemyTable[i]
 if invalid(enemy) then
 if enemy.MonsterController and enemy.MonsterController.isAlive then
 targetEntity = enemy
 break
 end
 else
 table.remove(self.enemyTable, i)
 end
 end
end
-- 만약 targetEntity가 nil일 경우 코드 실행을 중단합니다.
if not invalid(targetEntity) then
 return
end
-- 스킬 사용을 성공했다 가정하여 skillDelta값을 0으로 초기화합니다.
self.skillDelta = 0
-- 공격 애니메이션을 재생합니다.
local event = ActionStateChangedEvent(self.characterData.AttackAnimationID,
self.characterData.AttackAnimationID, 1, SpriteAnimClipPlayType.Onetime)
self.avatarBodyComponent:SendEvent(event)
-- 스킬 사용시 자기 자신에게 출력해야 하는 이펙트가 있을 경우 이펙트를 출력합니다.
if not _UtilLogic:IsNilorEmptyString(self.characterSkillData.CharacterSkillEffect)
then
 _EffectService:PlayEffect(self.characterSkillData.CharacterSkillEffect,
self.Entity, self.Entity.TransformComponent.WorldPosition, 0, Vector3.one)
end
-- 피격 대상에게 출력해야 하는 이펙트가 있을 경우 이펙트를 출력합니다.
if not _UtilLogic:IsNilorEmptyString(self.characterSkillData.SkillEffect) then
 _EffectService:PlayEffect(self.characterSkillData.SkillEffect, targetEntity,
targetEntity.TransformComponent.WorldPosition, 0, Vector3.one)
end
-- 대상이 된 targetEntity의 MonsterController에게 GetDamage를 수행하도록 요청합니다.
targetEntity.MonsterController:GetDamage(self.characterSkillData.SkillAttack)
end

```

해당 코드는 Plain Text로 작성된 코드입니다.

- OnUpdate를 통해 기본 공격의 쿨타임, 스킬 공격의 쿨타임을 계산합니다.
- 만약 스킬 쿨타임이 일정 시간에 도달한다면 스킬 공격을 우선합니다.
- 현재 스킬 공격을 수행중일 경우 기본 공격을 하지 않도록 예외처리를 진행합니다.
- 만약 스킬 공격이 대기 상태이며 기본 공격이 가능할 경우 기본 공격을 수행합니다.

## 몬스터의 피격 살펴보기

- 몬스터의 행동을 제어하는 MonsterController 내에 몬스터가 피격될 때 호출되는 Method가 존재합니다.
- 해당 코드는 MonsterController의 코드입니다.

```

@ExecSpace("ClientOnly")
method void GetDamage(number AtkValue)

```

```
-- 받은 데미지를 표기합니다.
_DamageSkinService:Play(self.Entity, "3271c3e79bf04ecba9a107d55495970d", 0.05,
{AtkValue}, DamageSkinTweenType.Default, false)
-- 현재 체력에서 전달받은 데미지만큼 감소시킵니다.
self.currentHP = self.currentHP - AtkValue
-- 만약 현재 체력이 0 이하일 경우 처치 관련 처리를 진행합니다.
if self.currentHP <= 0 then
 -- 오버플로우 방지를 위해 0으로 초기화합니다.
 self.currentHP = 0
 -- 현재 생존 여부를 false로 변경합니다.
 self.isAlive = false
 -- 몬스터의 이동을 중단시킵니다.
 self.tweenLineComponent:Pause()
 -- 몬스터의 애니메이션을 사망으로 변경합니다.
 self.monsterSpriteRenderer.SpriteRUID = self.dieAnimationKey
 -- HPBar의 크기를 0으로 초기화합니다.
 self.hpBar.TransformComponent.Scale.x = 0
 -- 자기 자신이 처치되었으므로 MonsterSpawnManager에게 잔존 수를 감소하도록 요청합니다.
 self.spawnController:UpdateRemainMonsterCount(-1)
 -- GameLogic에게 소환 토큰을 1 증가시키도록 요청합니다.
 _GameLogic:UpdateSummonToken(1)
else
 -- 이외의 경우에는 설정한 크기에 맞춰 UI가 변경되도록 계산한 값을 적용합니다.
 self.hpBar.TransformComponent.Scale.x = self.currentHP /
self.monsterData.StartHP * 20
end
end
```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

- AtkValue로 캐릭터가 주는 데미지를 받아옵니다.
- 자기 자신의 체력에서 전달 받은 데미지만큼 감소시킵니다.
- 현재 체력이 0일 경우, 사망과 관련된 처리를 수행합니다.
- HPBar를 감소시켜 남은 체력을 표기합니다.

## 최소 구현 기준

- 캐릭터가 몬스터를 감지하고 공격할 수 있습니다.
- 데미지 계산을 통해 몬스터의 상태를 생존과 사망상태로 전환할 수 있습니다.
- 캐릭터가 스킬을 사용하여 공격을 할 수 있습니다.

## 응용 및 확장 방향

- 캐릭터의 스킬을 범위형 공격을 제작하여 범위내 적들에게 공격하는 기능을 제작합니다.
- 캐릭터의 공격에 따라 사운드의 출력, 이펙트의 변경 등 시각적, 청각적 효과를 강화합니다.
- 몬스터를 공격하는 기준을 자신과 가장 가까운 적으로 공격 상대를 결정하도록 기능을 수정합니다.

## [💡 기초 학습] 몬스터 스폰하기

### 영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 맵 타입 이해하기: ``lua @ExecSpace("ClientOnly") method void OnUpdate(number delta) -- 델타타임은 조건 없이 항상 더해줍니다. self.skillDelta = self.skillDelta + delta self.basicAtkDelta = self.basicAtkDelta + delta

```
-- 스킬 공격 쿨타임을 확인하여 조건을 충족할 경우 스킬 공격을 수행합니다. if self.skillDelta >= self.characterSkillData.SkillCoolTime then self.SkillAttack() end
```

```
-- 만약 스킬 공격이 성공하여 쿨타임이 0으로 초기화되었다면, 이번 프레임엔 기본 공격을 쉬어갑니다. if self.skillDelta == 0 then return end
```

```
-- 기본 공격 쿨타임을 확인하여 조건을 충족할 경우 기본 공격을 수행합니다. if self.basicAtkDelta >= self.characterData.BasicAttackCoolTime then self.BasicAttack() end end
```

```
@ExecSpace("ClientOnly") method void BasicAttack() -- 캐릭터의 StateComponent를 가져옵니다. local bodySelector = self.Entity.StateComponent -- 캐릭터의 현재 상태 이름을 가져옵니다. local currentState = bodySelector.CurrentStateName -- 현재 상태가 IDLE이 아닐 경우, 기본 공격 수행을 하지 않습니다. if currentState ~= "IDLE" then return end ---@type Entity local targetEntity = nil -- 테이블을 역순으로 순회하며 유효하지 않은 엔티티를 안전하게 제거합니다. for i = #self.enemyTable, 1, -1 do local enemy = self.enemyTable[i] -- 엔티티가 아직 월드에 존재하는지 확인합니다. if isvalid(enemy) then -- 엔티티에 MonsterController가 있고 살아있는지 확인합니다. if enemy.MonsterController and enemy.MonsterController.isAlive then targetEntity = enemy break end else -- 이미 파괴된 엔티티라면 테이블에서 제거합니다. table.remove(self.enemyTable, i) end end -- 만약 지정된 targetEntity가 사용 불가능할 경우 공격을 수행하지 않습니다. if not isvalid(targetEntity) then return end -- 공격을 수행했다 가정하여 basicAtkDelta값을 0으로 설정합니다. self.basicAtkDelta = 0 -- 공격 애니메이션을 출력시키기 위해 event를 생성, 전달합니다. local event = ActionStateChangedEvent(self.characterData.AttackAnimationID, self.characterData.AttackAnimationID, 1, SpriteAnimClipPlayType.Onetime) self.avatarBodyComponent:SendEvent(event) -- 출력해야 하는 캐릭터쪽 이펙트가 있을 경우 이를 출력합니다. if not _UtilLogic.IsNilorEmptyString(self.characterData.BasicAttackEffectID) then _EffectService:PlayEffect(self.characterData.BasicAttackEffectID, self.Entity, self.Entity.TransformComponent.WorldPosition, 0, Vector3.one) end -- 적에게 출력해야 하는 이펙트가 있을 경우 이를 출력합니다. if not _UtilLogic.IsNilorEmptyString(self.characterData.BasicAttackHitEffectID) then _EffectService:PlayEffect(self.characterData.BasicAttackHitEffectID, targetEntity, targetEntity.TransformComponent.WorldPosition, 0, Vector3.one) end -- 타겟이 된 적의 MonsterController를 통해 GetDamage를 수행하도록 요청합니다. targetEntity.MonsterController:GetDamage(self.characterData.BasicAttackValue) end
```

```
@ExecSpace("ClientOnly") method void SkillAttack() -- 애니메이션 재생을 위한 bodySelector를 찾아 등록합니다. local bodySelector = self.Entity.StateComponent -- 현재 상태를 찾기 위해 CurrentStateName을 등록합니다. local currentState = bodySelector.CurrentStateName -- 현재 상태가 IDLE이 아닐 경우 코드 실행을 중단합니다. if currentState ~= "IDLE" then return end ---@type Entity local targetEntity = nil -- 테이블을 역순으로 순회하며 유효
```

```

효하지 않은 엔티티를 안전하게 제거합니다. for i = #self.enemyTable, 1, -1 do local enemy = self.enemyTable[i] if
isvalid(enemy) then if enemy.MonsterController and enemy.MonsterController.isAlive then targetEntity =
enemy break end else table.remove(self.enemyTable, i) end end -- 만약 targetEntity가 nil일 경우 코드 실행을 중
단합니다. if not isvalid(targetEntity) then return end -- 스킬 사용을 성공했다 가정하여 skillDelta값을 0으로 초기화
합니다. self.skillDelta = 0 -- 공격 애니메이션을 재생합니다. local event =
ActionStateChangedEvent(self.characterData.AttackAnimationID, self.characterData.AttackAnimationID, 1,
SpriteAnimClipPlayType.Onetime) self.avatarBodyComponent:SendEvent(event) -- 스킬 사용시 자기 자신에게 출
력해야 하는 이펙트가 있을 경우 이펙트를 출력합니다. if not
_UtilLogic:IsNilorEmptyString(self.characterSkillData.CharacterSkillEffect) then
_EffectService:PlayEffect(self.characterSkillData.CharacterSkillEffect, self.Entity,
self.Entity.TransformComponent.WorldPosition, 0, Vector3.one) end -- 피격 대상에게 출력해야 하는 이펙트가 있
을 경우 이펙트를 출력합니다. if not _UtilLogic:IsNilorEmptyString(self.characterSkillData.SkillEffect) then
_EffectService:PlayEffect(self.characterSkillData.SkillEffect, targetEntity,
targetEntity.TransformComponent.WorldPosition, 0, Vector3.one) end -- 대상이 된 targetEntity의
MonsterContorller에게 GetDamage를 수행하도록 요청합니다.
targetEntity.MonsterController:GetDamage(self.characterSkillData.SkillAttack) end

```

> 해당 코드는 Plain Text로 작성된 코드입니다.

- OnUpdate를 통해 기본 공격의 쿨타임, 스킬 공격의 쿨타임을 계산합니다.
- 만약 스킬 쿨타임이 일정 시간에 도달한다면 스킬 공격을 우선합니다.
- 현재 스킬 공격을 수행중일 경우 기본 공격을 하지 않도록 예외처리를 진행합니다.
- 만약 스킬 공격이 대기 상태이며 기본 공격이 가능할 경우 기본 공격을 수행합니다.

<br>

## ## 몬스터의 피격 살펴보기

- 몬스터의 행동을 제어하는 MonsterController 내에 몬스터가 피격될 때 호출되는 Method가 존재합니다.
- 해당 코드는 MonsterController의 코드입니다.

```

```lua
@ExecSpace("ClientOnly")
method void GetDamage(number AtkValue)
-- 받은 데미지를 표기합니다.
_DamageSkinService:Play(self.Entity, "3271c3e79bf04ecba9a107d55495970d", 0.05,
{AtkValue}, DamageSkinTweenType.Default, false)
-- 현재 체력에서 전달받은 데미지만큼 감소시킵니다.
self.currentHP = self.currentHP - AtkValue
-- 만약 현재 체력이 0 이하일 경우 처치 관련 처리를 진행합니다.
if self.currentHP <= 0 then
-- 오버플로우 방지를 위해 0으로 초기화합니다.
self.currentHP = 0
-- 현재 생존 여부를 false로 변경합니다.
self.isAlive = false
-- 몬스터의 이동을 중단시킵니다.
self.tweenLineComponent:Pause()
-- 몬스터의 애니메이션을 사망으로 변경합니다.

```

```

self.monsterSpriteRenderer.SpriteRUID = self.dieAnimationKey
-- HPBar의 크기를 0으로 초기화합니다.
self.hpBar.TransformComponent.Scale.x = 0
-- 자기 자신이 처치되었으므로 MonsterSpawnManager에게 잔존 수를 감소하도록 요청합니다.
self.spawnController.UpdateRemainMonsterCount(-1)
-- GameLogic에게 소환 토큰을 1 증가시키도록 요청합니다.
_GameLogic.UpdateSummonToken(1)
else
  -- 이외의 경우에는 설정한 크기에 맞춰 UI가 변경되도록 계산한 값을 적용합니다.
  self.hpBar.TransformComponent.Scale.x = self.currentHP /
self.monsterData.StartHP * 20
end
end

```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

- AtkValue로 캐릭터가 주는 데미지를 받아옵니다.
- 자기 자신의 체력에서 전달 받은 데미지만큼 감소시킵니다.
- 현재 체력이 0일 경우, 사망과 관련된 처리를 수행합니다.
- HPBar를 감소시켜 남은 체력을 표기합니다.

최소 구현 기준

- 캐릭터가 몬스터를 감지하고 공격할 수 있습니다.
- 데미지 계산을 통해 몬스터의 상태를 생존과 사망상태로 전환할 수 있습니다.
- 캐릭터가 스킬을 사용하여 공격을 할 수 있습니다.

응용 및 확장 방향

- 캐릭터의 스킬을 범위형 공격을 제작하여 범위내 적들에게 공격하는 기능을 제작합니다.
- 캐릭터의 공격에 따라 사운드의 출력, 이펙트의 변경 등 시각적, 청각적 효과를 강화합니다.
- 몬스터를 공격하는 기준을 자신과 가장 가까운 적으로 공격 상대를 결정하도록 기능을 수정합니다.

[💡 기초 학습] 몬스터 스폰하기

영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 맵 타입 이해하기: 01:03:48

학습 목표

- 몬스터 스폰을 관리하는 DataSet을 제작합니다.
- 제작된 DataSet에 맞춰 몬스터가 스폰될 수 있도록 기능을 구현합니다.

해당 영상 시청 후 수행해 볼 내용

- 지정한 DataSet을 활용하여 몬스터가 스폰될 수 있도록 기능을 구성합니다.
- 웨이브의 단계에 따라 스폰되는 몬스터가 변경될 수 있도록 DataSet과 기능을 구성합니다.

몬스터 소환 DataSet 구성하기

- 각 웨이브에 따라 소환되어야 하는 몬스터를 지정하기 위한 DataSet을 구성합니다.
- 샘플 월드에서는 WaveData라는 이름으로 DataSet을 구성했습니다.
- WaveData는 다음의 구조로 제작되어 있습니다.
- **WaveIndex**
 - 웨이브의 단계를 지정하기 위한 값입니다.
- **SpawnMonster**
 - 해당 웨이브에 소환되어야 하는 몬스터를 지정하기 위한 값입니다.
 - MonsterData의 MonsterID값을 입력하여 사용합니다.
- **SpawnMonsterCount**
 - 해당 웨이브에 스폰되어야 하는 몬스터의 수를 결정하기 위한 값입니다.

StructType 구성하기

	1	2	3
ID	WaveIndex	SpawnMonster	SpawnMonsterCount
1	1	Monster_Orang	10
2	2	Monster_Orang	20
3	3	Monster_Orang	30
4	4	Monster_Stone	10
5	5	Monster_Stone	20

```

1 WaveData {
2     Property:
3         number WaveIndex = 0
4         string SpawnMonster
5         number SpawnMonsterCount = 0
6     Method:
7 }
  
```

- WaveData의 값들과 동일하게 StructType을 구성합니다.
- Property에 WaveData의 요소들을 담을 수 있도록 동일한 이름을 사용하여 선언합니다.

Logic 구성하기

- 앞서 제작했던 MonsterSpawnLogic에 제작된 SpawnData를 등록합니다.
- 몬스터의 스폰을 요청받으면 현재 웨이브의 정보에 맞는 몬스터를 반환하도록 기능을 구성합니다.
- 아래는 MonsterSpawnLogic의 코드입니다.

```

@Logic
script MonsterSpawnLogic extends Logic

property table monsterData = {}

property table waveData = {}

@ExecSpace("ClientOnly")
method void OnBeginPlay()
self:InitMonsterData()
self:InitWaveData()
end

@ExecSpace("ClientOnly")
method void InitMonsterData()
-- monsterData가 사용 불가능한 상태인지 점검합니다.
if not invalid(self.monsterData) or not next(self.monsterData) then
  
```

```

-- MonsterData DataSet이 정상적으로 있는지 확인합니다.
local findDataSet = _DataService:GetTable("MonsterData")
-- 만약 불러오는데 실패했다면 에러 로그를 남깁니다.
if not isValid(findDataSet) then
    log_error("MonsterData DataSet을 불러오는데 실패했습니다.")
    return
end
-- 정상적으로 찾았다면 rowCount를 가져옵니다.
local rowCount = findDataSet:GetRowCount()
-- rowCount만큼 반복하면서 데이터를 순서대로 찾아 등록합니다.
for i = 1, rowCount do
    local monsterID = tostring(findDataSet:GetCell(i, "MonsterID"))
    -- MonsterID가 정상적으로 있는지 점검합니다.
    if not _UtilLogic:IsNilorEmptyString(monsterID) then
        -- MonsterID가 정상적으로 존재한다면 정상적인 값이라고 판단하여 데이터 불러오기
        -- 를 시작합니다.
        local getData = MonsterData()
        getData.MonsterID = monsterID
        getData.OneCycleMoveTime = tonumber(findDataSet:GetCell(i,
"OneCycleMoveTime"))
        getData.StartHP = tonumber(findDataSet:GetCell(i, "StartHP"))
        getData.ModelID = tostring(findDataSet:GetCell(i, "ModelID"))
        if not isValid(self.monsterData[monsterID]) then
            self.monsterData[monsterID] = {}
        end
        self.monsterData[monsterID] = getData
    end
end
end
end

@ExecSpace("ClientOnly")
method void InitWaveData()
-- waveData가 사용 가능한 상태인지 점검합니다.
if not isValid(self.waveData) or not next(self.waveData) then
    local findDataSet = _DataService:GetTable("WaveData")
    -- 정상적으로 DataSet을 찾았는지 확인합니다.
    if isValid(findDataSet) then
        -- 정상적으로 데이터가 존재할 경우 rowCount를 가져옵니다.
        local rowCount = findDataSet:GetRowCount()
        -- rowCount 수 만큼 반복하여 데이터를 탐색합니다.
        for i = 1, rowCount do
            local getData = WaveData()
            getData.WaveIndex = tonumber(findDataSet:GetCell(i, "WaveIndex"))
            getData.SpawnMonster = tostring(findDataSet:GetCell(i,
"SpawnMonster"))
            getData.SpawnMonsterCount = tonumber(findDataSet:GetCell(i,
"SpawnMonsterCount"))
            if not isValid(self.waveData[i]) then
                self.waveData[i] = {}
            end
            self.waveData[i] = getData
        end
    end
end
end

```

```
end
end

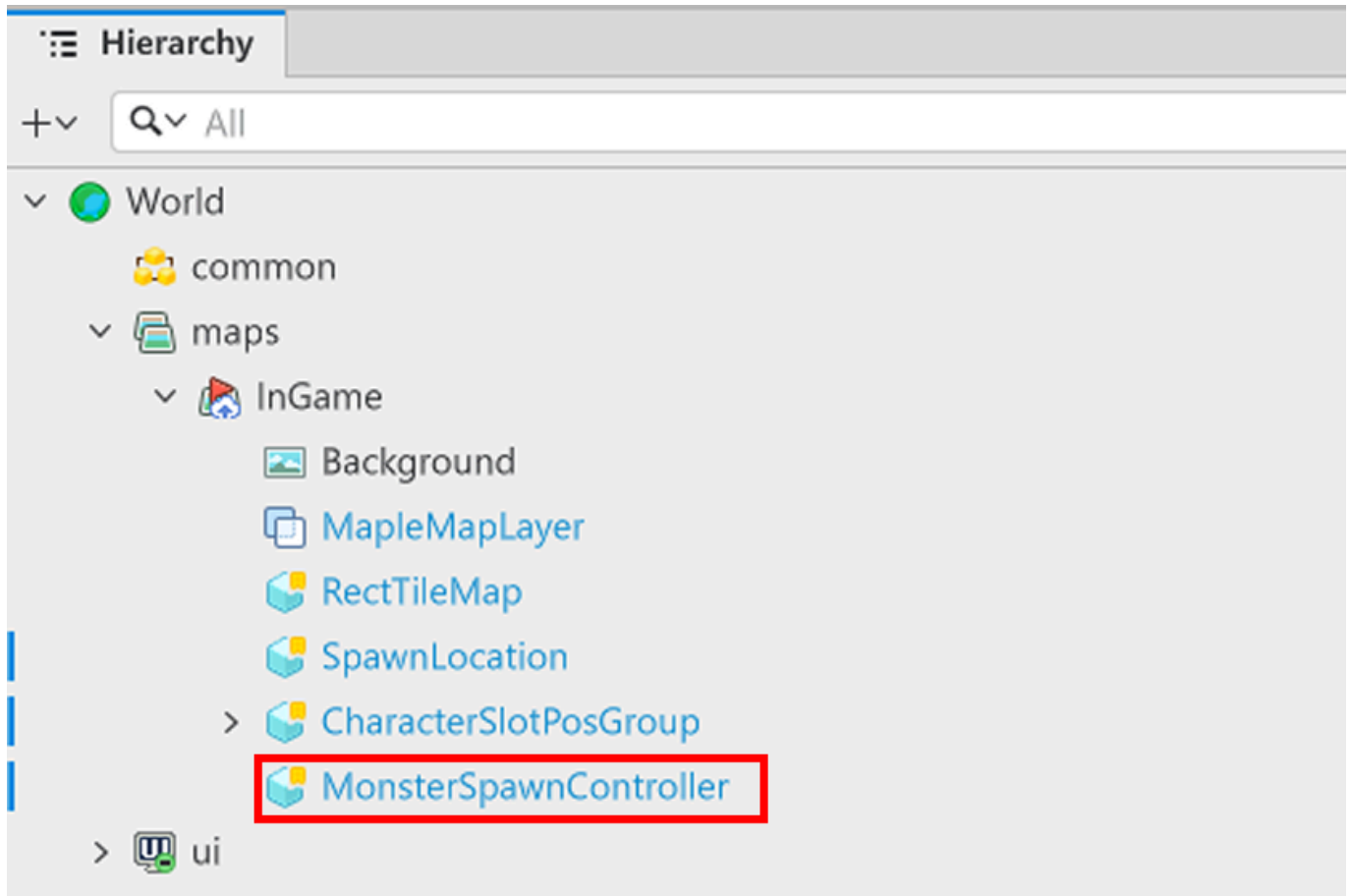
@ExecSpace("ClientOnly")
method MonsterData GetMonsterDataByMonsterID(string MonsterID)
-- MonsterID를 Key값으로 가지는 데이터가 있는지 확인합니다.
if invalid(self.monsterData[MonsterID]) then
-- 정상적으로 데이터가 존재할 경우 해당 데이터를 반환합니다.
return self.monsterData[MonsterID]
end
-- 만약 데이터가 없을 경우, 에러 로그와 함께 nil을 반환합니다.
log_warning(MonsterID.."is Nil Data!")
return nil
end

@ExecSpace("ClientOnly")
method WaveData GetSpawnList(integer WaveIndex)
-- WaveIndex 값이 있는지 확인합니다.
if invalid(self.waveData[WaveIndex]) then
-- 정상적으로 값이 있을 경우 해당 Data를 반환합니다.
return self.waveData[WaveIndex]
end
end

end
```

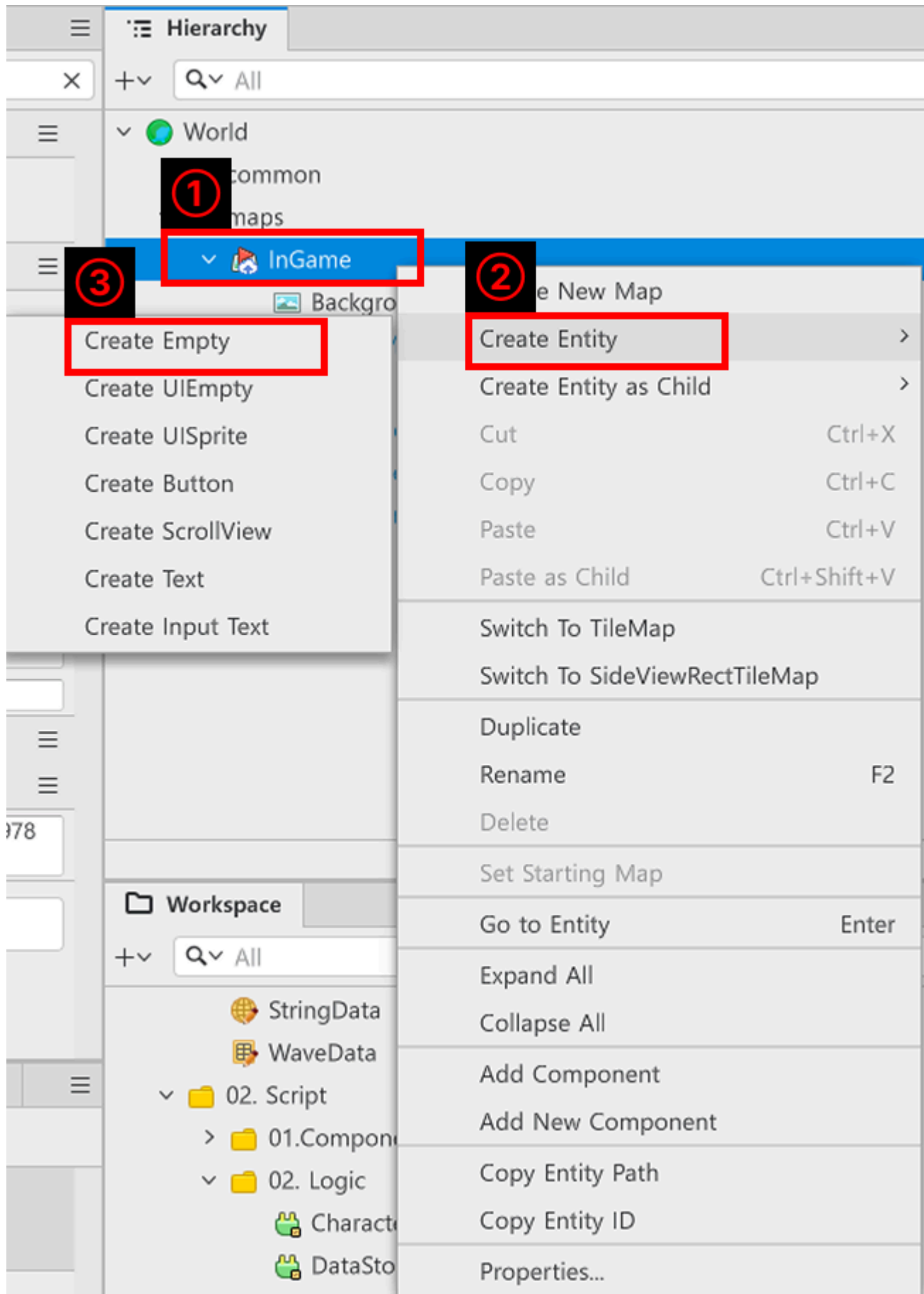
해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

MonsterSpawnController 제작하기



- MonsterController는 InGame 맵 안에 존재하는 엔티티입니다.
- InGame에서 Wave가 실행되는 동안 정해진 조건에 맞춰 몬스터의 생성 요청과 타이머의 요청을 수행합니다.

엔티티 준비하기



-

@Logic script MonsterSpawnLogic extends Logic

```
property table monsterData = {}
```

```
property table waveData = {}
```

```
@ExecSpace("ClientOnly") method void OnBeginPlay() self:InitMonsterData() self:InitWaveData() end
```

```
@ExecSpace("ClientOnly") method void InitMonsterData() -- monsterData가 사용 불가능한 상태인지 점검합니다.
if not isvalid(self.monsterData) or not next(self.monsterData) then -- MonsterData DataSet이 정상적으로 있는지
확인합니다. local findDataSet = _DataService:GetTable("MonsterData") -- 만약 불러오는데 실패했다면 에러 로그를
남깁니다. if not isvalid(findDataSet) then log_error("MonsterData DataSet을 불러오는데 실패했습니다.") return
end -- 정상적으로 찾았다면 rowCount를 가져옵니다. local rowCount = findDataSet:GetRowCount() -- rowCount
만큼 반복하면서 데이터를 순서대로 찾아 등록합니다. for i = 1, rowCount do local monsterID =
tostring(findDataSet:GetCell(i, "MonsterID")) -- MonsterID가 정상적으로 있는지 점검합니다. if not
_UtilLogic:IsNilorEmptyString(monsterID) then -- MonsterID가 정상적으로 존재한다면 정상적인 값이라고 판단하
여 데이터 불러오기를 시작합니다. local getData = MonsterData() getData.MonsterID = monsterID
getData.OneCycleMoveTime = tonumber(findDataSet:GetCell(i, "OneCycleMoveTime")) getData.StartHP =
tonumber(findDataSet:GetCell(i, "StartHP")) getData.ModelID = tostring(findDataSet:GetCell(i, "ModelID")) if
not isvalid(self.monsterData[monsterID]) then self.monsterData[monsterID] = {} end
self.monsterData[monsterID] = getData end end end end
```

```
@ExecSpace("ClientOnly") method void InitWaveData() -- waveData가 사용 가능한 상태인지 점검합니다. if not
isvalid(self.waveData) or not next(self.waveData) then local findDataSet = _DataService:GetTable("WaveData") -
- 정상적으로 DataSet을 찾았는지 확인합니다. if isvalid(findDataSet) then -- 정상적으로 데이터가 존재할 경우
rowCount를 가져옵니다. local rowCount = findDataSet:GetRowCount() -- rowCount 수 만큼 반복하여 데이터를
탐색합니다. for i = 1, rowCount do local getData = WaveData() getData.WaveIndex =
tonumber(findDataSet:GetCell(i, "WaveIndex")) getData.SpawnMonster = tostring(findDataSet:GetCell(i,
"SpawnMonster")) getData.SpawnMonsterCount = tonumber(findDataSet:GetCell(i, "SpawnMonsterCount")) if
not isvalid(self.waveData[i]) then self.waveData[i] = {} end self.waveData[i] = getData end end end end
```

```
@ExecSpace("ClientOnly") method MonsterData GetMonsterDataByMonsterID(string MonsterID) -- MonsterID
를 Key값으로 가지는 데이터가 있는지 확인합니다. if isvalid(self.monsterData[MonsterID]) then -- 정상적으로 데이
터가 존재할 경우 해당 데이터를 반환합니다. return self.monsterData[MonsterID] end -- 만약 데이터가 없을 경우,
에러 로그와 함께 nil을 반환합니다. log_warning(MonsterID.."is Nil Data!") return nil end
```

```
@ExecSpace("ClientOnly") method WaveData GetSpawnList(integer WaveIndex) -- WaveIndex 값이 있는지 확인
합니다. if isvalid(self.waveData[WaveIndex]) then -- 정상적으로 값이 있을 경우 해당 Data를 반환합니다. return
self.waveData[WaveIndex] end end
```

```
end
```

> 해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

MonsterSpawnController 제작하기

<p align="center">

</p>

- MonsterController는 InGame 맵 안에 존재하는 엔티티입니다.
- InGame에서 Wave가 실행되는 동안 정해진 조건에 맞춰 몬스터의 생성 요청과 타이머의 요청을 수행합니다.

엔티티 준비하기

<p align="center">

</p>

- `Hierarchy`에서 `InGame`을 마우스 우클릭합니다.
- `Create Entity`를 클릭하여 메뉴를 엽니다.
- `Create Empty`를 클릭해 빈 엔티티를 생성합니다.

<p align="center">

</p>

- 생성한 엔티티의 `TransformComponent`의 Position, WorldPosition을 0으로 설정합니다.
- 그 후 몬스터의 생성을 담당하는 `MonsterSpawnController`를 제작, 추가합니다.

MonsterSpawnController 제작하기

- MonsterSpawnController는 현재 자신의 웨이브 관리, 몬스터 생성, 남은 몬스터 수 관리를 수행합니다.
- 아래는 MonsterSpawnController Component의 전체 코드입니다.

```
```lua
```

```
@Component
```

```
script MonsterSpawnController extends Component
```

```
property table monsterList = {}
```

```
property integer waveIndex = 0
```

```
property number standbyTime = 5
```

```
property Vector3 spawnPos = Vector3(-2.5,2.5,0)
```

```
property number timerID = 0
```

```
property number spawnEntityCount = 50
```

```
property number maxSpawnCount = 0
```

```
property number currentSpawnCount = 0
```

```

property number remainMonsterCount = 0

property number maxRemainMonsterCount = 0

property boolean isFinishData = false

property boolean pauseSpawn = false

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 시작시 waveIndex를 1로 초기화합니다.
self.waveIndex = 1
-- 현재 몬스터 수를 초기화합니다.
_UIManageLogic:UpdateMonsterCount(self.remainMonsterCount,
self.maxRemainMonsterCount)
-- 웨이브 실행을 요청합니다.
self:InitWave()
end

@ExecSpace("ClientOnly")
method void InitWave()
-- 현재 웨이브 카운트를 UI 업데이트합니다.
_UIManageLogic:UpdateWaveCount(self.waveIndex)
-- 스폰 데이터를 가져옵니다.
local getSpawnData = _MonsterSpawnLogic:GetSpawnList(self.waveIndex)
if not isValid(getSpawnData) then
-- 만약 다음 스폰 데이터가 없을 경우 데이터가 없는 상태로 전환합니다.
self.isFinishData = true
-- 스폰 데이터가 없으므로 결과창을 호출합니다.
_GameLogic:NeedResult()
return
end
-- 최대 스폰하는 몬스터의 수를 스폰 데이터의 몬스터 스폰 수로 등록합니다.
self.maxRemainMonsterCount = getSpawnData.SpawnMonsterCount
-- 몬스터를 스폰해야 하는 수를 스폰 데이터의 몬스터 스폰 수로 등록합니다.
self.maxSpawnCount = getSpawnData.SpawnMonsterCount
-- MonsterID를 스폰 데이터에서 가져옵니다.
local monsterID = getSpawnData.SpawnMonster
-- 스폰 가능한 수를 0으로 초기화합니다.
local canSpawnCount = 0
-- spawnList를 사용해야 하는 몬스터를 Key로 가지는 monsterList로 등록합니다.
local spawnList = self.monsterList[monsterID]

-- 몬스터 풀을 점검 및 생성합니다.
if not isValid(spawnList) or not next(spawnList) then
-- 풀이 아예 비어있다면 초기 세팅 개수만큼 생성합니다.
for i = 1, self.spawnEntityCount do
self:SpawnMonster(monsterID)
end
else
-- 이미 풀이 있다면 비활성화되어 당장 스폰 가능한 개수를 파악합니다.
for i = 1, #spawnList do
if not spawnList[i].Enable then
canSpawnCount = canSpawnCount + 1

```



```

 end
 end

 -- 가용 개수가 웨이브의 최대 스폰량보다 적다면 모자란 만큼 추가 생성합니다.
 if canSpawnCount < self.maxSpawnCount then
 local needCount = self.maxSpawnCount - canSpawnCount
 for i = 1, needCount do
 self:SpawnMonster(monsterID)
 end
 end
end

-- 몬스터 출력을 Timer를 이용하여 일정 시간에 맞춰 출력시킵니다.
self.timerID = _TimerService:SetTimerRepeat(function()
self:PlayMonster(monsterID) end, 1.0, self.standbyTime)
end

@ExecSpace("ClientOnly")
method void SpawnMonster(string MonsterID)
-- 전달받은 MonsterID를 기준으로 몬스터 정보를 가져옵니다.
local getMonsterData = _MonsterSpawnLogic:GetMonsterDataByMonsterID(MonsterID)
-- 만약 사용 불가능한 MonsterData가 들어올 경우 코드 실행을 중단합니다.
if not isValid(getMonsterData) then
 return
end
-- 찾은 데이터를 기준으로 몬스터 Model을 찾아 스폰합니다.
local modelEntity = _SpawnService:SpawnByModelId(getMonsterData.ModelID,
getMonsterData.MonsterID, self.spawnPos, self.Entity)

-- 풀에 넣기 전 일단 화면에서 숨김 처리합니다.
modelEntity:SetEnable(false)
-- 생성한 modelEntity에 몬스터 정보를 등록합니다.
modelEntity.MonsterController:InitData(getMonsterData, self.spawnPos, self)
-- 만약 MonsterID를 Key로 가지는 monsterList가 존재하지 않을 경우 초기화합니다.
if not isValid(self.monsterList[MonsterID]) then
 self.monsterList[MonsterID] = {}
end
-- 생성한 몬스터 엔티티를 monsterList에 MonsterID를 Key로 등록합니다.
table.insert(self.monsterList[MonsterID], modelEntity)
end

@ExecSpace("ClientOnly")
method void PlayMonster(string MonsterID)
-- monsterList에서 전달받은 MonsterID를 Key로 하는 요소들을 spawnList에 등록합니다.
local spawnList = self.monsterList[MonsterID]

-- 타이머가 1초마다 호출될 때, 대기 중인 몬스터 한 마리를 찾아 스폰합니다.
for i = 1, #spawnList do
 local targetMonster = spawnList[i]

 -- 현재 화면에 없고 쉬고 있는 몬스터를 찾습니다.
 if not targetMonster.Enable then
 targetMonster:SetEnable(true)
 targetMonster.MonsterController:SetEnableState()
 end
end

```

```

 -- 스폰 카운트를 증가시켜 현재 몬스터 스폰 수를 증가시킵니다.
 self.currentSpawnCount = self.currentSpawnCount + 1
 -- UI 표기를 위한 값인 remainMonsterCount를 증가시킵니다.
 self.remainMonsterCount = self.remainMonsterCount + 1
 -- UIManageLogic에게 현재 상태를 업데이트하도록 요청합니다.
 _UIManageLogic:UpdateMonsterCount(self.remainMonsterCount,
self.maxRemainMonsterCount)
 -- 한 마리만 스폰하고 반복문을 종료합니다.
 break
 end
end

-- 웨이브 목표량을 모두 스폰했다면 다음 웨이브 전환 조건을 확인합니다.
if self.currentSpawnCount >= self.maxSpawnCount then
 self.currentSpawnCount = 0
 -- 스폰을 담당하던 타이머를 정지합니다.
 _TimerService:ClearTimer(self.timerID)

 if self.remainMonsterCount <= 0 then
 -- 만약 필드에 남은 몬스터가 없다면 즉시 다음 웨이브로 넘어갑니다.
 self.waveIndex = self.waveIndex + 1
 self:InitWave()
 else
 -- 필드에 남은 몬스터가 있다면 스폰 대기 상태(pauseSpawn)로 진입합니다.
 self.pauseSpawn = true
 _GameLogic:NeedTimer()
 end
end
end
end

@ExecSpace("ClientOnly")
method void UpdateRemainMonsterCount(number ChangeValue)
 -- 전달받은 값을 remainMonsterCount에 더합니다. (보통 몬스터 처치 시 -1 전달)
 self.remainMonsterCount = self.remainMonsterCount + ChangeValue

 -- 남은 몬스터 수가 0 밑으로 떨어지지 않도록 방어 코드를 추가해주는 것이 안전합니다.
 if self.remainMonsterCount < 0 then
 self.remainMonsterCount = 0
 end

 -- 변경된 값을 UI에 업데이트하도록 요청합니다.
 _UIManageLogic:UpdateMonsterCount(self.remainMonsterCount,
self.maxRemainMonsterCount)

 -- 스폰 대기 상태(pauseSpawn)이면서 남은 몬스터가 모두 처치(0)되었다면 다음 웨이브를 시작합니
 다.
 if self.pauseSpawn == true and self.remainMonsterCount == 0 then
 self.pauseSpawn = false -- 대기 상태 해제
 self.waveIndex = self.waveIndex + 1 -- 웨이브 증가
 _GameLogic:DisableTimer()
 self:InitWave() -- 다음 웨이브 스폰 시작
 end
end
end

```

```

@ExecSpace("ClientOnly")
method void ChangeResultUI()
-- monsterList 전체를 순회합니다.
for monsterID, spawnList in pairs(self.monsterList) do
-- 각 ID 리스트 안에 담긴 엔티티들을 순회합니다.
for i = 1, #spawnList do
local monster = spawnList[i]
-- 엔티티가 유효한지 확인 후 비활성화합니다.
if invalid(monster) then
monster:SetEnable(false)
end
end
end
end
_TimerService:ClearTimer(self.timerID)
end

@ExecSpace("ClientOnly")
method void CallRestart()
-- 모든 값들을 초기화 한 뒤 InitWave를 실행합니다.
self.isFinishData = false
self.pauseSpawn = false
self.remainMonsterCount = 0
self.currentSpawnCount = 0
self.waveIndex = 1
self:InitWave()
end

end

```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

## 최소 구현 기준

- 월드가 시작되면 일정 시간 뒤 몬스터가 DataSet에 등록된 값으로 스폰됩니다.
- DataSet의 값을 변경하여 스폰되는 몬스터의 수를 변경할 수 있습니다.

## 응용 및 확장 방향

- 웨이브에 따라 제한 시간을 제한할 수 있도록 DataSet을 확장합니다.
- 한 웨이브에 2종류 이상의 몬스터가 스폰될 수 있도록 기능을 수정합니다.
- 특정 웨이브의 경우 보스와 같이 강력한 몬스터가 스폰되도록 기능을 변경합니다.

---

## [💡 기초 학습] 오브젝트 풀 생성하기

---

## 영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 맵 타입 이해하기: ``lua @Component script MonsterSpawnController extends Component

```
property table monsterList = {}
```

```
property integer waveIndex = 0
```

```
property number standbyTime = 5
```

```
property Vector3 spawnPos = Vector3(-2.5,2.5,0)
```

```
property number timerID = 0
```

```
property number spawnEntityCount = 50
```

```
property number maxSpawnCount = 0
```

```
property number currentSpawnCount = 0
```

```
property number remainMonsterCount = 0
```

```
property number maxRemainMonsterCount = 0
```

```
property boolean isFinishData = false
```

```
property boolean pauseSpawn = false
```

```
@ExecSpace("ClientOnly") method void OnBeginPlay() -- 시작시 waveIndex를 1로 초기화합니다. self.waveIndex = 1 -- 현재 몬스터 수를 초기화합니다. _UIManageLogic:UpdateMonsterCount(self.remainMonsterCount, self.maxRemainMonsterCount) -- 웨이브 실행을 요청합니다. self:InitWave() end
```

```
@ExecSpace("ClientOnly") method void InitWave() -- 현재 웨이브 카운트를 UI 업데이트합니다. _UIManageLogic:UpdateWaveCount(self.waveIndex) -- 스폰 데이터를 가져옵니다. local getSpawnData = _MonsterSpawnLogic:GetSpawnList(self.waveIndex) if not isvalid(getSpawnData) then -- 만약 다음 스폰 데이터가 없을 경우 데이터가 없는 상태로 전환합니다. self.isFinishData = true -- 스폰 데이터가 없으므로 결과창을 호출합니다. _GameLogic:NeedResult() return end -- 최대 스폰하는 몬스터의 수를 스폰 데이터의 몬스터 스폰 수로 등록합니다. self.maxRemainMonsterCount = getSpawnData.SpawnMonsterCount -- 몬스터를 스폰해야 하는 수를 스폰 데이터의 몬스터 스폰 수로 등록합니다. self.maxSpawnCount = getSpawnData.SpawnMonsterCount -- MonsterID를 스폰 데이터에서 가져옵니다. local monsterID = getSpawnData.SpawnMonster -- 스폰 가능한 수를 0으로 초기화합니다. local canSpawnCount = 0 -- spawnList를 사용해야 하는 몬스터를 Key로 가지는 monsterList로 등록합니다. local spawnList = self.monsterList[monsterID]
```

```
-- 몬스터 풀을 점검 및 생성합니다. if not isvalid(spawnList) or not next(spawnList) then -- 풀이 아예 비어있다면 초기 세팅 개수만큼 생성합니다. for i = 1, self.spawnEntityCount do self:SpawnMonster(monsterID) end else -- 이미 풀이 있다면 비활성화되어 당장 스폰 가능한 개수를 파악합니다. for i = 1, #spawnList do if not spawnList[i].Enable then canSpawnCount = canSpawnCount + 1 end end
```

```
-- 가용 개수가 웨이브의 최대 스폰량보다 적다면 모자란 만큼 추가 생성합니다.
if canSpawnCount < self.maxSpawnCount then
 local needCount = self.maxSpawnCount - canSpawnCount
 for i = 1, needCount do
 self:SpawnMonster(monsterID)
 end
end
end
```

end

```
-- 몬스터 출력을 Timer를 이용하여 일정 시간에 맞춰 출력시킵니다. self.timerID =
_TimerService:SetTimerRepeat(function() self:PlayMonster(monsterID) end, 1.0, self.standbyTime) end
```

```
@ExecSpace("ClientOnly") method void SpawnMonster(string MonsterID) -- 전달받은 MonsterID를 기준으로 몬스터 정보를 가져옵니다. local getMonsterData =
_MonsterSpawnLogic:GetMonsterDataByMonsterID(MonsterID) -- 만약 사용 불가능한 MonsterData가 들어올 경우 코드 실행을 중단합니다. if not isvalid(getMonsterData) then return end -- 찾은 데이터를 기준으로 몬스터 Model을 찾아 스폰합니다. local modelEntity = _SpawnService:SpawnByModelId(getMonsterData.ModelID, getMonsterData.MonsterID, self.spawnPos, self.Entity)
```

```
-- 풀에 넣기 전 일단 화면에서 숨김 처리합니다. modelEntity:SetEnable(false) -- 생성한 modelEntity에 몬스터 정보를 등록합니다. modelEntity.MonsterController:InitData(getMonsterData, self.spawnPos, self) -- 만약 MonsterID를 Key로 가지는 monsterList가 존재하지 않을 경우 초기화합니다. if not isvalid(self.monsterList[MonsterID]) then self.monsterList[MonsterID] = {} end -- 생성한 몬스터 엔티티를 monsterList에 MonsterID를 Key로 등록합니다. table.insert(self.monsterList[MonsterID], modelEntity) end
```

```
@ExecSpace("ClientOnly") method void PlayMonster(string MonsterID) -- monsterList에서 전달받은 MonsterID를 Key로 하는 요소들을 spawnList에 등록합니다. local spawnList = self.monsterList[MonsterID]
```

```
-- 타이머가 1초마다 호출될 때, 대기 중인 몬스터 한 마리를 찾아 스폰합니다. for i = 1, #spawnList do local targetMonster = spawnList[i]
```

```
-- 현재 화면에 없고 쉬고 있는 몬스터를 찾습니다.
if not targetMonster.Enable then
 targetMonster:SetEnable(true)
 targetMonster.MonsterController:SetEnableState()
 -- 스폰 카운트를 증가시켜 현재 몬스터 스폰 수를 증가시킵니다.
 self.currentSpawnCount = self.currentSpawnCount + 1
 -- UI 표기를 위한 값인 remainMonsterCount를 증가시킵니다.
 self.remainMonsterCount = self.remainMonsterCount + 1
 -- UIManageLogic에게 현재 상태를 업데이트하도록 요청합니다.
 _UIManageLogic:UpdateMonsterCount(self.remainMonsterCount, self.maxRemainMonsterCount)
 -- 한 마리만 스폰하고 반복문을 종료합니다.
 break
end
end
```

end

```
-- 웨이브 목표량을 모두 스폰했다면 다음 웨이브 전환 조건을 확인합니다. if self.currentSpawnCount >=
self.maxSpawnCount then self.currentSpawnCount = 0 -- 스폰을 담당하던 타이머를 정지합니다.
_TimerService:ClearTimer(self.timerID)
```

```
if self.remainMonsterCount <= 0 then
 -- 만약 필드에 남은 몬스터가 없다면 즉시 다음 웨이브로 넘어갑니다.
 self.waveIndex = self.waveIndex + 1
 self:InitWave()
else
 -- 필드에 남은 몬스터가 있다면 스폰 대기 상태(pauseSpawn)로 진입합니다.
 self.pauseSpawn = true
 _GameLogic:NeedTimer()
end
```

```
end end
```

```
@ExecSpace("ClientOnly") method void UpdateRemainMonsterCount(number ChangeValue) -- 전달받은 값을
remainMonsterCount에 더합니다. (보통 몬스터 처치 시 -1 전달) self.remainMonsterCount =
self.remainMonsterCount + ChangeValue
```

```
-- 남은 몬스터 수가 0 밑으로 떨어지지 않도록 방어 코드를 추가해주는 것이 안전합니다. if self.remainMonsterCount
< 0 then self.remainMonsterCount = 0 end
```

```
-- 변경된 값을 UI에 업데이트하도록 요청합니다.
```

```
_UIManageLogic:UpdateMonsterCount(self.remainMonsterCount, self.maxRemainMonsterCount)
```

```
-- 스폰 대기 상태(pauseSpawn)이면서 남은 몬스터가 모두 처치(0)되었다면 다음 웨이브를 시작합니다. if
self.pauseSpawn == true and self.remainMonsterCount == 0 then self.pauseSpawn = false -- 대기 상태 해제
self.waveIndex = self.waveIndex + 1 -- 웨이브 증가 _GameLogic:DisableTimer() self:InitWave() -- 다음 웨이브 스
폰 시작 end end
```

```
@ExecSpace("ClientOnly") method void ChangeResultUI() -- monsterList 전체를 순회합니다. for monsterID,
spawnList in pairs(self.monsterList) do -- 각 ID 리스트 안에 담긴 엔티티들을 순회합니다. for i = 1, #spawnList do
local monster = spawnList[i] -- 엔티티가 유효한지 확인 후 비활성화합니다. if invalid(monster) then
monster:SetEnable(false) end end end _TimerService:ClearTimer(self.timerID) end
```

```
@ExecSpace("ClientOnly") method void CallRestart() -- 모든 값들을 초기화 한 뒤 InitWave를 실행합니다.
self.isFinishData = false self.pauseSpawn = false self.remainMonsterCount = 0 self.currentSpawnCount = 0
self.waveIndex = 1 self:InitWave() end
```

```
end
```

> 해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

<br>

## 최소 구현 기준

- 월드가 시작되면 일정 시간 뒤 몬스터가 DataSet에 등록된 값으로 스폰됩니다.

- DataSet의 값을 변경하여 스폰되는 몬스터의 수를 변경할 수 있습니다.

<br>

## ## 응용 및 확장 방향

- 웨이브에 따라 제한 시간을 제한할 수 있도록 DataSet을 확장합니다.
- 한 웨이브에 2종류 이상의 몬스터가 스폰될 수 있도록 기능을 수정합니다.
- 특정 웨이브의 경우 보스와 같이 강력한 몬스터가 스폰되도록 기능을 변경합니다.

---

## # [💡 기초 학습] 오브젝트 풀 생성하기

<br>

<br>

<br>

## ## 영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 맵 타입 이해하기: `01:10:50`

<br>

## ## 학습 목표

- 오브젝트 풀 기법이 무엇인지 작동 원리를 이해합니다.
- 오브젝트 풀 기법을 통해 얻는 이점과, 주의해야 하는 항목에 대해 학습합니다.

<br>

## ## 해당 영상 시청 후 수행해 볼 내용

- 오브젝트 풀에 대한 이해와 사용 방법을 학습합니다.
- 최적화를 위해 제작중인 월드에서 생성해야 하는 엔티티의 개수를 최적화합니다.

<br>

## ## 오브젝트 풀이란?

<p align="center">



</p>

- 오브젝트 풀은 동일한 엔티티를 \*\*미리\*\* 만들어 사용하는 기법입니다.
- 동일한 엔티티를 반복적으로 사용해야 할 경우 오브젝트 풀에 저장된 엔티티를 사용합니다.
- 사용이 완료된 엔티티는 다시 오브젝트 풀에 \*\*대기\*\* 상태로 전환됩니다.

<br>

## ## 오브젝트 풀의 강점

<p align="center">



</p>

- **\*\*성능 향상\*\***
  - 매번 객체를 새롭게 생성하고 파괴하는 행위를 반복할 경우 CPU의 부하가 강해집니다.
  - 객체가 많이 생성되고 파괴될 수록 CPU의 부하가 강해지며 프레임 저하의 문제로 이어지게 됩니다.
  - 오브젝트 풀은 미리 생성한 객체를 불러오고, 다시 저장하는 방식이므로 생성과 파괴를 반복하는 것 보다 CPU의 점유율이 크게 내려갑니다.
- **\*\*빠른 활성화, 비활성화\*\***
  - 미리 생성된 객체를 켜고 끄는 행위를 하기 때문에 객체를 생성, 파괴하는 것 보다 속도가 빠릅니다.
- **\*\*가비지 컬렉션(GC) 감소\*\***
  - 객체가 파괴되면 불필요한 메모리가 남게 됩니다.
  - 생성, 파괴의 방식으로 제작할 경우 가비지 컬렉션이 빈번하게 호출됩니다.
  - 오브젝트 풀은 가비지 컬렉션을 적게 호출하도록 만듭니다.
  - 가비지 컬렉션의 호출이 적을수록 월드의 성능은 향상됩니다.
- **\*\*메모리 관리 효율화\*\***
  - 오브젝트 풀은 필요한 객체를 미리 메모리에 올려두고 사용합니다.
  - 따라서 메모리의 사용량을 예측하기 쉽고 메모리의 파편화를 방지할 수 있습니다.

<br>

###  가비지 컬렉션(GC)이란?

<p align="center">



</p>

- 가비지 컬렉션은 메모리에서 사용되지 않는 힙(Heap) 영역의 객체를 자동으로 제거합니다.
- 사용되지 않는 객체를 제거함으로 메모리의 여유 공간을 확보합니다.
- 가비지 컬렉션은 불필요한 메모리가 지속적으로 쌓이는걸 방지하는 역할을 수행합니다.

<p align="center">



</p>

- 가비지 컬렉션(GC)이 작동하는 동안 스레드가 멈추어 성능 저하가 발생할 수 있습니다.
- 또한 청소되는 메모리가 많을수록 대기 시간도 길어지게 됩니다.
- 월드가 진행되는 동안 플레이어가 대기해야 하는 시간이 길 수록 사용자는 불편함을 느끼게 됩니다.

<p align="center">



</p>

- 따라서 메모리의 관리를 가비지 컬렉션(GC)에 의존하는게 아닌 불필요한 메모리를 최소화 하도록 만들어야 합니다.
- 적은 양의 메모리라면 가비지 컬렉션(GC)이 처리해야 하는 양이 적어지므로 게임의 성능을 크게 향상시킬 수 있습니다.

<br>

## 오브젝트 풀의 단점



```
<p align="center">

</p>
```

- **\*\*객체의 등록으로 소모되는 메모리\*\***
  - 오브젝트 풀은 미리 객체를 생성하여 메모리에 등록합니다.
  - 만약 등록은 되었으나 사용이 이뤄지지 않을 경우 메모리에 존재만 하고 사용은 안되는 불필요한 메모리가 발생할 수 있습니다.
  - 이런 객체가 많아질수록 낭비되는 메모리가 늘어나게 됩니다.
- **\*\*적절한 수량의 조절\*\***
  - 오브젝트 풀을 생성하기 위해 생성하는 객체의 수를 적절하게 조절해야 합니다.
  - 이를 위해 게임에서 사용되는 양을 예측하고, 재사용성을 감안하여 적절한 수량을 생성하도록 제작해야 합니다.
- **\*\*예외 사항에 대한 취약점\*\***
  - 만약 객체의 사용량 예측에 실패할 경우 구현 상태에 따라 문제가 발생할 수 있습니다.
    - EX: 사용 가능한 오브젝트 풀 내 객체가 없을 경우
      - 사용 가능한 객체가 나올 때 까지 대기한다.
      - 새롭게 객체를 추가로 생성한다.
    - 사용 가능한 객체가 나올 때 까지 대기할 경우 사용자는 불쾌한 경험을 하게 됩니다.
    - 새롭게 객체를 생성할 경우 CPU 사용량이 상승하기 때문에 게임의 성능이 저하될 수 있습니다.
  - 따라서 적절한 수량의 조절이 최우선의 과제입니다.

## ## 샘플 월드의 오브젝트 풀 사용 상태

```
<p align="center">

</p>
```

- 몬스터를 호출하면 오브젝트 풀에 있는 몬스터를 요구합니다.
- 만약 오브젝트 풀에 몬스터가 없을 경우 오브젝트 풀은 몬스터를 생성합니다.
- 이후 사용 가능한 몬스터가 있을 경우, 사용 가능한 객체를 반환합니다.

<br>

## ## 최소 구현 기준

- 오브젝트 풀을 통해 몬스터가 관리될 수 있어야 합니다.
- 사용 가능한 객체가 없을 경우 추가적으로 객체를 생성하여 사용할 수 있어야 합니다.

<br>

## ## 응용 및 확장 방향

- 월드가 시작될 때 미리 오브젝트 풀에 생성하여 사용할 수 있도록 기능을 구성합니다.
- 몬스터 또는 캐릭터를 적절한 수 까지 생성하여 메모리가 과도하게 사용되지 않도록 생성 수량을 조절합니다.
- 리소스 로딩 기능과 함께 활용하여 월드에 진입한 이후에는 추가적인 로딩이 필요하지 않도록 월드를 수정합니다.

---

# [💡 기초 학습] 소환 시스템 구현하기

<br>

<br>

<br>

## ## 영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 맵 타입 이해하기: `01:15:40`

<br>

## ## 학습 목표

- 플레이어가 캐릭터를 소환할 수 있도록 버튼과 기능을 구현합니다.
- 무한히 캐릭터를 소환하지 못하도록 재화와 소환을 검증하도록 기능을 변경합니다.
- 몬스터 처치시 재화를 지급하여 추가적인 소환이 가능하도록 기능을 변경합니다.

<br>

## ## 해당 영상 시청 후 수행해 볼 내용

- 플레이어가 캐릭터를 소환할 수 있도록 기능을 구성합니다.
- 소환 재화 부족, 소환 슬롯이 부족할 경우 이를 안내하는 메시지가 출력되도록 기능을 구성합니다.

<br>

## ## 소환 기능 구현하기

<p align="center">



</p>

- 플레이어가 캐릭터를 소환하여 몬스터를 처치할 수 있도록 기능을 구현합니다.
- 캐릭터를 소환하기 위해 조건을 지정하고, 소환에 따른 재화를 감소시켜 무한하게 소환할 수 없도록 제약을 설정합니다.
- 플레이어는 소환을 위한 재화의 필요성을 느끼고 재화를 모으기 위해 월드를 진행하게 됩니다.

<br>

## ## UI 제작하기

- 플레이어가 버튼을 눌러 캐릭터를 소환할 수 있도록 UI를 구성해야 합니다.
- UI의 버튼을 클릭하여 소환을 요청하고, 조건 충족 여부 판단과 소환을 수행할 수 있도록 기능을 구성합니다.

<br>

## ### UI Groups 제작하기

<p align="center">



</p>

- 메이플스토리 월드 에디터 상단의 UI 버튼을 클릭합니다.

- 메이플스토리 월드 에디터 하단의 UI Groups 창에서 `+` 버튼을 클릭합니다.
- 새롭게 UI Group이 생성되었다면, `Hierarchy`에서 생성한 UI Group의 이름을 변경합니다.
  - 샘플 월드에서는 InGameUI로 제작했습니다.

<br>

### ### 버튼 제작하기

<p align="center">  
   
 </p>

- `Hierarchy`에서 InGameUI를 마우스 우클릭합니다.
- `Create Entity`를 클릭하여 메뉴를 엽니다.
- `Create UIEmpy`를 클릭하여 빈 엔티티를 생성합니다.
- 생성한 엔티티 이름을 SummonButton으로 지정합니다.

<p align="center">  
   
 </p>

- 제작한 엔티티에 다음의 Component들을 부여합니다.
- `SpriteGUIRendererComponent`
  - 버튼 이미지를 출력하기 위한 Component입니다.
- `ButtonComponent`
  - 버튼의 기능을 수행하기 위한 Componnet입니다.

<p align="center">  
   
 </p>

- `Hierarchy`에서 SummonButton의 자식 엔티티로 Text를 생성합니다.
- 생성한 Text 엔티티에 `TextGUIRendererComponent`를 추가합니다.

<br>

### ### Component 편집하기

<p align="center">  
   
 </p>

- `Hierarchy`에서 SummonButton의 Property를 편집합니다.
- `SpriteGUIRendererComponent`의 ImageUID에 버튼 이미지를 등록합니다.
- `UITransformComponent`의 Width, Height, Pos값을 조정하여 버튼의 디자인을 편집합니다.

<p align="center">  
   
 </p>

- Text 엔티티의 `TextGUIRendererComponent`를 편집합니다.
- Text에 버튼의 이름을 작성합니다.
  - 샘플 월드에서는 \*\*소환\*\*으로 출력하기 위해 StringData의 Key값을 입력했습니다.
- 만약 LocaleDataSet의 값을 사용하고자 할 경우 IsLocalizationKey의 체크를 활성화합니다.

<br>

#### #### LocaleDataSet

<p align="center">



</p>

- `Workspace`에서 `MyDesk`를 우클릭합니다.
- `Create LocaleDataSet`을 클릭합니다.
- `Default`를 클릭하여 LocaleDataSet을 제작합니다.
- LocaleDataSet은 로컬라이징을 위한 DataSet입니다.

<p align="center">



</p>

- `Key`
  - 번역을 위해 사용되는 값입니다.
  - `TextGUIRendererComponent`의 Text에 해당 값을 입력한 뒤 IsLocalizationKey를 활성화 하면 Source, ko, en등 번역된 결과를 출력할 수 있습니다.
- `Source`
  - 번역의 기준이 되는 값입니다.
  - 메이플스토리 월드의 자동 번역 기능을 사용 할 경우 해당 값을 기준으로 결과값을 가져옵니다.
- `Note`
  - 다른 월드 크리에이터에게 해당 값을 어디에서 사용할 것 인지 정보를 전달하기 위한 값입니다.
- `ko, en, zh-tw...`
  - 한국어, 영어, 중국어와 같이 월드가 실행되는 국가의 언어에 맞춰 입력한 값이 출력됩니다.

<br>

#### ## Script 제작하기

<p align="center">



</p>

- `Hierarchy`에서 InGameUI에 InGameUIController Component를 등록합니다.
- 이후 아래의 코드를 작성합니다.

```
```lua
```

```
@Component
```

```
script InGameUIController extends Component
```

```

property TextGUIRendererComponent monsterCountText = nil

property TextGUIRendererComponent waveCountText = nil

property TextGUIRendererComponent summonTokenText = nil

property Entity timerEntity = nil

property TextGUIRendererComponent timerText = nil

@ExecSpace("ClientOnly")
method void OnBeginPlay()
self:InitMonsterCountText()
self:InitWaveCountText()
self:InitTimer()
self:InitSummonToken()
end

@ExecSpace("ClientOnly")
method void InitMonsterCountText()
-- monsterCountText가 nil 상태인지 점검합니다.
if not isvalid(self.monsterCountText) then
-- 경로를 기준으로 CountText 엔티티를 찾습니다.
local findEntity =
_EntityService:GetEntityByPath("/ui/InGameUI/MonsterCount/CountText")
-- 정상적으로 엔티티를 찾는데 성공했다면 찾은 엔티티의 TextGUIRendererComponent를 등록
합니다.
if isvalid(findEntity) then
self.monsterCountText = findEntity.TextGUIRendererComponent
end
end
end

@ExecSpace("ClientOnly")
method void CountLeftMonster(number CurrentMonsterCount, number MaxMonsterCount)
-- 현재 남은 몬스터 수와 현재 웨이브에서 생성되는 최대 몬스터 수를 받아온 뒤 UI에 갱신합니다.
if not isvalid(self.monsterCountText) then
self:InitMonsterCountText()
end
self.monsterCountText.Text =
_LocalizationService:GetTextFormat("UI_RemainMonsterCount",CurrentMonsterCount,
MaxMonsterCount)
end

@ExecSpace("ClientOnly")
method void InitWaveCountText()
-- waveCountText가 nil인지 검사합니다.
if not isvalid(self.waveCountText) then
-- 만약 nil일 경우 경로를 기반으로 WaveCountText 엔티티를 찾습니다.
local findText =
_EntityService:GetEntityByPath("/ui/InGameUI/WaveCount/WaveCountText")
if isvalid(findText) then
-- 정상적으로 찾는데 성공했다면 찾은 엔티티의 TextGUIRendererComponent를 등록합니

```

```

다.
        self.waveCountText = findText.TextGUIRendererComponent
    end
end
end

@ExecSpace("ClientOnly")
method void UpdateWaveCount(number WaveCount)
-- waveCountText가 nil일 경우 초기화를 요청합니다.
if not isvalid(self.waveCountText) then
    self:InitWaveCountText()
end
-- 전달받은 현재 웨이브 값으로 UI를 갱신합니다.
self.waveCountText.Text = _LocalizationService.GetTextFormat("UI_WaveCount",
WaveCount)
end

@ExecSpace("ClientOnly")
method void InitTimer()
-- timerEntity가 nil인지 점검합니다.
if not isvalid(self.timerEntity) then
    -- 만약 nil일 경우 경로를 통해 LeftTime 엔티티를 찾습니다.
    local findEntity = _EntityService.GetEntityByPath("/ui/InGameUI/LeftTime")
    -- 만약 findEntity가 True일 경우 해당 값을 timerEntity에 적용합니다.
    if isvalid(findEntity) then
        self.timerEntity = findEntity
    end
end
-- timerText가 nil인지 점검합니다.
if not isvalid(self.timerText) then
    -- 만약 nil일 경우, TimerText 엔티티를 찾습니다.
    local findTextEntity =
_EntityService.GetEntityByPath("/ui/InGameUI/LeftTime/TimerText")
    -- 찾은 findTextEntity가 True일 경우 해당 값을 timerText에 적용합니다.
    if isvalid(findTextEntity) then
        self.timerText = findTextEntity.TextGUIRendererComponent
    end
end
if self.timerEntity.Enable == true then
    self.timerEntity.Enable = false
end
end

@ExecSpace("ClientOnly")
method void UpdateTimer(number deltaTimer)
-- 전달받은 deltaTimer 값을 기준으로 남은 시간 UI를 갱신합니다.
if self.timerEntity.Enable == false then
    self.timerEntity.Enable = true
end

local timeString = string.format("%.2f", deltaTimer)

self.timerText.Text = "LeftTime\n"..timeString
end

```

```

@ExecSpace("ClientOnly")
method void DisableTimer()
-- Timer를 비활성화하도록 요청합니다.
self.timerEntity.Enable = false
end

@ExecSpace("ClientOnly")
method void InitSummonToken()
-- summonTokenText가 사용 가능한 상태인지 점검합니다.
if not isvalid(self.summonTokenText) then
-- 만약 summonTokenText가 nil일 경우 경로를 기준으로 SummonTokenText를 찾습니다.
local findEntity =
_EntityService:GetEntityByPath("/ui/InGameUI/SummonToken/IconImage/SummonTokenText")
if isvalid(findEntity) then
-- SummonTokenText를 찾았다면 해당 엔티티의 TextGUIRendererComponent를 등록합니다.
self.summonTokenText = findEntity.TextGUIRendererComponent
end
end

@ExecSpace("ClientOnly")
method void UpdateSummonTokenUI(number TokenValue)
-- summonTokenText가 nil일 경우 초기화를 시도합니다.
if not isvalid(self.summonTokenText) then
self:InitSummonToken()
end
-- 전달받은 TokenValue를 기준으로 남은 토큰 수 UI를 갱신합니다.
self.summonTokenText.Text = tostring(math.floor(TokenValue))
end

@EventSender("Entity", "b3964a11-f347-420c-a354-2698b88a6181")
handler HandleButtonClickEvent(ButtonClickEvent event)
-- 소환하기 버튼을 클릭 할 경우 캐릭터 소환을 실행합니다.
_CharacterGachaLogic:RequestSpawnCharacter()
end

end

```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

최소 구현 기준

- 플레이어가 UI를 통해 캐릭터를 소환할 수 있어야 합니다.
- 소환에 따라 소환 재화의 차감, 예외 사항에 대한 안내 UI가 출력되어야 합니다.

응용 및 확장 방향

- 플레이어가 캐릭터 슬롯을 비울 수 있는 수단(캐릭터 판매, 캐릭터 합성 등)을 추가하여 지속적으로 캐릭터를 소환할 수 있도록 기능을 추가합니다.
- 소환 재화를 점점 비싸게 만들어 재화를 부족한 환경을 조성합니다.
- 소환 재화의 비용 증가와 함께 캐릭터 강화 기능을 추가하여 플레이어가 재화를 선택하고, 전략적으로 사용할 수 있도록 기능을 확장합니다.

[📖 심화 학습] 저장 및 불러오기 구현하기

영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 맵 타입 이해하기: ``lua @Component script InGameUIController extends Component

```
property TextGUIRendererComponent monsterCountText = nil
```

```
property TextGUIRendererComponent waveCountText = nil
```

```
property TextGUIRendererComponent summonTokenText = nil
```

```
property Entity timerEntity = nil
```

```
property TextGUIRendererComponent timerText = nil
```

```
@ExecSpace("ClientOnly") method void OnBeginPlay() self:InitMonsterCountText() self:InitWaveCountText()
self:InitTimer() self:InitSummonToken() end
```

```
@ExecSpace("ClientOnly") method void InitMonsterCountText() -- monsterCountText가 nil 상태인지 점검합니다.
if not isvalid(self.monsterCountText) then -- 경로를 기준으로 CountText 엔티티를 찾습니다. local findEntity =
_EntityService:GetEntityByPath("/ui/InGameUI/MonsterCount/CountText") -- 정상적으로 엔티티를 찾는데 성공했
다면 찾은 엔티티의 TextGUIRendererComponent를 등록합니다. if isvalid(findEntity) then self.monsterCountText
= findEntity.TextGUIRendererComponent end end end
```

```
@ExecSpace("ClientOnly") method void CountLeftMonster(number CurrentMonsterCount, number
MaxMonsterCount) -- 현재 남은 몬스터 수와 현재 웨이브에서 생성되는 최대 몬스터 수를 받아온 뒤 UI에 갱신합니
다. if not isvalid(self.monsterCountText) then self:InitMonsterCountText() end self.monsterCountText.Text =
_LocalizationService:GetTextFormat("UI_RemainMonsterCount",CurrentMonsterCount, MaxMonsterCount) end
```

```
@ExecSpace("ClientOnly") method void InitWaveCountText() -- waveCountText가 nil인지 검사합니다. if not
isvalid(self.waveCountText) then -- 만약 nil일 경우 경로를 기반으로 WaveCountText 엔티티를 찾습니다. local
findText = _EntityService:GetEntityByPath("/ui/InGameUI/WaveCount/WaveCountText") if isvalid(findText) then
-- 정상적으로 찾는데 성공했다면 찾은 엔티티의 TextGUIRendererComponent를 등록합니다. self.waveCountText =
findText.TextGUIRendererComponent end end end
```



```
@ExecSpace("ClientOnly") method void UpdateWaveCount(number WaveCount) -- waveCountText가 nil일 경우 초기화를 요청합니다. if not isvalid(self.waveCountText) then self.InitWaveCountText() end -- 전달받은 현재 웨이브 값으로 UI를 갱신합니다. self.waveCountText.Text = _LocalizationService.GetTextFormat("UI_WaveCount", WaveCount) end
```

```
@ExecSpace("ClientOnly") method void InitTimer() -- timerEntity가 nil인지 점검합니다. if not isvalid(self.timerEntity) then -- 만약 nil일 경우 경로를 통해 LeftTime 엔티티를 찾습니다. local findEntity = _EntityService.GetEntityByPath("/ui/InGameUI/LeftTime") -- 만약 findEntity가 True일 경우 해당 값을 timerEntity에 적용합니다. if isvalid(findEntity) then self.timerEntity = findEntity end end -- timerText가 nil인지 점검합니다. if not isvalid(self.timerText) then -- 만약 nil일 경우, TimerText 엔티티를 찾습니다. local findTextEntity = _EntityService.GetEntityByPath("/ui/InGameUI/LeftTime/TimerText") -- 찾은 findTextEntity가 True일 경우 해당 값을 timerText에 적용합니다. if isvalid(findTextEntity) then self.timerText = findTextEntity.TextGUIRendererComponent end end if self.timerEntity.Enable == true then self.timerEntity.Enable = false end end
```

```
@ExecSpace("ClientOnly") method void UpdateTimer(number deltaTimer) -- 전달받은 deltaTimer 값을 기준으로 남은 시간 UI를 갱신합니다. if self.timerEntity.Enable == false then self.timerEntity.Enable = true end
```

```
local timeString = string.format("%.2f", deltaTimer)
```

```
self.timerText.Text = "LeftTime\n"..timeString end
```

```
@ExecSpace("ClientOnly") method void DisableTimer() -- Timer를 비활성화하도록 요청합니다. self.timerEntity.Enable = false end
```

```
@ExecSpace("ClientOnly") method void InitSummonToken() -- summonTokenText가 사용 가능한 상태인지 점검합니다. if not isvalid(self.summonTokenText) then -- 만약 summonTokenText가 nil일 경우 경로를 기준으로 SummonTokenText를 찾습니다. local findEntity = _EntityService.GetEntityByPath("/ui/InGameUI/SummonToken/IconImage/SummonTokenText") if isvalid(findEntity) then -- SummonTokenText를 찾았다면 해당 엔티티의 TextGUIRendererComponent를 등록합니다. self.summonTokenText = findEntity.TextGUIRendererComponent end end end
```

```
@ExecSpace("ClientOnly") method void UpdateSummonTokenUI(number TokenValue) -- summonTokenText가 nil일 경우 초기화를 시도합니다. if not isvalid(self.summonTokenText) then self.InitSummonToken() end -- 전달받은 TokenValue를 기준으로 남은 토큰 수 UI를 갱신합니다. self.summonTokenText.Text = tostring(math.floor(TokenValue)) end
```

```
@EventSender("Entity", "b3964a11-f347-420c-a354-2698b88a6181") handler  
HandleButtonClickEvent(ButtonClickEvent event) -- 소환하기 버튼을 클릭 할 경우 캐릭터 소환을 실행합니다.  
_CharacterGachaLogic.RequestSpawnCharacter() end
```

```
end
```

> 해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

최소 구현 기준

- 플레이어가 UI를 통해 캐릭터를 소환할 수 있어야 합니다.

- 소환에 따라 소환 재화의 차감, 예외 사항에 대한 안내 UI가 출력되어야 합니다.

응용 및 확장 방향

- 플레이어가 캐릭터 슬롯을 비울 수 있는 수단(캐릭터 판매, 캐릭터 합성 등)을 추가하여 지속적으로 캐릭터를 소환할 수 있도록 기능을 추가합니다.
- 소환 재화를 점점 비싸게 만들어 재화를 부족한 환경을 조성합니다.
- 소환 재화의 비용 증가와 함께 캐릭터 강화 기능을 추가하여 플레이어가 재화를 선택하고, 전략적으로 사용할 수 있도록 기능을 확장합니다.

[📖 심화 학습] 저장 및 불러오기 구현하기

영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 맵 타입 이해하기: `01:23:43`

학습 목표

- 플레이어의 최고 기록을 저장하는 기능을 구현합니다.
- 플레이어의 최고 기록을 가져오는 기능을 구현합니다.
- DataStorageService를 활용하여 정보를 가져오는 방법에 대해 학습합니다.

해당 영상 시청 후 수행해 볼 내용

- 플레이어의 최고 기록을 저장하는 기능을 제작합니다.
- 플레이어의 최고 기록을 가져온 뒤 이번에 달성한 웨이브 단계와 비교하는 기능을 구성합니다.

저장 및 불러오기 기능 구현하기

<p align="center">

</p>

- 플레이어의 기록을 저장하고 불러오는 기능을 구현하여 정보가 유실되지 않도록 기록해야 합니다.
- 메이플스토리 월드는 DataStorage를 활용하여 서버에 저장하고 불러오는 기능을 구현할 수 있습니다.
- 다만, 샘플 월드의 경우 클라이언트 기반으로 작동하기 때문에 정식적인 방법은 아닙니다.
- 따라서 여러분들이 샘플 월드를 상용화 수준으로 개발하고자 하실 경우 값들이 서버에서 관리되도록 구현해야 합니다.

DataStorageLogic 제작하기

- DataStorageLogic은 샘플 월드에서 플레이어의 정보를 저장하고 불러오기 위해 제작된 Logic입니다.
- 해당 Logic을 통하여 플레이어의 최고 기록을 저장하거나 불러올 수 있습니다.

```

```lua
@Logic
script DataStorageLogic extends Logic

@Sync
property number serverBestWave = 0

property number clientBestWave = 0

property string storageKey = "BestWave"

@ExecSpace("ServerOnly")
method void SaveBestWave(string userId, number BestWave)
-- 서버에 있는 serverBestWave를 Client에서 전달받은 BestWave 값으로 업데이트합니다.
self.serverBestWave = BestWave
-- userId를 기준으로 해당 user의 데이터를 가져옵니다.
local userDataStorage = _DataStorageService:GetUserDataStorage(userId)
-- 최고 웨이브 값을 저장합니다.
userDataStorage:SetAsync(self.storageKey, tostring(self.serverBestWave),
function(errorCode, key)
 if errorCode == 0 then
 log(userId .. "의 최고 Wave값 저장 완료! : " .. self.serverBestWave)
 else
 log("저장 실패! 에러 코드: " .. errorCode)
 end
end)
end

@ExecSpace("ServerOnly")
method void LoadBestWave(string userId)
--userId를 기준으로 유저의 정보를 가져옵니다.
local userDataStorage = _DataStorageService:GetUserDataStorage(userId)
-- 가져온 정보에서 storageKey를 가지는 정보를 가져옵니다.
userDataStorage:GetAsync(self.storageKey, function(errorCode, key, value)
-- 정상적으로 가져오는데 성공했다면 serverBestWave 값을 가져온 값으로 갱신합니다.
if errorCode == 0 then
 self.serverBestWave = tonumber(value) or 0
end
end)
end

@ExecSpace("ClientOnly")
method void OnSyncProperty(string name, any value)
-- 만약 serverBestWave의 값이 변경되었다면 clientBestWave의 값도 같이 갱신합니다.
if name == "serverBestWave" then
 self.clientBestWave = value
end
end

@ExecSpace("Server")

```

```

method void RequestLoadBestWave(string userId)
self:LoadBestWave(userId)
end

@ExecSpace("Server")
method void RequestSaveBestWave(string userId, number CurrentBestWave)
self:SaveBestWave(userId, CurrentBestWave)
end

end

```

해당 코드는 PlainText를 기준으로 작성된 코드입니다.

- 코드 내 storageKey의 값이 변경되면 다양한 값들을 저장할 수 있습니다.

## 최소 구현 기준

- 플레이어의 최고 웨이브를 저장하는 기능을 제작합니다.
- 플레이어의 저장된 최고 웨이브를 가져오도록 기능을 구현합니다.

## 응용 및 확장 방향

- 소환 재화와 같이 사용자가 저장해야 하는 값들을 저장하고 불러오는 기능을 구현합니다.
- 값들이 서버에서 관리하고 사용되도록 기능을 구성합니다.

---

## [📖 심화 학습] 오프라인 보상 구현하기

---

### 영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 맵 타입 이해하기: ``lua @Logic script DataStorageLogic extends Logic

```
@Sync property number serverBestWave = 0
```

```
property number clientBestWave = 0
```

```
property string storageKey = "BestWave"
```

```
@ExecSpace("ServerOnly") method void SaveBestWave(string userId, number BestWave) -- 서버에 있는
serverBestWave를 Client에서 전달받은 BestWave 값으로 업데이트합니다. self.serverBestWave = BestWave --
userId를 기준으로 해당 user의 데이터를 가져옵니다. local userDataStorage =
```

```

_DataStorageService:GetUserDataStorage(userId) -- 최고 웨이브 값을 저장합니다.
userDataStorage:SetAsync(self.storageKey, tostring(self.serverBestWave), function(errorCode, key) if errorCode
== 0 then log(userId .. "의 최고 Wave값 저장 완료! : " .. self.serverBestWave) else log("저장 실패! 에러 코드: " ..
errorCode) end end) end

@ExecSpace("ServerOnly") method void LoadBestWave(string userId) --userId를 기준으로 유저의 정보를 가져옵
니다. local userDataStorage = _DataStorageService:GetUserDataStorage(userId) -- 가져온 정보에서 storageKey
를 가지는 정보를 가져옵니다. userDataStorage:GetAsync(self.storageKey, function(errorCode, key, value) -- 정상
적으로 가져오는데 성공했다면 serverBestWave 값을 가져온 값으로 갱신합니다. if errorCode == 0 then
self.serverBestWave = tonumber(value) or 0 end end) end

@ExecSpace("ClientOnly") method void OnSyncProperty(string name, any value) -- 만약 serverBestWave의 값
이 변경되었다면 clientBestWave의 값도 같이 갱신합니다. if name == "serverBestWave" then self.clientBestWave
= value end end

@ExecSpace("Server") method void RequestLoadBestWave(string userId) self:LoadBestWave(userId) end

@ExecSpace("Server") method void RequestSaveBestWave(string userId, number CurrentBestWave)
self:SaveBestWave(userId, CurrentBestWave) end

end

```

> 해당 코드는 PlainText를 기준으로 작성된 코드입니다.

- 코드 내 storageKey의 값이 변경되면 다양한 값들을 저장할 수 있습니다.

<br>

## 최소 구현 기준

- 플레이어의 최고 웨이브를 저장하는 기능을 제작합니다.
- 플레이어의 저장된 최고 웨이브를 가져오도록 기능을 구현합니다.

<br>

## 응용 및 확장 방향

- 소환 재화와 같이 사용자가 저장해야 하는 값들을 저장하고 불러오는 기능을 구현합니다.
- 값들이 서버에서 관리하고 사용되도록 기능을 구성합니다.

---

# [📖 심화 학습] 오프라인 보상 구현하기

<br>

<br>

<br>

## 영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 맵 타입 이해하기: `01:28:07`

<br>

## ## 학습 목표

- 플레이어의 마지막 접속 시간과 현재 시간을 비교하는 기능을 구현합니다.
- 지정한 계산식에 따라 재화를 지급하는 기능을 구현합니다.

<br>

## ## 오프라인 보상

```
<p align="center">

</p>
```

- 오프라인 보상은 플레이어가 월드를 실행하고 있지 않더라도 보상이 축적되는 기능입니다.
- 플레이어가 월드에 재접속하도록 유도하는 장치이면서 플레이어는 보상을 통해 자신의 캐릭터를 강화시킬 수 있습니다.

<br>

## ### Logic 제작하기

- 오프라인 보상을 구현하기 위해 재화를 관리하는 Logic과 시간에 따른 보상을 지급하는 Logic을 제작했습니다.
- 샘플 월드에서는 **\*\*GameLogic\*\***, **\*\*TimeRewardLogic\*\***이란 이름으로 제작되었습니다.
- 아래는 **\*\*GameLogic\*\***, **\*\*TimeRewardLogic\*\***의 코드입니다.

```
```lua
@Logic
script GameLogic extends Logic

property number summonToken = 0

property boolean needTimer = false

property number maxLeftTime = 10.0

property number leftTimeDelta = 0

@ExecSpace("ClientOnly")
method void NeedResult()
-- 현재 맵이 InGame인지 점검합니다.
if _UserService.LocalPlayer.CurrentMapName == "InGame" then
-- InGame 상태일 경우 MonsterSpawnController를 찾아 부여합니다.
local MonsterSpawnController =
_EntityService.GetEntityByPath("/maps/InGame/MonsterSpawnController")
local monsterSpawnComponent = MonsterSpawnController.MonsterSpawnController
local userId = _UserService.LocalPlayer.PlayerComponent.UserId
monsterSpawnComponent:ChangeResultUI()
-- 현재 웨이브와 최고 웨이브를 변수에 저장합니다.
local currentWave = monsterSpawnComponent.waveIndex
local bestWave = _DataStorageLogic.clientBestWave
-- 이번 웨이브의 달성도가 기존 Wave보다 클 경우에만 이를 저장합니다.
if currentWave > bestWave then
_DataStorageLogic.RequestSaveBestWave(userId, currentWave)
```

```

        -- 갱신 직후 클라이언트 값도 즉시 최신화해줍니다.
        _DataStorageLogic.clientBestWave = currentWave
        bestWave = currentWave
    end
    -- 달성한 Wave와 저장된 Wave를 비교하여 UI에 출력되도록 넘겨줍니다.
    _UIManageLogic:ShowResultUI(currentWave, bestWave)
end
end

@ExecSpace("ClientOnly")
method void RestartGame()
    -- 현재 맵 상태가 InGame인지 점검합니다.
    if _UserService.LocalPlayer.CurrentMapName == "InGame" then
        -- MonsterSpawnController를 경로를 기준으로 찾아 등록합니다.
        local MonsterSpawnController =
        _EntityService:GetEntityByPath("/maps/InGame/MonsterSpawnController")
        local monsterSpawnComponent = MonsterSpawnController.MonsterSpawnController
        -- 게임 재시작시 MonsterSpawnController가 수행해야 하는 기능을 요청합니다.
        monsterSpawnComponent:CallRestart()
        -- characterSlotPosGroup을 찾아 등록합니다.
        local characterSlotController =
        _EntityService:GetEntityByPath("/maps/InGame/CharacterSlotPosGroup")
        local characterSlotGroupComponent =
        characterSlotController.CharacterSlotGroupComponent
        -- characetrSlotGroupComponent의 자식 엔티티인 캐릭터들을 모두 지웁니다.
        characterSlotGroupComponent:ClearCharacterList()
        -- 만약 현재 summonToken이 0 이하일 경우 게임의 플레이는 가능하도록 만들기 위해
        summonToken을 1로 변경합니다.
        if self.summonToken <= 0 then
            self:UpdateSummonToken(1)
        end
        self:DisableTimer()
    end
end

@ExecSpace("ClientOnly")
method void OnBeginPlay()
    -- 오프라인 보상을 요청하고 이를 적용합니다.
    self:CallOfflineReward()
    -- 로컬 플레이어의 ID값을 가져온 뒤 해당 유저의 최고 기록을 미리 가져옵니다.
    local userId = _UserService.LocalPlayer.PlayerComponent.ProfileCode
    _DataStorageLogic:RequestLoadBestWave(userId)
end

@ExecSpace("ClientOnly")
method void CallOfflineReward()
    -- offlineReward를 요청한 뒤 이를 적용합니다.
    local getUserId = _UserService.LocalPlayer.PlayerComponent.ProfileCode
    self:RequestOfflineReward(getUserId)
end

@ExecSpace("Server")
method void RequestOfflineReward(string UserID)
    -- 마지막으로 접속했던 시간을 요청합니다.

```

```

_TimeRewardLogic:LoadLastLoginTime(UserID)
end

@ExecSpace("ClientOnly")
method void UpdateSummonToken(number ChangeValue)
-- 전달받은 값으로 SummonToken을 갱신한 뒤 UI를 갱신합니다.
self.summonToken = self.summonToken + ChangeValue
_UIManageLogic:UpdateSummonToken(self.summonToken)
end

@ExecSpace("ClientOnly")
method void NeedTimer()
-- 타이머가 필요한 상태로 전환하여 타이머가 작동하도록 요청합니다.
self.needTimer = true
self.leftTimeDelta = self.maxLeftTime
end

@ExecSpace("ClientOnly")
method void DisableTimer()
-- 웨이브의 적을 모두 처치하여 타이머가 필요 없는 상태로 변경합니다.
self.needTimer = false
_UIManageLogic:DisableTimer()
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- 만약 needTimer가 true일 경우 Timer를 시작합니다.
if self.needTimer then
-- 만약 leftTimeDelta가 0 이하일 경우 게임오버 처리를 시작합니다.
if self.leftTimeDelta <= 0 then
self.needTimer = false
self.leftTimeDelta = 0
_UIManageLogic:UpdateTimer(self.leftTimeDelta)
self:NeedResult()
else
-- 아직 남은 시간이 존재할 경우 시간 감소 및 UI 처리를 수행합니다.
self.leftTimeDelta = self.leftTimeDelta - delta
if self.leftTimeDelta <= 0 then
self.leftTimeDelta = 0
end
_UIManageLogic:UpdateTimer(self.leftTimeDelta)
end
end
end
end
end

```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

```

@Logic
script TimeRewardLogic extends Logic

```



```

property string storageKey = "LastLoginTime"

@Sync
property number offlineReward = 0

@Sync
property boolean offlineRewardComplete = false

@ExecSpace("ServerOnly")
method void SaveCurrentTime(string userId)
    local currentTime = os.time()
    local userDataStorage = _DataStorageService:GetUserDataStorage(userId)

    -- "LastLoginTime"이라는 키로 현재 os.time() 값을 저장합니다.
    userDataStorage:SetAsync(self.storageKey, tostring(currentTime),
function(errorCode, key)
    if errorCode == 0 then
        log(userId .. "의 접속 시간이 성공적으로 저장되었습니다: " .. currentTime)
    else
        log("저장 실패! 에러 코드: " .. errorCode)
    end
end)
end

@ExecSpace("ServerOnly")
method void LoadLastLoginTime(string userId)
    local userDataStorage = _DataStorageService:GetUserDataStorage(userId)

    userDataStorage:GetAsync(self.storageKey, function(errorCode, key, value)
if errorCode == 0 then
    -- 이전에 저장된 시간이 없다면 value는 nil일 수 있습니다.
    local lastTime = tonumber(value) or 0

    -- 앞서 만든 계산 로직을 호출하여 보상 수치를 구합니다.
    self.offlineReward = self:CalculateTimeReward(lastTime)
    -- 보상 수량을 완료했으니 현재 시간을 저장합니다.
    self:SaveCurrentTime(userId)
end
end)
end

@ExecSpace("ServerOnly")
method number CalculateTimeReward(number lastLoginTime)
    -- 마지막 접속 시간과 현재 시간을 비교하여 필요한 보상을 계산하여 지급합니다.
    self.offlineRewardComplete = true
    if lastLoginTime == nil or lastLoginTime == 0 then
        return 1
    end

    local currentTime = os.time()
    local timeDiff = currentTime - lastLoginTime

    if timeDiff < 0 then
        timeDiff = 0
    end
end

```

```

end

if timeDiff <= 1 then
    return 1
else
    -- 10초마다 SummonToken을 1씩 추가로 지급합니다.
    local returnValue = math.floor(timeDiff / 10)
    -- 최대 10개까지만 지급하기 위한 처리를 진행합니다.
    if returnValue >= 10 then
        returnValue = 10
    end
    return returnValue
end

end

@ExecSpace("ClientOnly")
method void OnSyncProperty(string name, any value)
    -- offlineReward 값이 변경되었을 경우 SummonToken을 갱신합니다.
    if name == "offlineReward" then
        _GameLogic:UpdateSummonToken(value)
    end
end

end

```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

- 샘플 월드에서는 미접속 시간의 차이가 10초일 때 소환 토큰 1개씩을 지급하며, 최대 10개까지 지급합니다.
- **TimeRewardLogic**이 계산한 결과값을 **GameLogic**의 summonToken에 대입합니다.
- 플레이어는 summonToken의 여부에 따라 캐릭터를 소환할 수 있습니다.

오프라인 보상 구현시 주의사항

- **보안 및 어뷰징**
 - 사용자가 디바이스의 시간을 설정하여 시간을 미래로 설정할 경우 그만큼의 시간이 흐른 것으로 간주하여 대량의 보상을 지급할 수 있습니다.
- **게임 경제 및 밸런스**
 - 오프라인 보상을 구현했으나 상한치인 캡(Max Cap)을 설정하지 않을 경우 게임의 경제가 크게 무너질 수 있습니다.
- **비율 및 소수점 처리**
 - 1분당 10골드를 지급한다고 가정 할 경우 30초만 지나고 접속했을 경우 기준을 어떻게 설정할 것 인지 문제가 발생합니다.
- **타임존 문제**
 - 이용자가 해외여행을 할 경우 서버에 저장된 시간대와 달라 보상을 더 크게 지급하거나 못받을 수 있습니다.
- **유저 경험 및 예외 처리**

- 만약 인벤토리의 개수 제한이 있는 게임일 경우 오프라인 보상이 슬롯보다 많을 경우 보상의 유실이 발생할 수 있습니다.
- 보상을 수령할 때 강제 종료 또는 네트워크 끊김으로 인해 보상 지급 도중 데이터가 날아갈 수 있음

해결 방안

- 위의 문제들을 해결하기 위한 방안으로 다음과 같은 방법들이 있습니다.
- **보안 및 어뷰징**
 - 값의 처리를 서버에서만 연산하도록 만들어 클라이언트가 개입할 상황을 막는 방법이 있습니다.
- **게임 경제 및 밸런스**
 - 보상의 상한선인 캡(Max Cap)을 만들어 사용한다면 보상이 과도하게 지급되는걸 막을 수 있습니다.
- **비율 및 소수점 처리**
 - 이는 기획적 문제로 규칙을 설정한뒤 해당 규칙에 예외가 발생하지 않도록 조절해야 합니다.
- **타임존 문제**
 - 국제 표준시간(UTC)을 활용하여 이러한 문제를 예방할 수 있습니다.
- **유저 경험 및 예외처리**
 - 보상이 지급되기 전 충분한 여유 공간이 있는지 예외처리를 수행합니다.
 - 네트워크 통신이 필요한 기능일 경우 네트워크 통신에 실패했는지 확인하고 실패했다면 2~3회 반복 요청하여 통신 오류를 줄일 수 있습니다.

해당 영상 시청 후 수행해 볼 내용

- 플레이어에게 접속 시간의 차이에 따른 보상을 지급할 수 있도록 보상 재화를 설계 및 제작합니다.
- 시간에 따라 일정한 비율로 보상을 제공할 수 있도록 로직을 구성, 제작합니다.

최소 구현 기준

- 접속 시간의 편차에 따라 플레이어에게 소환 재화를 지급합니다.
- 플레이어에게 재화가 정상적으로 지급되며, 지급된 시점에 접속 시간을 저장합니다.

응용 및 확장 방향

- 플레이어가 사용 가능한 재화의 종류를 확장하고 오프라인 보상을 통해 획득할 수 있도록 기능을 추가합니다.
- 플레이어의 재화를 저장하고 불러올 수 있도록 제작하여 플레이어가 지속적으로 월드에 접근하도록 콘텐츠를 추가합니다.
- 오프라인 보상을 데이터 테이블을 활용하여 다양한 재화의 지급량을 조절할 수 있도록 구조를 변경합니다.

[💡 기초 학습] UI 구성하기

영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 맵 타입 이해하기: ``lua @Logic script GameLogic extends Logic

```
property number summonToken = 0
```

```
property boolean needTimer = false
```

```
property number maxLeftTime = 10.0
```

```
property number leftTimeDelta = 0
```

```
@ExecSpace("ClientOnly") method void NeedResult() -- 현재 맵이 InGame인지 점검합니다. if
_UserService.LocalPlayer.CurrentMapName == "InGame" then -- InGame 상태일 경우 MonsterSpawnController
를 찾아 부여합니다. local MonsterSpawnController =
_EntityService.GetEntityByPath("/maps/InGame/MonsterSpawnController") local monsterSpawnComponent =
MonsterSpawnController.MonsterSpawnController local userId =
_UserService.LocalPlayer.PlayerComponent.UserId monsterSpawnComponent:ChangeResultUI() -- 현재 웨이브
와 최고 웨이브를 변수에 저장합니다. local currentWave = monsterSpawnComponent.waveIndex local bestWave
= _DataStorageLogic.clientBestWave -- 이번 웨이브의 달성도가 기존 Wave보다 클 경우에만 이를 저장합니다. if
currentWave > bestWave then _DataStorageLogic:RequestSaveBestWave(userId, currentWave) -- 갱신 직후 클
라이언트 값도 즉시 최신화해줍니다. _DataStorageLogic.clientBestWave = currentWave bestWave = currentWave
end -- 달성한 Wave와 저장된 Wave를 비교하여 UI에 출력되도록 넘겨줍니다.
_UIManageLogic:ShowResultUI(currentWave, bestWave) end end
```

```
@ExecSpace("ClientOnly") method void RestartGame() -- 현재 맵 상태가 InGame인지 점검합니다. if
_UserService.LocalPlayer.CurrentMapName == "InGame" then -- MonsterSpawnController를 경로를 기준으로
찾아 등록합니다. local MonsterSpawnController =
_EntityService.GetEntityByPath("/maps/InGame/MonsterSpawnController") local monsterSpawnComponent =
MonsterSpawnController.MonsterSpawnController -- 게임 재시작시 MonsterSpawnController가 수행해야 하는
기능을 요청합니다. monsterSpawnComponent:CallRestart() -- characterSlotPosGroup을 찾아 등록합니다. local
characterSlotController = _EntityService.GetEntityByPath("/maps/InGame/CharacterSlotPosGroup") local
characterSlotGroupComponent = characterSlotController.CharacterSlotGroupComponent --
characterSlotGroupComponent의 자식 엔티티인 캐릭터들을 모두 지웁니다.
characterSlotGroupComponent:ClearCharacterList() -- 만약 현재 summonToken이 0 이하일 경우 게임의 플레이
는 가능하도록 만들기 위해 summonToken을 1로 변경합니다. if self.summonToken <= 0 then
self:UpdateSummonToken(1) end self.DisableTimer() end end
```

```
@ExecSpace("ClientOnly") method void OnBeginPlay() -- 오프라인 보상을 요청하고 이를 적용합니다.
self:CallOfflineReward() -- 코걸 플레이어의 ID값을 가져온 뒤 해당 유저의 최고 기록을 미리 가져옵니다. local userId
= _UserService.LocalPlayer.PlayerComponent.ProfileCode _DataStorageLogic:RequestLoadBestWave(userId)
end
```

```
@ExecSpace("ClientOnly") method void CallOfflineReward() -- offlineReward를 요청한 뒤 이를 적용합니다. local
getUserId = _UserService.LocalPlayer.PlayerComponent.ProfileCode self:RequestOfflineReward(getUserId) end
```

```
@ExecSpace("Server") method void RequestOfflineReward(string UserID) -- 마지막으로 접속했던 시간을 요청합
니다. _TimeRewardLogic:LoadLastLoginTime(UserID) end
```

```
@ExecSpace("ClientOnly") method void UpdateSummonToken(number ChangeValue) -- 전달받은 값으로
SummonToken을 갱신한 뒤 UI를 갱신합니다. self.summonToken = self.summonToken + ChangeValue
_UIManageLogic:UpdateSummonToken(self.summonToken) end
```

```
@ExecSpace("ClientOnly") method void NeedTimer() -- 타이머가 필요한 상태로 전환하여 타이머가 작동하도록 요
청합니다. self.needTimer = true self.leftTimeDelta = self.maxLeftTime end
```

```
@ExecSpace("ClientOnly") method void DisableTimer() -- 웨이브의 적을 모두 처치하여 타이머가 필요 없는 상태로
변경합니다. self.needTimer = false _UIManageLogic:DisableTimer() end
```

```
@ExecSpace("ClientOnly") method void OnUpdate(number delta) -- 만약 needTimer가 true일 경우 Timer를 시
작합니다. if self.needTimer then -- 만약 leftTimeDelta가 0 이하일 경우 게임오버 처리를 시작합니다. if
self.leftTimeDelta <= 0 then self.needTimer = false self.leftTimeDelta = 0
_UIManageLogic:UpdateTimer(self.leftTimeDelta) self:NeedResult() else -- 아직 남은 시간이 존재할 경우 시간 감
소 및 UI 처리를 수행합니다. self.leftTimeDelta = self.leftTimeDelta - delta if self.leftTimeDelta <= 0 then
self.leftTimeDelta = 0 end _UIManageLogic:UpdateTimer(self.leftTimeDelta) end end end
```

```
end
```

> 해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

```
```lua
@Logic
script TimeRewardLogic extends Logic

property string storageKey = "LastLoginTime"

@Sync
property number offlineReward = 0

@Sync
property boolean offlineRewardComplete = false

@ExecSpace("ServerOnly")
method void SaveCurrentTime(string userId)
 local currentTime = os.time()
 local userDataStorage = _DataStorageService:GetUserDataStorage(userId)

 -- "LastLoginTime"이라는 키로 현재 os.time() 값을 저장합니다.
 userDataStorage:SetAsync(self.storageKey, tostring(currentTime),
function(errorCode, key)
 if errorCode == 0 then
 log(userId .. "의 접속 시간이 성공적으로 저장되었습니다: " .. currentTime)
 else
 log("저장 실패! 에러 코드: " .. errorCode)
 end
end)
end

@ExecSpace("ServerOnly")
method void LoadLastLoginTime(string userId)
```

```

local userDataStorage = _DataStorageService:GetUserDataStorage(userId)

userDataStorage:GetAsync(self.storageKey, function(errorCode, key, value)
if errorCode == 0 then
 -- 이전에 저장된 시간이 없다면 value는 nil일 수 있습니다.
 local lastTime = tonumber(value) or 0

 -- 앞서 만든 계산 로직을 호출하여 보상 수치를 구합니다.
 self.offlineReward = self:CalculateTimeReward(lastTime)
 -- 보상 수량을 완료했으니 현재 시간을 저장합니다.
 self:SaveCurrentTime(userId)
end
end)
end

@ExecSpace("ServerOnly")
method number CalculateTimeReward(number lastLoginTime)
-- 마지막 접속 시간과 현재 시간을 비교하여 필요한 보상을 계산하여 지급합니다.
self.offlineRewardComplete = true
if lastLoginTime == nil or lastLoginTime == 0 then
 return 1
end

local currentTime = os.time()
local timeDiff = currentTime - lastLoginTime

if timeDiff < 0 then
 timeDiff = 0
end

if timeDiff <= 1 then
 return 1
else
 -- 10초마다 SummonToken을 1씩 추가로 지급합니다.
 local returnValue = math.floor(timeDiff / 10)
 -- 최대 10개까지만 지급하기 위한 처리를 진행합니다.
 if returnValue >= 10 then
 returnValue = 10
 end
 return returnValue
end

end

@ExecSpace("ClientOnly")
method void OnSyncProperty(string name, any value)
-- offlineReward 값이 변경되었을 경우 SummonToken을 갱신합니다.
if name == "offlineReward" then
 _GameLogic:UpdateSummonToken(value)
end
end

end

```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

- 샘플 월드에서는 미접속 시간의 차이가 10초일 때 소환 토큰 1개씩을 지급하며, 최대 10개까지 지급합니다.
- **TimeRewardLogic**이 계산한 결과값을 **GameLogic**의 `summonToken`에 대입합니다.
- 플레이어는 `summonToken`의 여부에 따라 캐릭터를 소환할 수 있습니다.

## ⚠ 오프라인 보상 구현시 주의사항

- **보안 및 어뷰징**
  - 사용자가 디바이스의 시간을 설정하여 시간을 미래로 설정할 경우 그만큼의 시간이 흐른 것으로 간주하여 대량의 보상을 지급할 수 있습니다.
- **게임 경제 및 밸런스**
  - 오프라인 보상을 구현했으나 상한치인 캡(Max Cap)을 설정하지 않을 경우 게임의 경제가 크게 무너질 수 있습니다.
- **비율 및 소수점 처리**
  - 1분당 10골드를 지급한다고 가정 할 경우 30초만 지나고 접속했을 경우 기준을 어떻게 설정할 것 인지 문제가 발생합니다.
- **타임존 문제**
  - 이용자가 해외여행을 할 경우 서버에 저장된 시간대와 달라 보상을 더 크게 지급하거나 못받을 수 있습니다.
- **유저 경험 및 예외 처리**
  - 만약 인벤토리의 개수 제한이 있는 게임일 경우 오프라인 보상이 슬롯보다 많을 경우 보상의 유실이 발생할 수 있습니다.
  - 보상을 수령할 때 강제 종료 또는 네트워크 끊김으로 인해 보상 지급 도중 데이터가 날아갈 수 있음

## 해결 방안

- 위의 문제들을 해결하기 위한 방안으로 다음과 같은 방법들이 있습니다.
- **보안 및 어뷰징**
  - 값의 처리를 서버에서만 연산하도록 만들어 클라이언트가 개입할 상황을 막는 방법이 있습니다.
- **게임 경제 및 밸런스**
  - 보상의 상한선인 캡(Max Cap)을 만들어 사용하면 보상이 과도하게 지급되는걸 막을 수 있습니다.
- **비율 및 소수점 처리**
  - 이는 기획적 문제로 규칙을 설정한뒤 해당 규칙에 예외가 발생하지 않도록 조절해야 합니다.
- **타임존 문제**
  - 국제 표준시간(UTC)을 활용하여 이러한 문제를 예방할 수 있습니다.
- **유저 경험 및 예외처리**
  - 보상이 지급되기 전 충분한 여유 공간이 있는지 예외처리를 수행합니다.
  - 네트워크 통신이 필요한 기능일 경우 네트워크 통신에 실패했는지 확인하고 실패했다면 2~3회 반복 요청하여 통신 오류를 줄일 수 있습니다.

## 해당 영상 시청 후 수행해 볼 내용

- 플레이어에게 접속 시간의 차이에 따른 보상을 지급할 수 있도록 보상 재화를 설계 및 제작합니다.

- 시간에 따라 일정한 비율로 보상을 제공할 수 있도록 로직을 구성, 제작합니다.

## 최소 구현 기준

- 접속 시간의 편차에 따라 플레이어에게 소환 재화를 지급합니다.
- 플레이어에게 재화가 정상적으로 지급되며, 지급된 시점에 접속 시간을 저장합니다.

## 응용 및 확장 방향

- 플레이어가 사용 가능한 재화의 종류를 확장하고 오프라인 보상을 통해 획득할 수 있도록 기능을 추가합니다.
- 플레이어의 재화를 저장하고 불러올 수 있도록 제작하여 플레이어가 지속적으로 월드에 접근하도록 콘텐츠를 추가합니다.
- 오프라인 보상을 데이터 테이블을 활용하여 다양한 재화의 지급량을 조절할 수 있도록 구조를 변경합니다.

---

## [💡 기초 학습] UI 구성하기

---

### 영상 타임라인

- 강의 영상내 아래의 타임라인에서 학습할 수 있습니다.
- 맵 타입 이해하기: 01:33:06

### 학습 목표

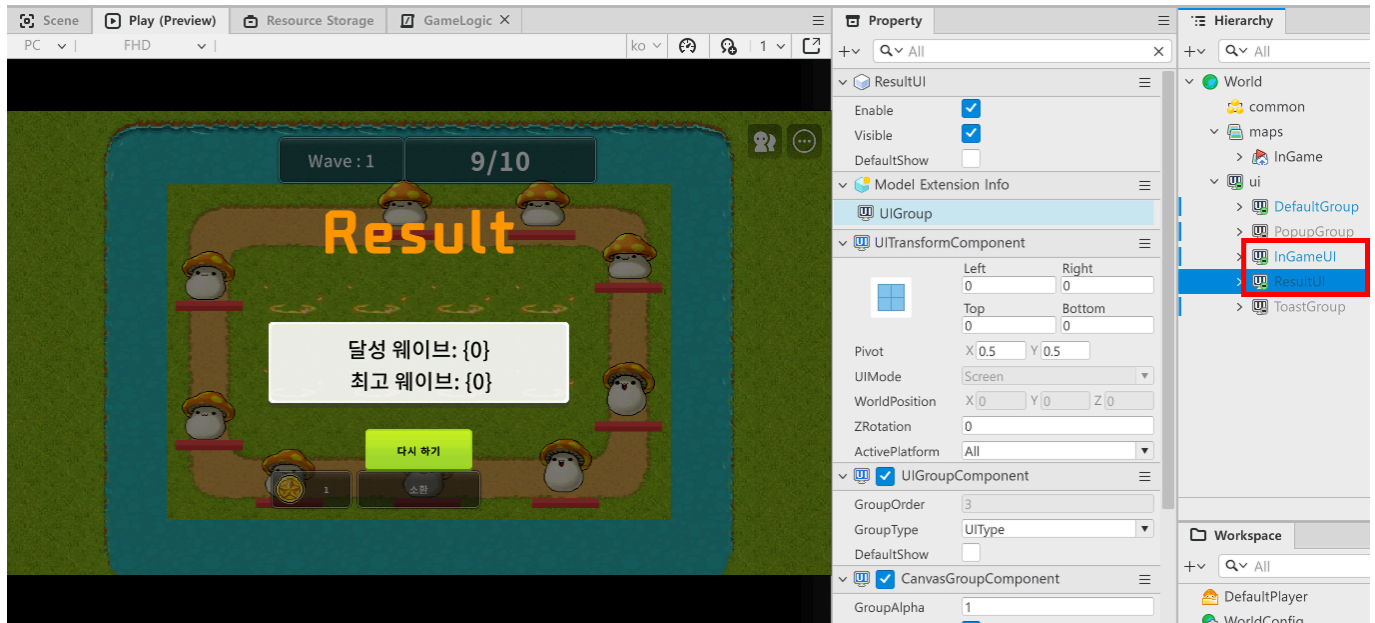
- UI를 구성하는 방법에 대해 학습합니다.
- 플레이어가 유닛을 소환하기 위한 재화의 보유량을 확인하기 위한 UI를 구성합니다.
- 결과 UI를 제작하는 방법을 학습합니다.
- 현재 웨이브와 관련된 정보 UI를 제작하는 방법에 대해 학습합니다.
- 남은 시간을 표기하는 타이머 UI를 구현하는 방법을 학습합니다.

### 해당 영상 시청 후 수행해 볼 내용

- 샘플 월드의 UI를 구성하여 플레이어에게 필요한 정보를 제공하도록 화면을 구성합니다.
- 샘플 월드에서 사용하는 UI 외에 다른 리소스를 사용하여 UI의 디자인을 변경합니다.

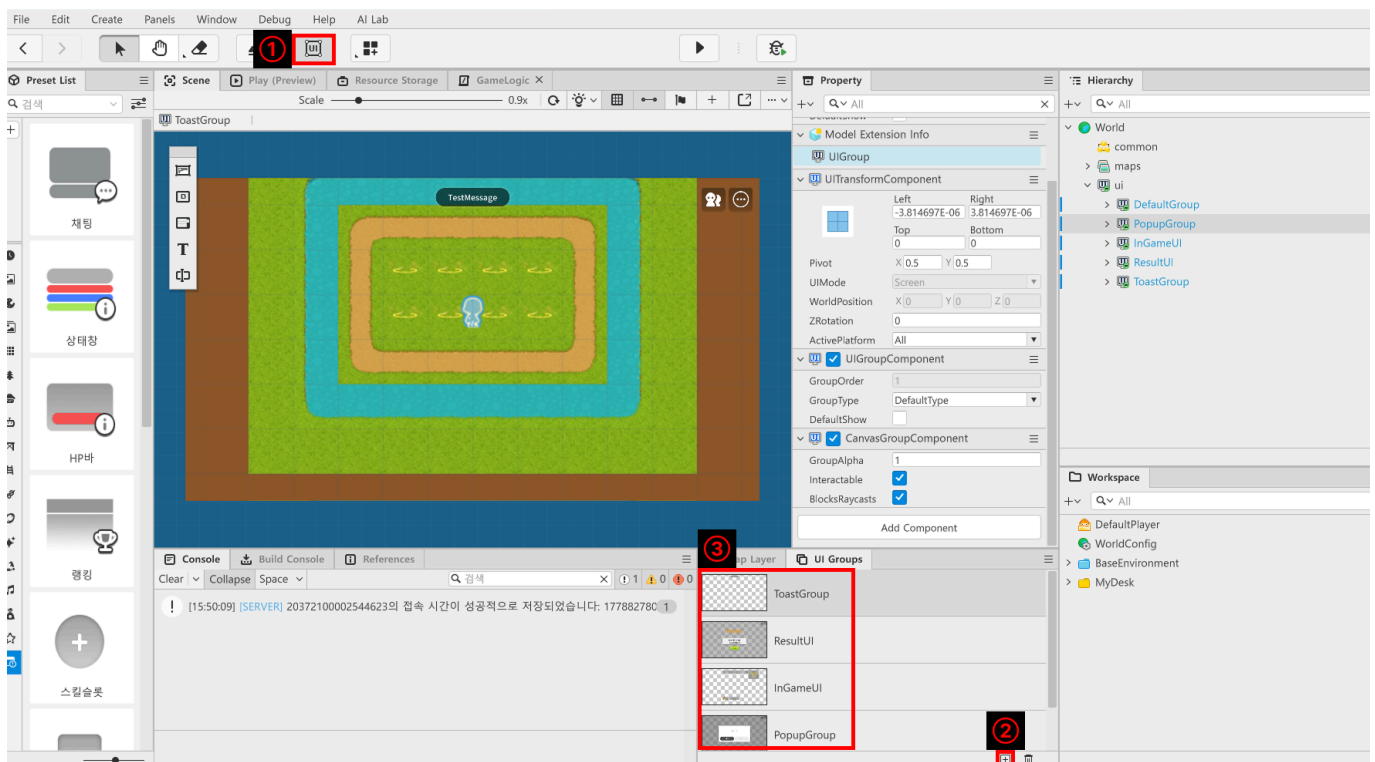
## UI 구성하기





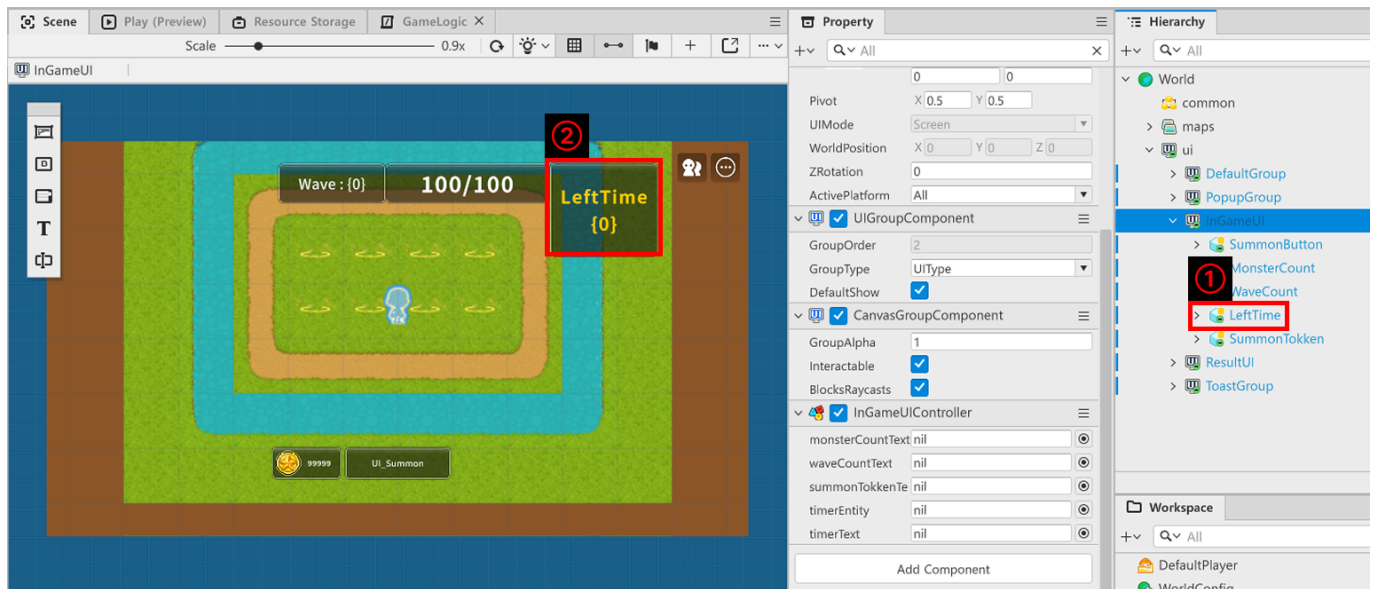
- UI를 구성하기 위해선 UI Group이 필요합니다.
- UI Group은 UI 엔티티를 담기 위한 단위이며 UI Group의 순서에 따라 화면에 출력되는 순서가 변경됩니다.
- 따라서 UI Group의 순서가 의도하는 순서대로 구성되어 있는지 확인이 필요합니다.

## UI Group 생성하기



- 메이플스토리 월드 에디터 상단의 UI 버튼을 클릭합니다.
- UI Groups 창이 + 버튼을 클릭합니다.
- 새롭게 생성된 UI Group은 UI Groups 창에 추가됩니다.

## Timer 생성하기



- 샘플 월드에서는 **InGameUI**내 LeftTime 엔티티가 남은 시간을 출력하는 기능을 수행합니다.
- LeftTime 엔티티는 남은 시간을 표기하는 타이머의 배경 이미지를 출력합니다.
- LeftTime의 자식 엔티티인 TimerText는 남은 시간을 출력합니다.
- 각 엔티티는 다음의 Component를 가지고 있습니다.
- **LeftTime**
  - **SpriteGUIRendererComponent**: 이미지 출력을 위한 Component입니다.
- **TimerText**
  - **TextGUIRendererComponent**: 남은 시간을 텍스트로 출력하기 위한 Component입니다.
- **InGameUIController**에 타이머를 작동하기 위한 스크립트를 작성합니다.
- 샘플 월드에서는 UpdateTimer를 통해 남은 시간을 표기하고 있습니다.
- 아래는 UpdateTimer 코드입니다.

```
@ExecSpace("ClientOnly")
method void UpdateTimer(number deltaTimer)
-- 전달받은 deltaTimer 값을 기준으로 남은 시간 UI를 갱신합니다.
if self.timerEntity.Enable == false then
 self.timerEntity.Enable = true
end

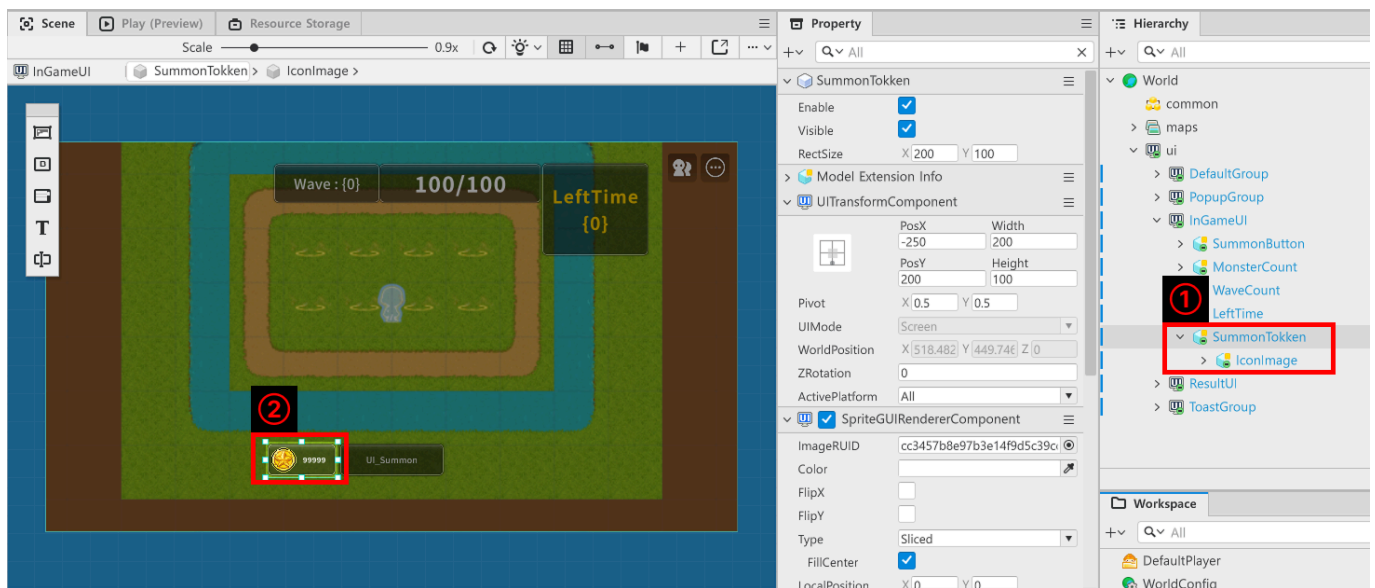
local timeString = string.format("%.2f", deltaTimer)

self.timerText.Text = "LeftTime\n"..timeString
end
```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

- string.format을 통해 남은 시간을 15.5와 같이 남은 시간을 표시합니다.
- GameLogic에서 남은 시간을 연산하면 해당 Method를 통해 UI를 갱신합니다.

## 재화 UI 구성하기



- 샘플 월드에서 InGameUI의 SummonToken이 소환 토큰의 개수를 출력하는 UI를 담당합니다.
- 소환 토큰 UI의 구성은 다음과 같습니다.
- **SummonToken**
  - 소환 토큰의 프레임을 출력하기 위한 엔티티입니다.

◦

@ExecSpace("ClientOnly") method void UpdateTimer(number deltaTimer) -- 전달받은 deltaTimer 값을 기준으로 남은 시간 UI를 갱신합니다. if self.timerEntity.Enable == false then self.timerEntity.Enable = true end

local timeString = string.format("%.2f", deltaTimer)

self.timerText.Text = "LeftTime\n"..timeString end

> 해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

<br>

- string.format을 통해 남은 시간을 15.5와 같이 남은 시간을 표시합니다.
- GameLogic에서 남은 시간을 연산하면 해당 Method를 통해 UI를 갱신합니다.

<br>

### 재화 UI 구성하기

<p align="center">



</p>

- 샘플 월드에서 InGameUI의 SummonToken이 소환 토큰의 개수를 출력하는 UI를 담당합니다.
- 소환 토큰 UI의 구성은 다음과 같습니다.
- **\*\*SummonToken\*\***
  - 소환 토큰의 프레임을 출력하기 위한 엔티티입니다.
  - `SpriteGUIRendererComponent`를 보유하고 있으며 프레임을 출력합니다.
- **\*\*IconImage\*\***
  - 소환 토큰의 아이콘을 출력하기 위한 엔티티입니다.
  - **\*\*SummonToken\*\***의 자식 엔티티입니다.
  - 소환 토큰 아이콘을 출력하기 위해 `SpriteGUIRendererComponent`를 가지고 있습니다.
- **\*\*SummonTokenText\*\***
  - 소환 토큰의 개수를 출력하기 위한 엔티티입니다.
  - **\*\*IconImage\*\***의 자식 엔티티입니다.
  - 소환 토큰의 개수를 출력하기 위해 `TextGUIRendererComponent`를 가지고 있습니다.
- `InGameUIController`의 UpdateSummonTokenUI를 통해 남은 소환 토큰의 개수를 출력합니다.
- 아래는 UpdateSummonTokenUI의 코드입니다.

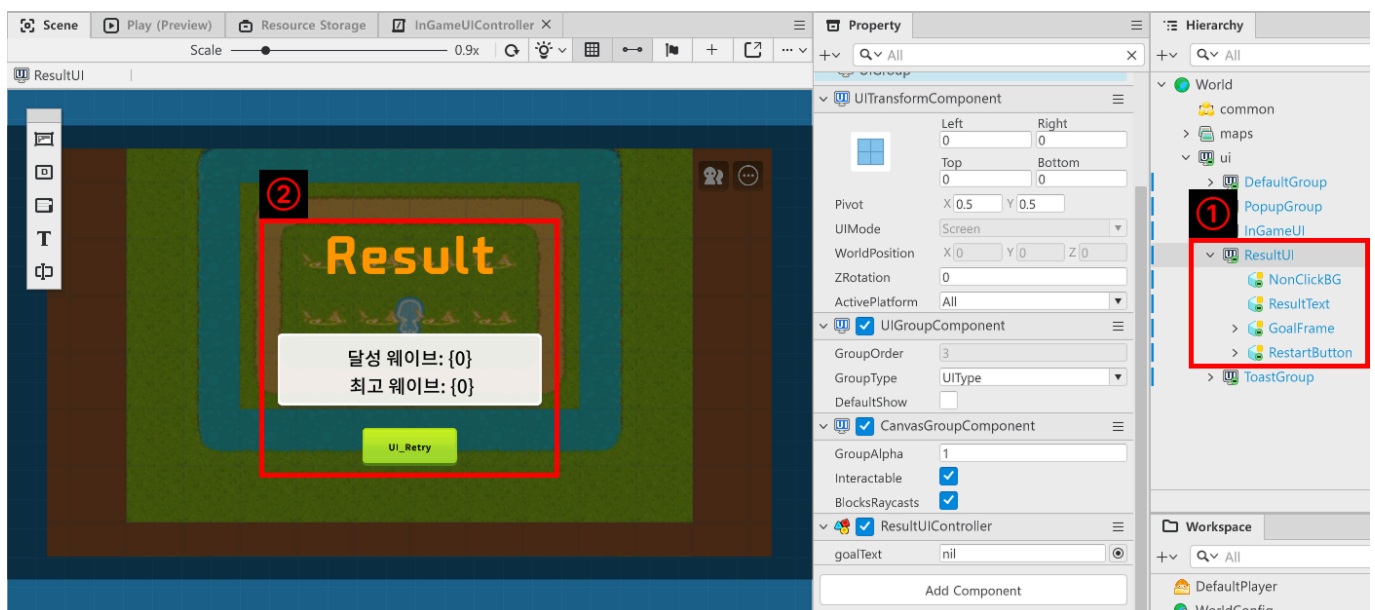
```

` ``lua
@ExecSpace("ClientOnly")
method void UpdateSummonTokenUI(number TokenValue)
-- summonTokenText가 nil일 경우 초기화를 시도합니다.
if not invalid(self.summonTokenText) then
 self:InitSummonToken()
end
-- 전달받은 TokenValue를 기준으로 남은 토큰 수 UI를 갱신합니다.
self.summonTokenText.Text = tostring(math.floor(TokenValue))
end

```

해당 코드는 Plain Text로 작성된 코드입니다.

## 결과 UI 구성하기



- 샘플 월드에서는 **ResultUI**를 통해 결과 UI를 출력합니다.
- **ResultUI**는 다음의 구성으로 이뤄져 있습니다.

- **NonClickBG**

- 다른 UI를 시각적으로 가리기 위해 제작된 엔티티입니다.

- 

```
@ExecSpace("ClientOnly") method void UpdateSummonTokenUI(number TokenValue) --
 summonTokenText가 nil일 경우 초기화를 시도합니다. if not isvalid(self.summonTokenText) then
 self:InitSummonToken() end -- 전달받은 TokenValue를 기준으로 남은 토큰 수 UI를 갱신합니다.
 self.summonTokenText.Text = tostring(math.floor(TokenValue)) end
```

> 해당 코드는 Plain Text로 작성된 코드입니다.

<br>

### 결과 UI 구성하기

```
<p align="center">

</p>
```

- 샘플 월드에서는 **ResultUI**를 통해 결과 UI를 출력합니다.
- **ResultUI**는 다음의 구성으로 이뤄져 있습니다.
- **NonClickBG**
  - 다른 UI를 시각적으로 가리기 위해 제작된 엔티티입니다.
  - `SpriteGUIRendererComponent`를 보유하고 있습니다.
- **ResultText**
  - 결과 UI임을 표기하기 위한 텍스트 엔티티입니다.
  - `TextGUIRendererComponent`를 가지고 있습니다.
- **GoalFrame**
  - 결과를 출력하기 위한 프레임 엔티티입니다.
  - `SpriteGUIRendererComponent`를 가지고 있습니다.
- **GoalText**
  - 결과를 텍스트로 출력하기 위한 엔티티입니다.
  - `TextGUIRendererComponent`를 가지고 있습니다.
- **RestartButton**
  - 다시 시작하기 기능을 실행하는 버튼 엔티티입니다.
  - `SpriteRendererComponent`와 `ButtonComponent`를 가지고 있습니다.
- **ButtonText**
  - 다시 시작하기 버튼의 이름을 출력하기 위한 엔티티입니다.
  - `TextGUIRendererComponent`를 가지고 있습니다.
- 해당 UI의 구성을 사용하기 위해 **ResultUI**는 **ResultUIController**를 가지고 있습니다.
- 아래는 **ResultUIController**의 코드입니다.

```
```lua
@Component
script ResultUIController extends Component

property TextGUIRendererComponent goalText = nil
```

```

@ExecSpace("ClientOnly")
method void OnBeginPlay()
self:InitGoalText()
end

@ExecSpace("ClientOnly")
method void InitGoalText()
-- goalText가 nil인지 검사합니다.
if not isvalid(self.goalText) then
-- GoalText 엔티티를 찾아 등록합니다.
local findEntity =
_EntityService:GetEntityByPath("/ui/ResultUI/GoalFrame/GoalText")
-- findEntity가 정상적으로 등록되었는지 확인합니다.
if isvalid(findEntity) then
-- findEntity의 TextGUIRendererComponent를 등록합니다.
self.goalText = findEntity.TextGUIRendererComponent
end
end
end

@ExecSpace("ClientOnly")
method void UpdateResultText(number AchieveGoalCount, number BestGoalCount)
-- 결과 텍스트를 전달받은 값으로 UI 갱신을 시도합니다.
self.goalText.Text = _LocalizationService:GetTextFormat("UI_ResultWave",
AchieveGoalCount, BestGoalCount)
end

@ExecSpace("ClientOnly")
method void Show()
self.Entity.Enable = true
end

@ExecSpace("ClientOnly")
method void Hide()
self.Entity.Enable = false
end

@EventSender("Entity", "336d7e78-d49b-4d6e-8f66-4cec32b01c85")
handler HandleButtonClickEvent(ButtonClickEvent event)
-- 다시하기 버튼을 클릭할 경우 재시작을 GameLogic에게 요청합니다.
self:Hide()
_GameLogic:RestartGame()
end

end

```

해당 코드는 Plain Text를 기준으로 작성된 코드입니다.

UI 관리를 위한 Logic 제작하기

- Component가 UI에 접근하기 위해 매번 UI 엔티티를 찾기엔 번거롭습니다.
- 따라서 UI를 관리하는 Component들을 하나의 Logic에 저장한 뒤, 해당 Logic을 통해 필요한 기능을 요청하거나 UI를 켜고 끌 수 있습니다.
- 샘플 월드에서는 **UIManageLogic**을 제작하여 사용하고 있습니다.

최소 구현 기준

- 게임의 결과를 알 수 있는 UI가 결과와 함께 출력되어야 합니다.
- 플레이어의 소환 토큰의 개수를 확인할 수 있도록 UI가 구성되어야 합니다.
- 웨이브의 몬스터 소환이 마무리되었다면 남은 시간을 표기하도록 기능이 구성되어야 합니다.
- 아래는 UIManageLogic의 코드입니다.

```
@Logic
script UIManageLogic extends Logic

property InGameUIController ingameUIController = nil

property ResultUIController resultUIController = nil

@ExecSpace("ClientOnly")
method void OnBeginPlay()
self:InitIngameUIController()
self:InitResultUIController()
end

@ExecSpace("ClientOnly")
method void InitIngameUIController()
-- UI 그룹에서 InGameUI를 찾아 Component를 등록합니다.
if not isvalid(self.ingameUIController) then
    local findUI = _EntityService:GetEntityByPath("/ui/InGameUI")
    if isvalid(findUI) then
        self.ingameUIController = findUI.InGameUIController
    end
end
end

@ExecSpace("ClientOnly")
method void UpdateMonsterCount(number CurrentMonsterCount, number MaxMonsterCount)
-- ingameUIController가 nil일 경우 초기화를 요청합니다.
if not isvalid(self.ingameUIController) then
    self:InitIngameUIController()
end
-- 전달받은 값을 기준으로 현재 몬스터의 현황 UI를 업데이트합니다.
self.ingameUIController:CountLeftMonster(CurrentMonsterCount, MaxMonsterCount)
end

@ExecSpace("ClientOnly")
method void UpdateWaveCount(number WaveCount)
-- ingameUIController가 nil일 경우 초기화를 요청합니다.
if not isvalid(self.ingameUIController) then
```



```

        self:InitIngameUIController()
    end
    -- 현재 Wave를 갱신하도록 요청합니다.
    self.ingameUIController:UpdateWaveCount(WaveCount)
end

@ExecSpace("ClientOnly")
method void DisableTimer()
    -- TimerUI를 가리도록 요청합니다.
    self.ingameUIController:DisableTimer()
end

@ExecSpace("ClientOnly")
method void InitResultUIController()
    -- resultUIController가 nil일 경우 ui에서 ResultUI를 찾아 등록합니다.
    if not isvalid(self.resultUIController) then
        local findEntity = _EntityService:GetEntityByPath("/ui/ResultUI")
        self.resultUIController = findEntity.ResultUIController
    end
end

@ExecSpace("ClientOnly")
method void ShowResultUI(number CurrentWave, number BestWave)
    -- 여기서 현재 전달받은 WaveCount와 스토리지에서 동기화된 최고 Wave 값을 함께 넘겨줍니다.
    self.resultUIController:UpdateResultText(CurrentWave, BestWave)
    self.resultUIController:Show()
end

@ExecSpace("ClientOnly")
method void UpdateSummonToken(number TokenValue)
    -- 소환 토큰 UI를 업데이트합니다.
    self.ingameUIController:UpdateSummonTokenUI(TokenValue)
end

@ExecSpace("ClientOnly")
method void UpdateTimer(number TimerValue)
    -- Timer를 업데이트 하도록 요청합니다.
    self.ingameUIController:UpdateTimer(TimerValue)
end

end

```

해당 코드는 Plain Text로 작성된 코드입니다.

응용 및 확장 방향

- 오프라인 보상과 함께 추가된 재화가 있다면 이를 표기하는 UI를 제작합니다.
- 캐릭터의 정보를 출력할 수 있는 UI를 구성하고, 캐릭터의 정보를 출력할 수 있도록 기능을 구현합니다.